MES AI User Guide

Release 1.9.1

Introduction 2
Installation 3
- GitHub Repository 3
- Database 3
  - PostgreSQL 3
  - MSSQL or Oracle 4
  - Database Schema 4
- Python and Project Dependencies 4
  - Windows 5
  - Linux or macOS 5

MES Server 6
- Windows Shell 6
- Windows Service 6
- Linux or macOS Shell 7
- Linux or macOS Daemon 7

Client Applications 8
- Windows Shell 8
- Windows Service 9
- Linux or macOS Shell 9
- Linux or macOS Daemon 10
- Client Startup 10
- First Run 10
- Demonstration Applications 10
- Authentication 11
  - Local 11
  - OIDC OpenID Connect / Enterprise SSO 11
  - Settings Page Configuration 11

Design Time Client (dt-client) 12
- Work Cells, Equipment & Equipment Classes 13
- Routes and Dispositions 16
  - Routes 16
  - Dispositions 27
- Products 28
- Materials 30
- Storage Locations 31
- Units of Measure 31
- Data Definitions 32
- Reason Codes 33
- Users and Roles 33
- Work Schedule 36
- Plugins 38
  - Enterprise Plugins 40
  - Factory Floor Plugins 42
  - Native SDK Bridge Sidecar Plugin 43
- Import/Export 46
  - Export 46
  - Import 47
- Settings 47

Runtime Client (rt-client) 47
- Dashboard 48
- Dispatch 48

Execution 50
Status Signals 50
WIP 50
Orders 54
Inventory 55
Equipment 56
Live Events 58
WIP Client (wip-client) 59
WIP 59
Consumption 61
Data Collection 62
Step Parameters 63
Inventory Client (inv-client) 64
Operations 64
Receive Lot 65
Receive Unit 69
Transfer 69
Adjust 70
Ship 71
Transaction Log 72
ERP Simulator (erp-sim) 72
Inbound 73
Production Orders 73
Materials 74
Products 75
Outbound 76
Report Completion 76
Report Consumption 77
Report Scrap 77
Confirmations 78
Equipment Simulator (equipment-sim) 78
Dashboard 78
Equipment 79
State History 80
OEE Analysis 81
Auto-Simulator 81
State Models 82
Software Quality Assurance 84
REST API Unit Tests 84
User Interface Tests 84
Documentation 84
REST API 84
Architecture 85
Enhancements for Forward Deployed Engineers 85

Introduction

MES AI is an open-source Manufacturing Execution System platform built around a FastAPI server, multiple browser clients, and a plugin-oriented extension model. The repository includes installation and startup scripts for local development on Windows,

Linux, and macOS.

This guide covers:

- cloning the GitHub repository
- installing and preparing the database
- installing Python and project dependencies with the provided install scripts
- starting the MES AI server
- starting the available client applications

Installation

GitHub Repository

Open a terminal in the directory where you want the project to live, then clone the repository.

```
git clone https://github.com/point85/mes-ai.git
cd mes-ai
```

If you prefer SSH and have GitHub SSH access configured:

```
git clone git@github.com:point85/mes-ai.git
cd mes-ai
```

Or, the zip archive can be downloaded from the GitHub web URL: point85/mes-ai: Manufacturing Execution System powered by AI.

Database

MES AI supports PostgreSQL, Microsoft SQL Server, and Oracle.

- PostgreSQL can be installed with the provided installation script.
- MSSQL and Oracle are supported by the main installer and server launcher, but the database itself must already exist before the main installer runs.

PostgreSQL

Use the PostgreSQL helper script:
.\install-postgresql.ps1 on Windows or .sh on Linux.

After installation:

1. Start the PostgreSQL service if it is not already running.
2. Set the postgres user password (postgres is the default)
3. Run the main installer.

Example password commands:
Windows: psql -U postgres -c "ALTER USER postgres PASSWORD 'your_password';"
Linux: sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'your_password';"
MacOS: psql postgres -c "ALTER USER postgres PASSWORD 'your_password';"

MSSQL or Oracle

If you are using SQL Server or Oracle instead of PostgreSQL:

1. Install and configure the database server separately.
2. Create the target database before running MES AI migrations.
3. Make sure the required database drivers are available.

Driver notes:

- MSSQL requires a usable SQL Server ODBC driver.
- Oracle requires the Python `oracledb package, and may also need Oracle client libraries depending on your connection mode.

Database Schema

Database table names follow the ISA-S95 specification. There are no views or indexes created by the SQL Alchemy code. SQL Alchemy data types are used for portability. Alembic is the migration mechanism.

Python and Project Dependencies

The main install scripts create a local virtual environment, install Python dependencies, write server/.env, and run Alembic migrations.

Important behavior:

- DbProvider is required. There is no default database provider.
- DbName defaults to mes_ai.
- If you supply a full database URL, the installer uses it as-is.
- Automatic database creation is implemented only for PostgreSQL.
- If you do not supply any arguments to the install script, it will print usage/help

Windows

Use PowerShell:
.\install.ps1 -DbProvider PostgreSQL -DbPassword your_password

Examples:
.\install.ps1 -DbProvider PostgreSQL
.\install.ps1 -DbProvider PostgreSQL -DbName mes_ai -DbPassword your_password
.\install.ps1 -DbProvider MSSQL -DbServer sql-host:1433 -DbUser sa -DbPassword secret
.\install.ps1 -DbProvider Oracle -DatabaseUrl oracle+oracledb://mes:secret@oracle-host:1521/mes_ai

Linux or macOS

Make the script executable, then run it:

chmod +x ./install.sh
./install.sh --db-provider PostgreSQL --db-password your_password

Examples:
./install.sh --db-provider PostgreSQL
./install.sh --db-provider PostgreSQL --db-name mes_ai --db-password your_password
./install.sh --db-provider MSSQL --db-server sql-host:1433 --db-user sa --db-password secret
./install.sh --db-provider Oracle --database-url oracle+oracledb://mes:secret@oracle-host:1521/mes_ai

What the Installer Does

The main install script performs these high-level steps:

1. Verifies Python 3.12+.
2. Verifies Node.js 20+ unless client installation is skipped.
3. Creates the local .venv virtual environment.
4. Installs server dependencies.

5. Creates server/.env if it does not already exist.
6. Creates the database automatically only when using PostgreSQL without a full database URL override.
7. Runs Alembic migrations.
8. Installs npm dependencies for the browser clients unless skipped.

MES Server

The server startup scripts set the database connection string, apply migrations, and start the FastAPI application with Uvicorn. If not arguments are supplied, the script will print usage/help. There are two sets of scripts, one targeted for fast turnaround during development with console output and one for production as a Windows service or Unix daemon. The development script is named run-server and the production one run-service.

Windows Shell

Use:
.\run-server.ps1 <Database> [DbName] [options]

Examples:
.\run-server.ps1 PostgreSQL
.\run-server.ps1 PostgreSQL mes_ai -Username postgres -Password (ConvertTo-SecureString 'your_password' -AsPlainText -Force)
.\run-server.ps1 MSSQL mes_ai -Username sa -Password (ConvertTo-SecureString 'secret' -AsPlainText -Force)
.\run-server.ps1 Oracle mes_ai -DbServer oracle-host:1521 -Username mes -Password (ConvertTo-SecureString 'secret' -AsPlainText -Force)

Windows Service

The service name, description and arguments are set in the configuration file mes-service.conf. Edit this file for your specific service. To install the service and run database migrations, use an admin Powershell:

.\run-service.ps1 install -ConfigFile C:\dev\mes_ai\mes-service.conf -RunMigrations

The log files are in ~\server\logs folder.

Additional commands are:
Start now : .\run-service.ps1 start -ConfigFile "C:\dev\mes_ai\mes-service.conf"

```
Stop        : .\run-service.ps1 stop   -ConfigFile "C:\dev\mes_ai\mes-service.conf"
Status      : .\run-service.ps1 status -ConfigFile "C:\dev\mes_ai\mes-service.conf"
Uninstall   : .\run-service.ps1 uninstall -ConfigFile "C:\dev\mes_ai\mes-service.conf"
```

Linux or macOS Shell

Use:
./run-server.sh <Database> [DbName] [options]

Examples:
./run-server.sh PostgreSQL
./run-server.sh PostgreSQL mes_ai -u postgres -p your_password
./run-server.sh MSSQL mes_ai -u sa -p secret
./run-server.sh Oracle mes_ai -s oracle-host:1521 -u mes -p secret

Supported database names for the launcher scripts:

- PostgreSQL
- MSSQL
- Oracle

Default server port:

- MES AI server runs on http://localhost:8082 unless overridden.

Linux or macOS Daemon

The daemon name, description and arguments are set in the configuration file mes-service.conf. Edit this file for your specific service. First install the daemon and run database migrations. The commands are:

```
sudo ./run-service.sh install --config /etc/mes_ai/mes-service.conf --run-migrations
sudo ./run-service.sh start
sudo ./run-service.sh stop
sudo ./run-service.sh restart
./run-service.sh status
sudo ./run-service.sh uninstall
```

After installation, useful systemd commands are:

```
journalctl -u mes-ai -f          # follow logs (live)
systemctl status mes-ai          # detailed status
systemctl cat mes-ai             # view unit file
systemctl reload-or-restart mes-ai
```

The log files are in ~\server\logs folder.

Client Applications

MES AI includes five browser-based clients:

- dt-client for the Design-Time client on default port 5173
- rt-client for the Run-Time client on default port 5176
- wip-client for the WIP operator client on default port 5177
- inv-client for the inventory operator client on default port 5178
- erp-sim for the ERP Simulator on default port 5174
- equipment-sim for the Equipment Simulator on default port 5175

These applications will run on Edge, Chrome, Safari and Firefox as Progressive Web Applications (PWA).

Native-like experience – Dedicated app window (no address bar/tabs)
Offline support – Service worker caches app shell and essential resources
App icon – Branded icon on home screen/dock with app name
Adaptive icon (Android) – Maskable icon automatically resizes for device design language
Auto-updates – Service worker checks for updates on each launch (autoUpdate strategy)
Standalone mode – Full-screen display (no browser chrome)

To install, open any MES AI app in a Chrome, Edge or Firefox browser (e.g., http://localhost:5173 during development or your production URL). Then, click on the install icon in the brower's address bar. Select installation options in the dialog to launch the dialog in a dedicated window.

Safari doesn't support direct PWA installation, but can add to Dock as a web clip. Click File → Add to Dock (or use Share menu). The app opens in full-screen mode when launched from Dock.

The scripts with "production" in their names (e.g. run-client-production.sh) execute from a npm build in the client's "dist" folder. The other client scripts execute from the source for development purposes. The following usages show development scripts, but the production ones have the same arguments.

Windows Shell

Use:

.\run-client.ps1 <Client> [options]

Examples:
.\run-client.ps1 dt-client
.\run-client.ps1 rt-client
.\run-client.ps1 wip-client -WIP lot
.\run-client.ps1 inv-client
.\run-client.ps1 erp-sim
.\run-client.ps1 equipment-sim
.\run-client.ps1 rt-client -ServerUrl http://localhost:8082
.\run-client.ps1 equipment-sim -Port 5200

Windows Service

USAGE
.\run-client-service.ps1 <action> [-ConfigFile <path>]

ACTIONS
install Register the Vite client as a Windows service (requires Admin)
uninstall Remove the Windows service (requires Admin)
start Start the service (requires Admin)
stop Stop the service (requires Admin)
restart Restart the service (requires Admin)
status Show current service status

OPTIONS
-ConfigFile <path> Path to config file (e.g.: rt-service.conf)

EXAMPLES
.\run-client-service.ps1 install
.\run-client-service.ps1 install -ConfigFile .\dt-service.conf
.\run-client-service.ps1 start
.\run-client-service.ps1 status
.\run-client-service.ps1 uninstall

See client-service.conf for all available configuration keys.

Linux or macOS Shell

Use:
./run-client.sh <Client> [options]

Examples:
./run-client.sh dt-client
./run-client.sh rt-client
./run-client.sh wip-client
./run-client.sh inv-client
./run-client.sh erp-sim
./run-client.sh equipment-sim
./run-client.sh rt-client --server-url http://localhost:8082
./run-client.sh equipment-sim --port 5200

Linux or macOS Daemon

The actions are the same as for the Windows service:
sudo ./run-client-service.sh <action> [-ConfigFile <path>]

Client Startup

- The client scripts install npm dependencies automatically if node_modules is missing.
- The clients are Vite development servers intended for local development and testing.
- The default backend URL used by the clients is http://localhost:8082.

First Run

For a typical PostgreSQL-based local setup:

1. Clone the repository.
2. Install PostgreSQL with install-postgresql.ps1 or install-postgresql.sh.
3. Run the main install script with DbProvider set to PostgreSQL.
4. Start the server with run-server.ps1 or run-server.sh.
5. Start dt-client and rt-client or wip-client or inv-client with run-client.ps1 or run-client.sh.

Demonstration Applications

To illustrate the capabilities of MES AI in three different industries, database seeding links have been implemented in dt-client:

- CPG Demo: creates an orange juice bottling line for a Consumer Packaged Goods industry. Juice is tracked by lots.
- Electronics Demo: creates a SMT (Surface Mount Technology) production line for electronic circuit boards for the discrete assembly industry. Boards are tracked by units.
- Pharmaceutical Demo: creates an ibuprofen pill manufacturing line. The blister

packs are traced by lots.

Screenshots in this document have been captured from these apps.

Authentication

The authentication level (none, local or OIDC) is a setting in dt-client. "None" is the default and requires no credentials. The local and OIDC setting applies only to dt-client.

If changes are made and a user can't log in, run the script reset-auth.ps1/sh to set the authentication mode back to none.

Local

For local authentication, users log in with credentials stored in the MES database. Passwords are hashed with bcrypt; the server never stores plain text credentials. JWT (JSON Web Token) access token settings are also configured in dt-client.

OIDC OpenID Connect / Enterprise SSO

Authentication is delegated entirely to an external Identity Provider (IdP) such as Keycloak, Okta, Azure AD, or Auth0.

Settings Page Configuration

The relevant settings are grouped into two sections:

Authentication section

Setting	Description
Authentication Mode	Dropdown: none / local / oidc
JWT Secret Key	Masked; signs all JWTs — must be changed from default in production
JWT Algorithm	Dropdown: HS256, HS384, HS512, RS256
Access Token Expiry	Minutes until access token expires (default 15)
Refresh Token Expiry	Days until refresh token expires (default 7)

OIDC Settings section

Setting	Description
OIDC Issuer URL	Provider discovery URL (e.g. https://auth.company.com/realms/mes)
OIDC Client ID	Client ID registered with the IdP

OIDC Client Secret	Masked; write-only after first save
OIDC Scopes	Comma-separated scopes (default openid,profile,email)
OIDC Role Claim	JWT claim carrying the user's roles/groups (default groups)
OIDC Redirect URI	OAuth callback URL registered with the provider

Design Time Client (dt-client)

dt-client is the design-time application for configuring products, routes, dispositions, equipment, plugins, and other master data. The main editors are:

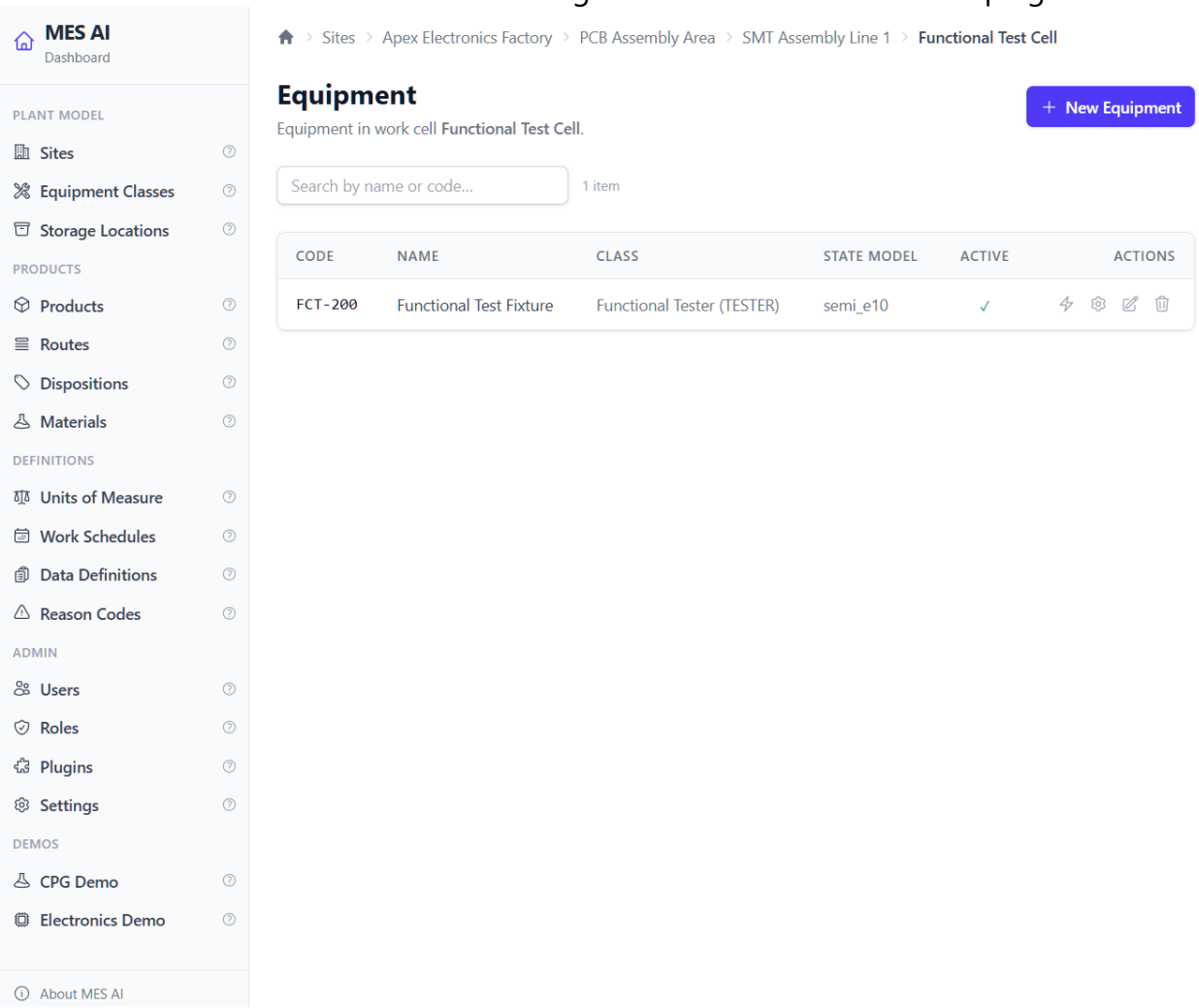
- ISA-S95 sites hierarchy to define organize equipment and define equipment properties
- equipment class to define properties of a type of equipment to be used in dispatching WIP to a piece of equipment
- route editor to configure product routing.

Supporting editors include:

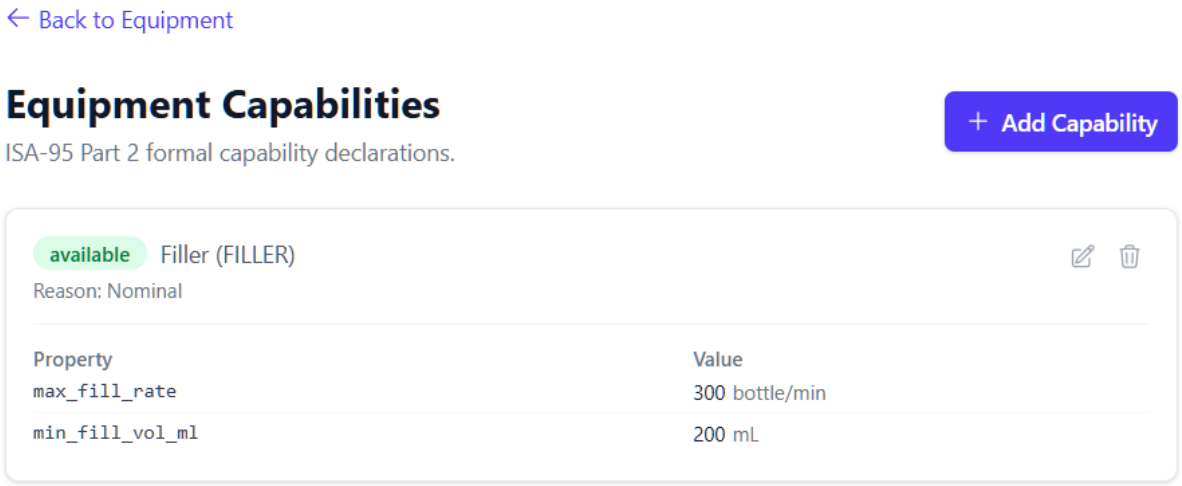
- storage locations to define raw and in-process locations where materials are consumed from or produced to
- products to define properties of discrete (unit-tracked) or process (lot-tracked) products produced from a route
- materials to define raw, intermediate or finished materials consumed or produced
- dispositions to control product routing, or hold scrap or release from hold reasons
- units of measure to define the measurement units for lots or units and associated conversions
- work schedules to define working and non-working periods for shifts and teams working them
- data definitions to define data collected by an in-process lot or unit that can be used for quality analytics or other purposes
- reason code hierarchy to define loss and downtime reason codes for OEE availability tracking
- users and roles to define users and their groups used for local log in and team membership
- plugins to install, uninstall, enable and disable plugins that are considered to be outside of core MES functionality, but commonly needed in real world applications.
- settings to define property values that control how the clients behave

Work Cells, Equipment & Equipment Classes

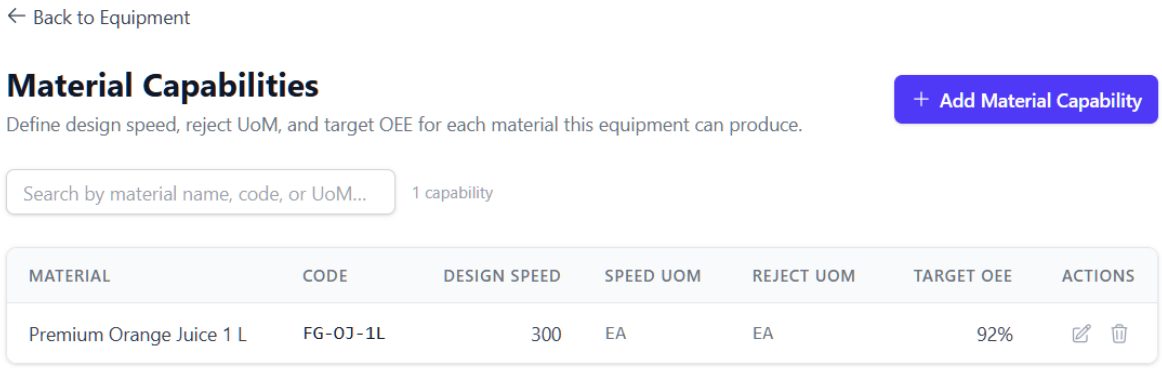
The screenshot below shows the Functional Test Fixture equipment in the Functional Tester class from the SMT demo. It is using the Semi E10 state machine plugin.



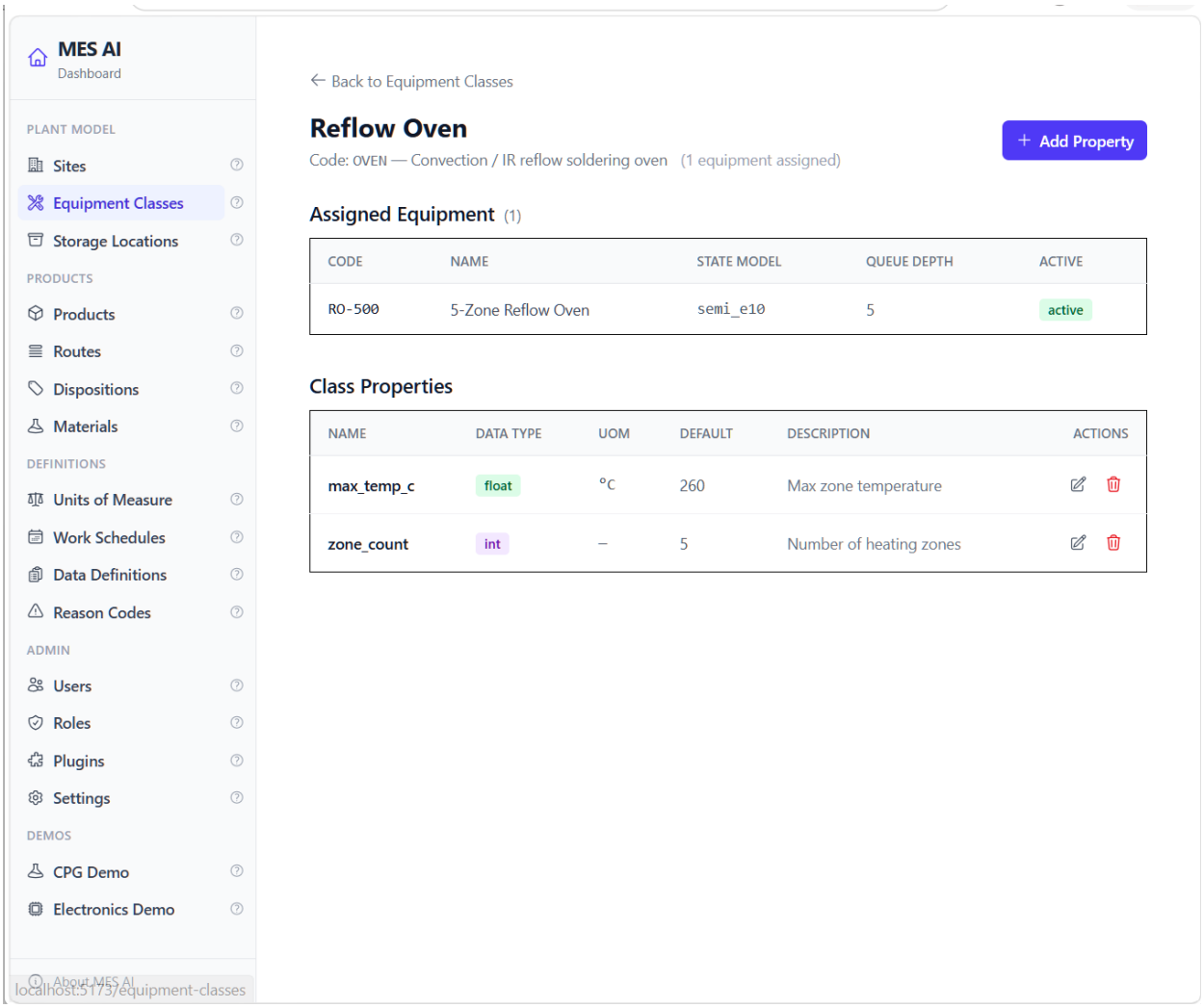
Equipment can have capabilities that are used in the WIP dispatching algorithm:



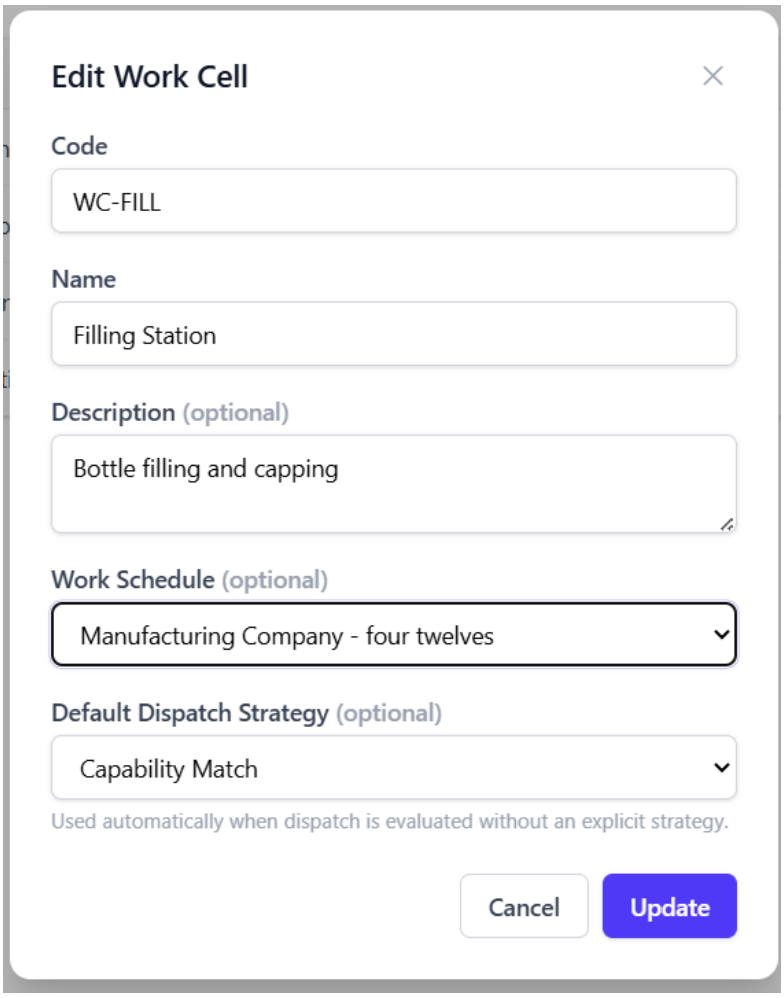
It can also have material (WIP) production capabilities used in calculating the performance component of OEE:



This is a screenshot of the Reflow Oven class editor with two class properties



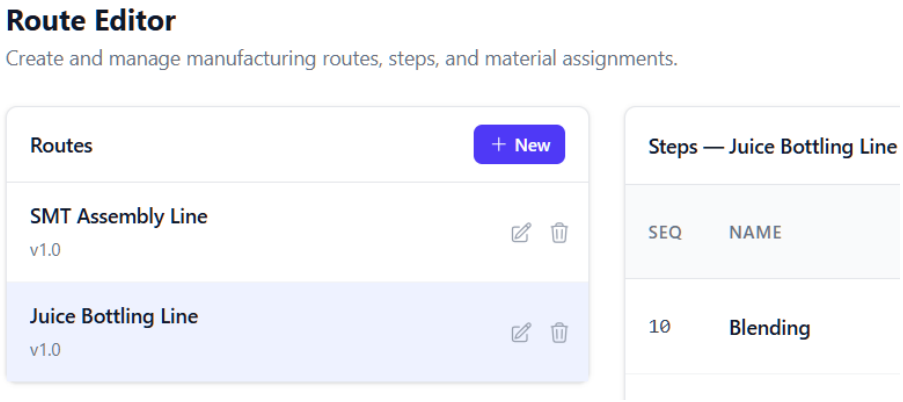
Sites, areas, production lines and work cells have some common properties like work schedule. Additionally a work cell can define the WIP dispatching strategy for all equipment that belongs to it. In this example dispatching is by capability match:



Routes and Dispositions

Routes

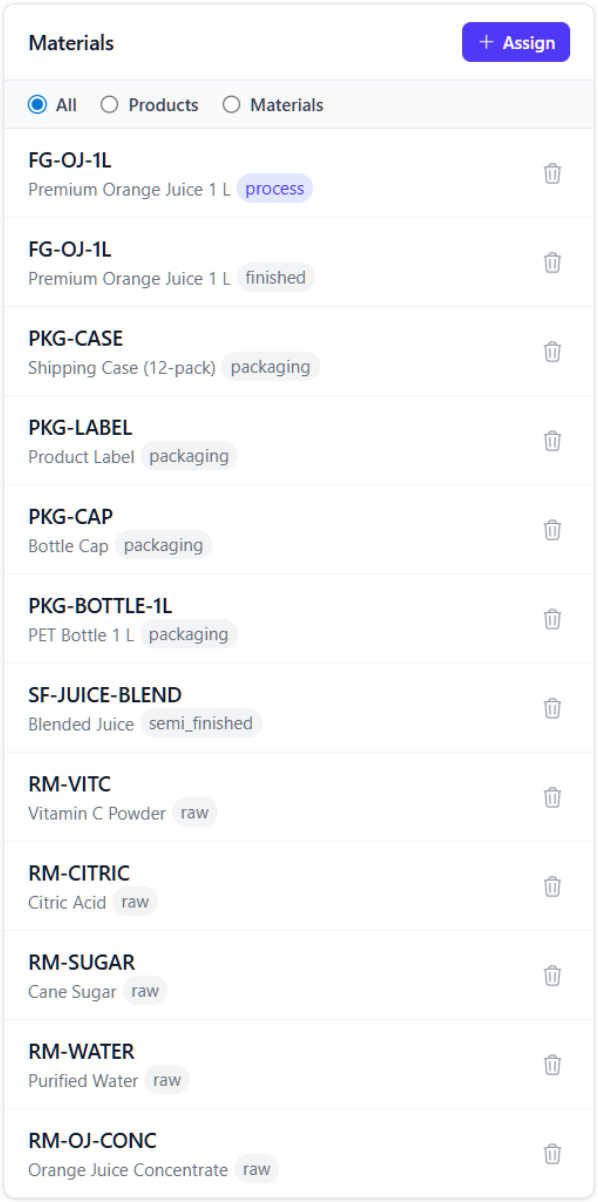
The screenshot below shows the route editor for the SMT Assembly Line. This editor is used to create and manage manufacturing routes, steps, and material assignments.



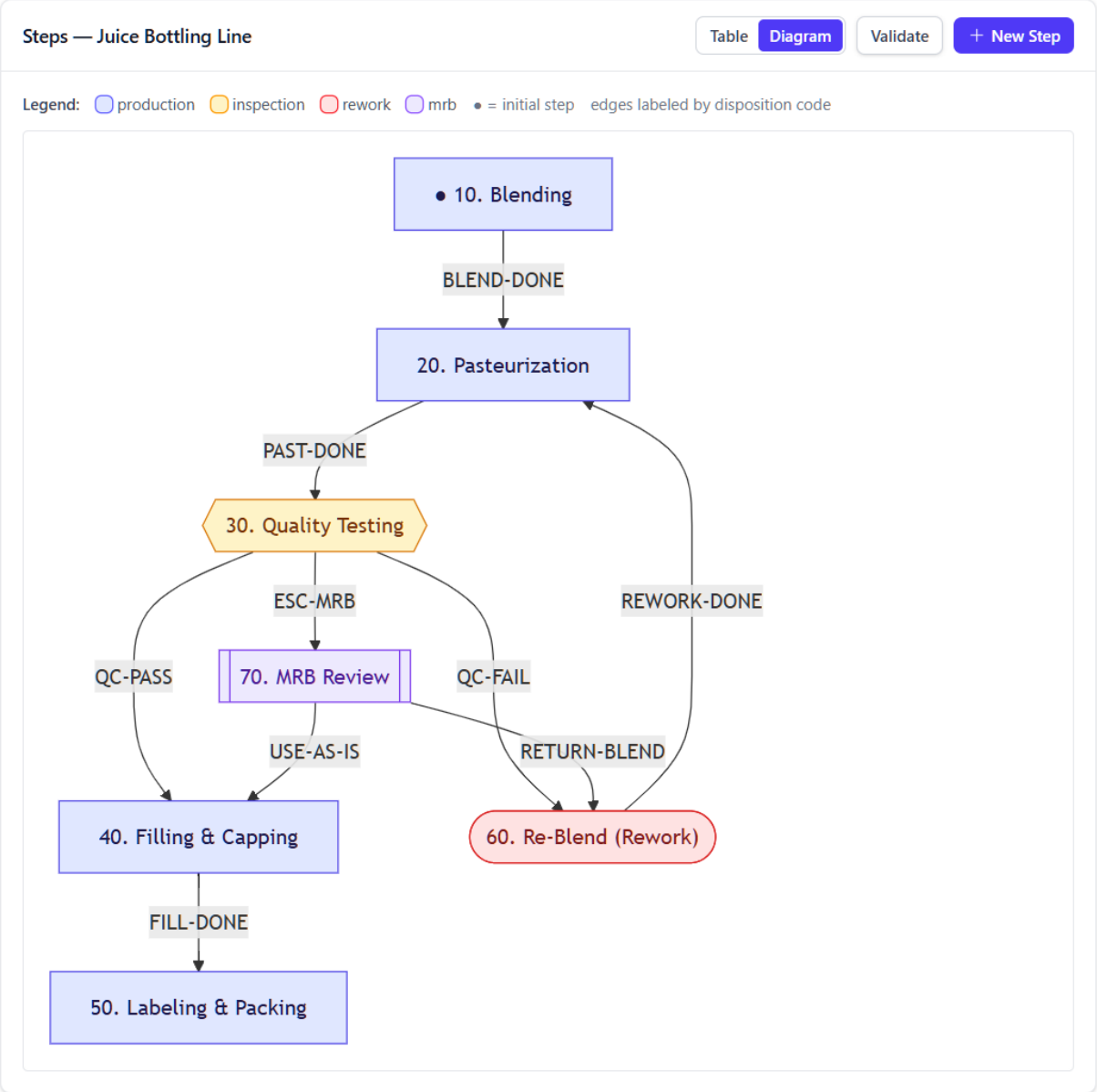
Steps — Juice Bottling Line

Table | Diagram | Validate | + New Step

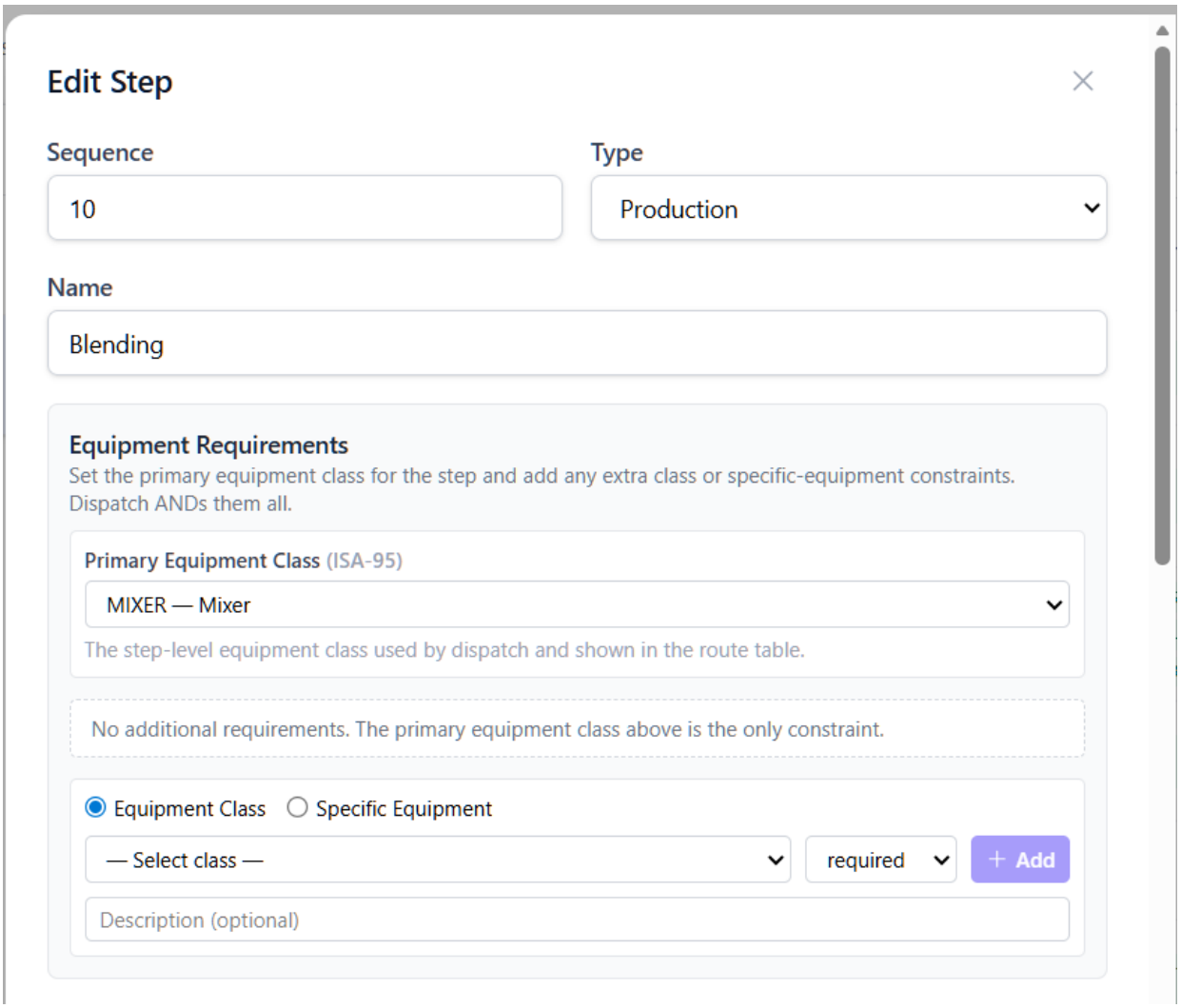
SEQ	NAME	EQUIPMENT CLASS	HOSTED WORK CELLS	TYPE	INPUTS	OUTPUTS	CYCLE TIME	ACTIONS
10	**Blending**	MIXER	WC-BLEND	production INITIAL	—	BLEND-DONE	900s	
20	**Pasteurization**	PASTEURIZER +1	WC-PAST	production	BLEND-DONE REWORK-DONE	PAST-DONE	600s	
30	**Quality Testing**	ANALYZER +1	WC-QC	inspection	PAST-DONE	QC-PASS QC-FAIL ESC-MRB	300s	
40	**Filling & Capping**	FILLER +3	WC-FILL	production	QC-PASS USE-AS-IS	FILL-DONE	1200s	
50	**Labeling & Packing**	LABELER +1	WC-PACK	production	FILL-DONE	—	600s	
60	**Re-Blend (Rework)**	TANK +1	WC-REWORK	rework	QC-FAIL RETURN-BLEND	REWORK-DONE	600s	
70	**MRB Review**	ANALYZER +1	WC-QC	mrb	ESC-MRB	RETURN-BLEND USE-AS-IS	600s	

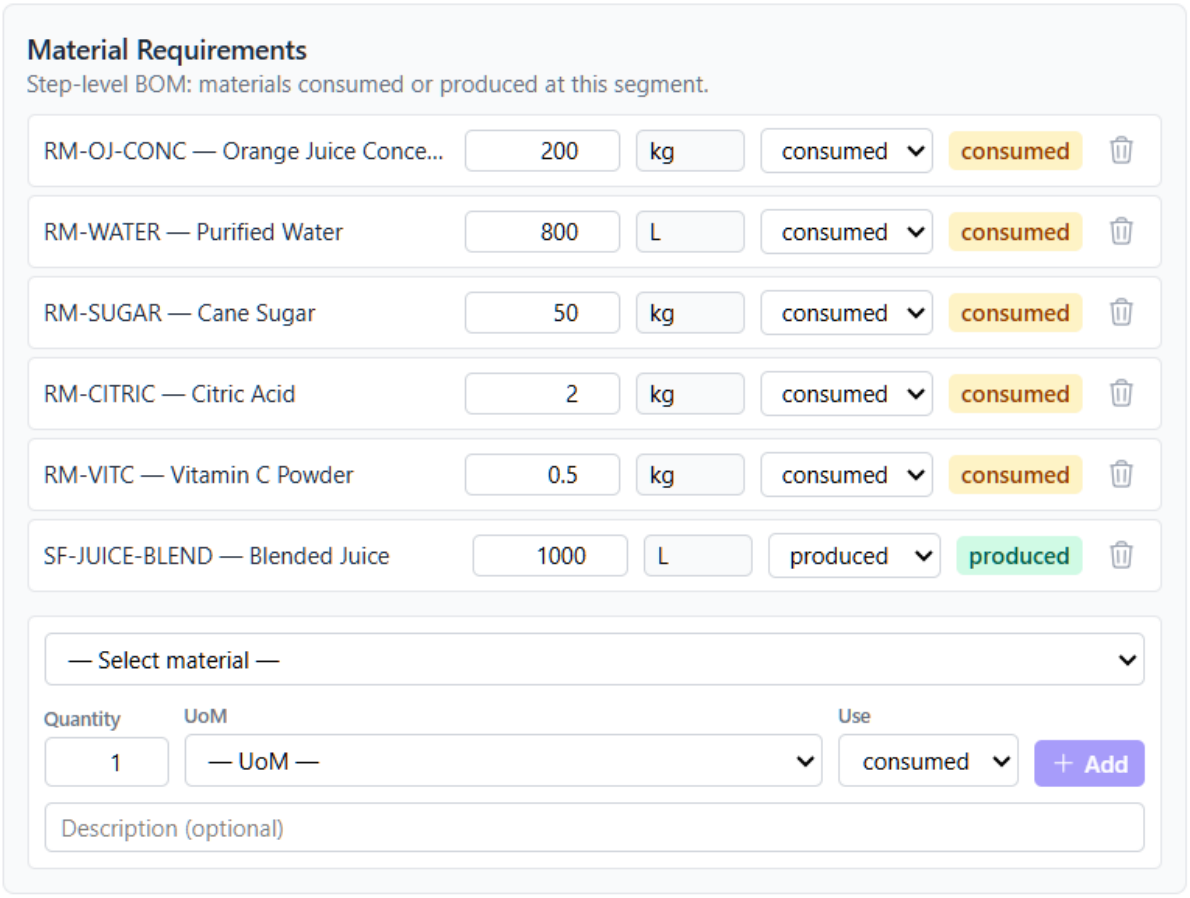


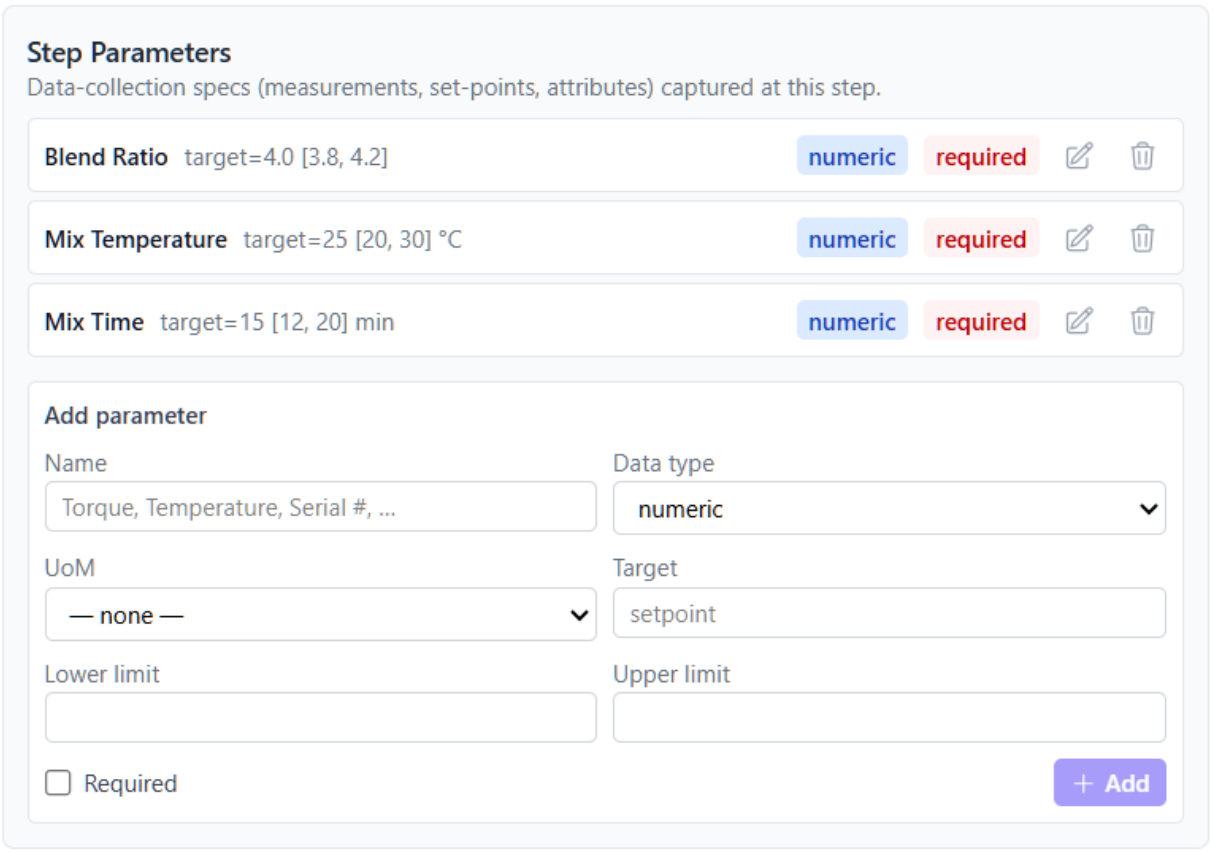
The steps can be viewed as a graph diagram:

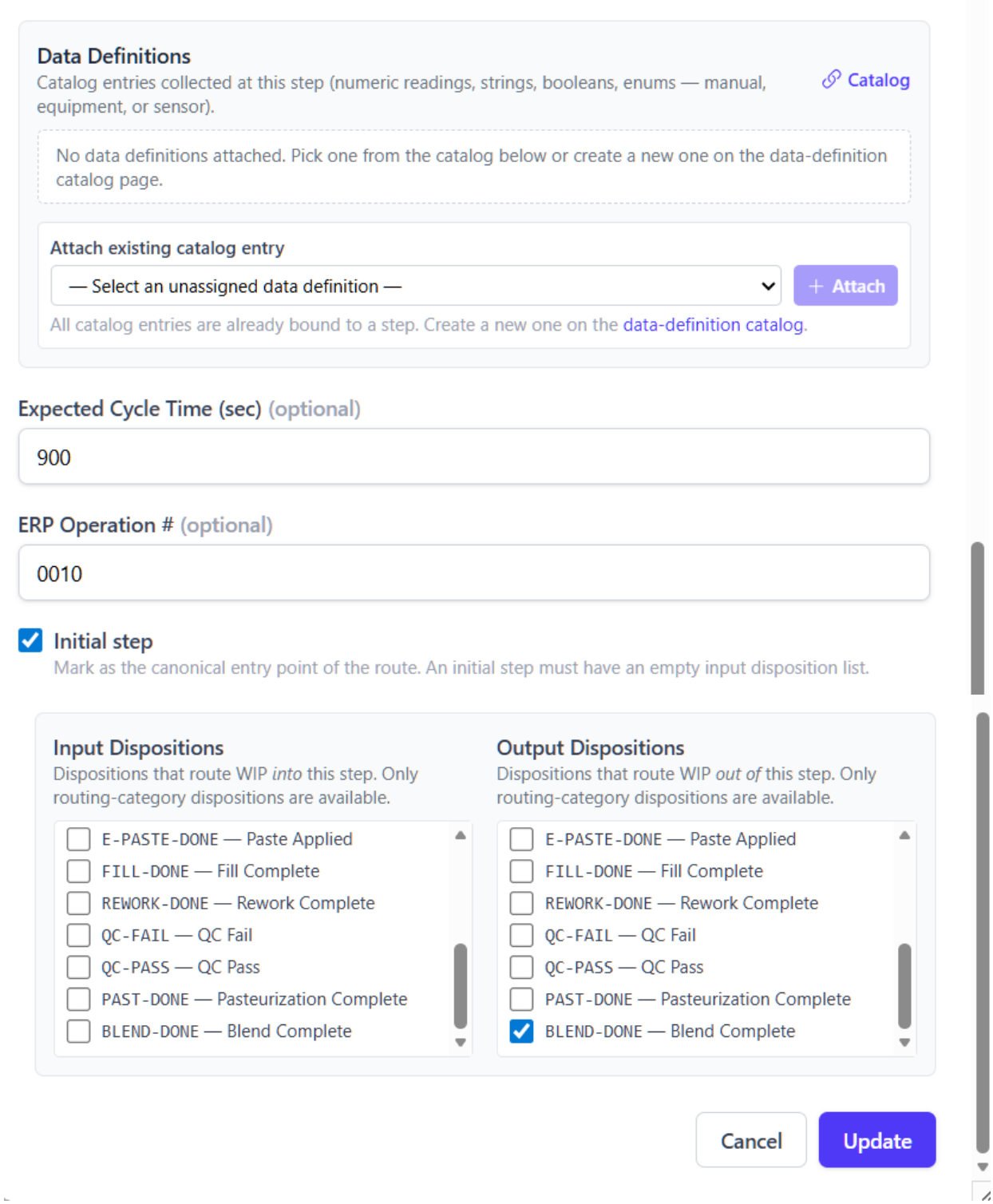


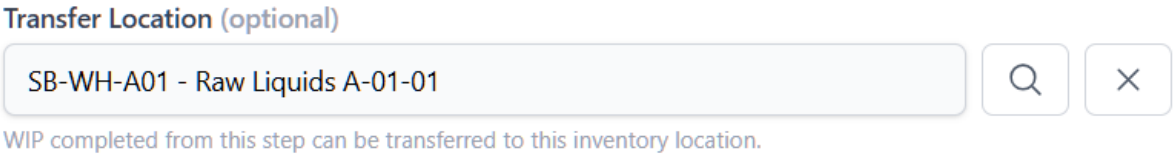
When a step is created or selected in the table, the step editor is used to define step-related properties.


Edit Step
Sequence
10
Type
Production
Name
Blending
Equipment Requirements
Set the primary equipment class for the step and add any extra class or specific-equipment constraints. Dispatch ANDs them all.
Primary Equipment Class (ISA-95)
MIXER — Mixer
The step-level equipment class used by dispatch and shown in the route table.
No additional requirements. The primary equipment class above is the only constraint.
Equipment Class
Specific Equipment
— Select class —
required
+ Add
Description (optional)


Material Requirements
Step-level BOM: materials consumed or produced at this segment.
RM-OJ-CONC — Orange Juice Conce... 200 kg consumed consumed
RM-WATER — Purified Water 800 L consumed consumed
RM-SUGAR — Cane Sugar 50 kg consumed consumed
RM-CITRIC — Citric Acid 2 kg consumed consumed
RM-VITC — Vitamin C Powder 0.5 kg consumed consumed
SF-JUICE-BLEND — Blended Juice 1000 L produced produced
— Select material —
Quantity
1
UoM
— UoM —
Use
consumed
+ Add
Description (optional)


Step Parameters
Data-collection specs (measurements, set-points, attributes) captured at this step.
Blend Ratio target=4.0 [3.8, 4.2]
numeric
required
Mix Temperature target=25 [20, 30] °C
numeric
required
Mix Time target=15 [12, 20] min
numeric
required
Add parameter
Name
Torque, Temperature, Serial #, ...
Data type
numeric
UoM
— none —
Target
setpoint
Lower limit
Upper limit
Required
+ Add

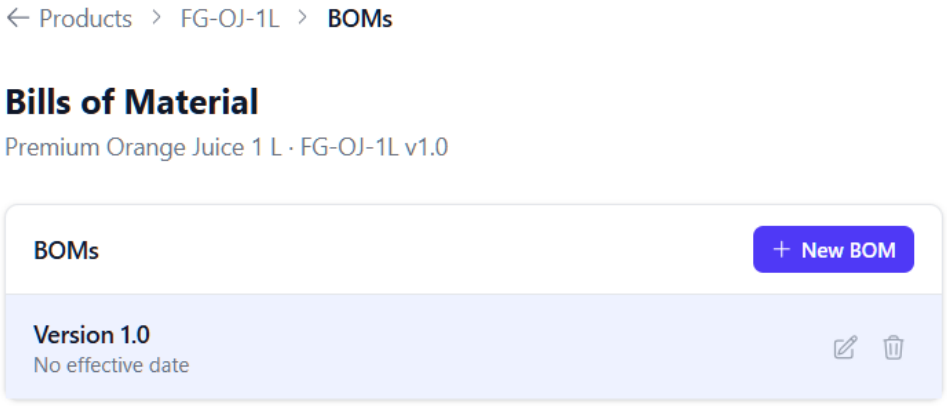

Data Definitions
Catalog entries collected at this step (numeric readings, strings, booleans, enums — manual, equipment, or sensor).
Catalog
No data definitions attached. Pick one from the catalog below or create a new one on the data-definition catalog page.
Attach existing catalog entry
— Select an unassigned data definition —
Attach
All catalog entries are already bound to a step. Create a new one on the data-definition catalog.
Expected Cycle Time (sec) (optional)
900
ERP Operation # (optional)
0010
Initial step
Mark as the canonical entry point of the route. An initial step must have an empty input disposition list.
Input Dispositions
Dispositions that route WIP into this step. Only routing-category dispositions are available.
E-PASTE-DONE — Paste Applied
FILL-DONE — Fill Complete
REWORK-DONE — Rework Complete
QC-FAIL — QC Fail
QC-PASS — QC Pass
PAST-DONE — Pasteurization Complete
BLEND-DONE — Blend Complete
Output Dispositions
Dispositions that route WIP out of this step. Only routing-category dispositions are available.
E-PASTE-DONE — Paste Applied
FILL-DONE — Fill Complete
REWORK-DONE — Rework Complete
QC-FAIL — QC Fail
QC-PASS — QC Pass
PAST-DONE — Pasteurization Complete
BLEND-DONE — Blend Complete
Cancel
Update

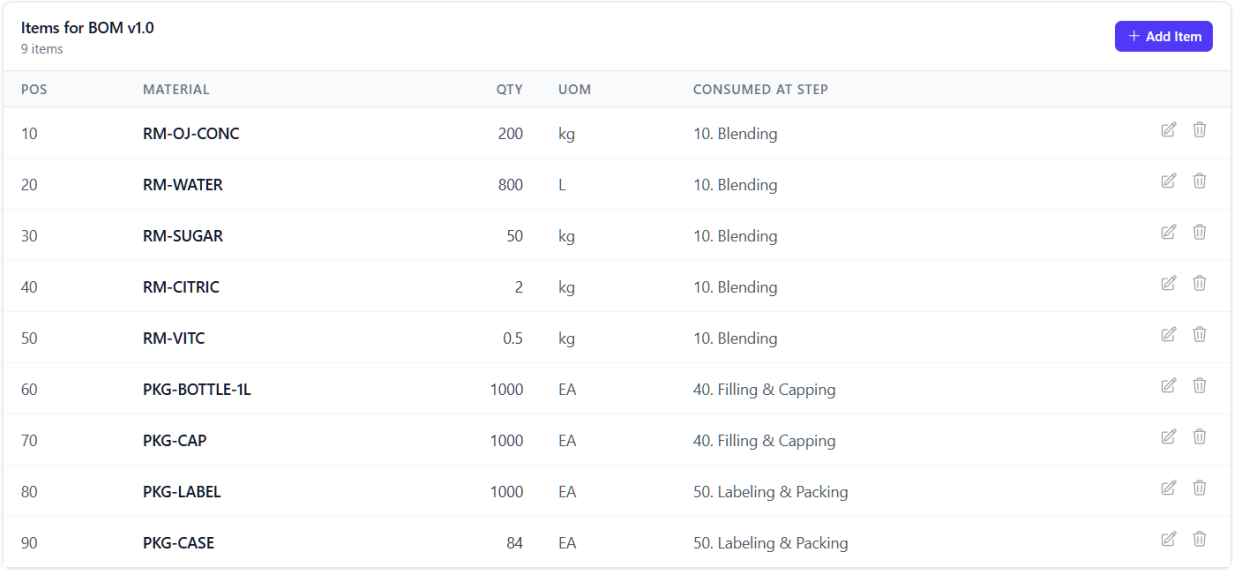


In this example, step #10 (Blending) is the initial step on the juice bottling line and has only one output disposition “BLEND-DONE” which sends the lot to step #20 Pasteurization.

The equipment class panel states that equipment must be in the MIXER class to be acceptable for dispatching a lot, and no specific equipment is required.

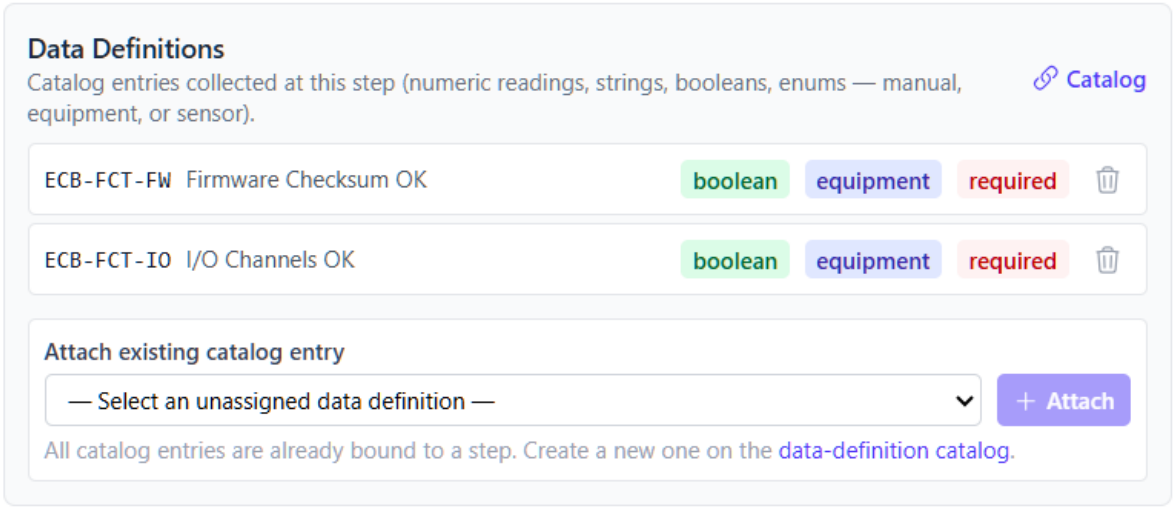
The material requirements panel defines five raw materials that are consumed and one intermediate material that is produced. The consumed materials are in the product Bill of Material (BOM). Click on the Products link in the sidebar, then on the name of the product to display the details editor:





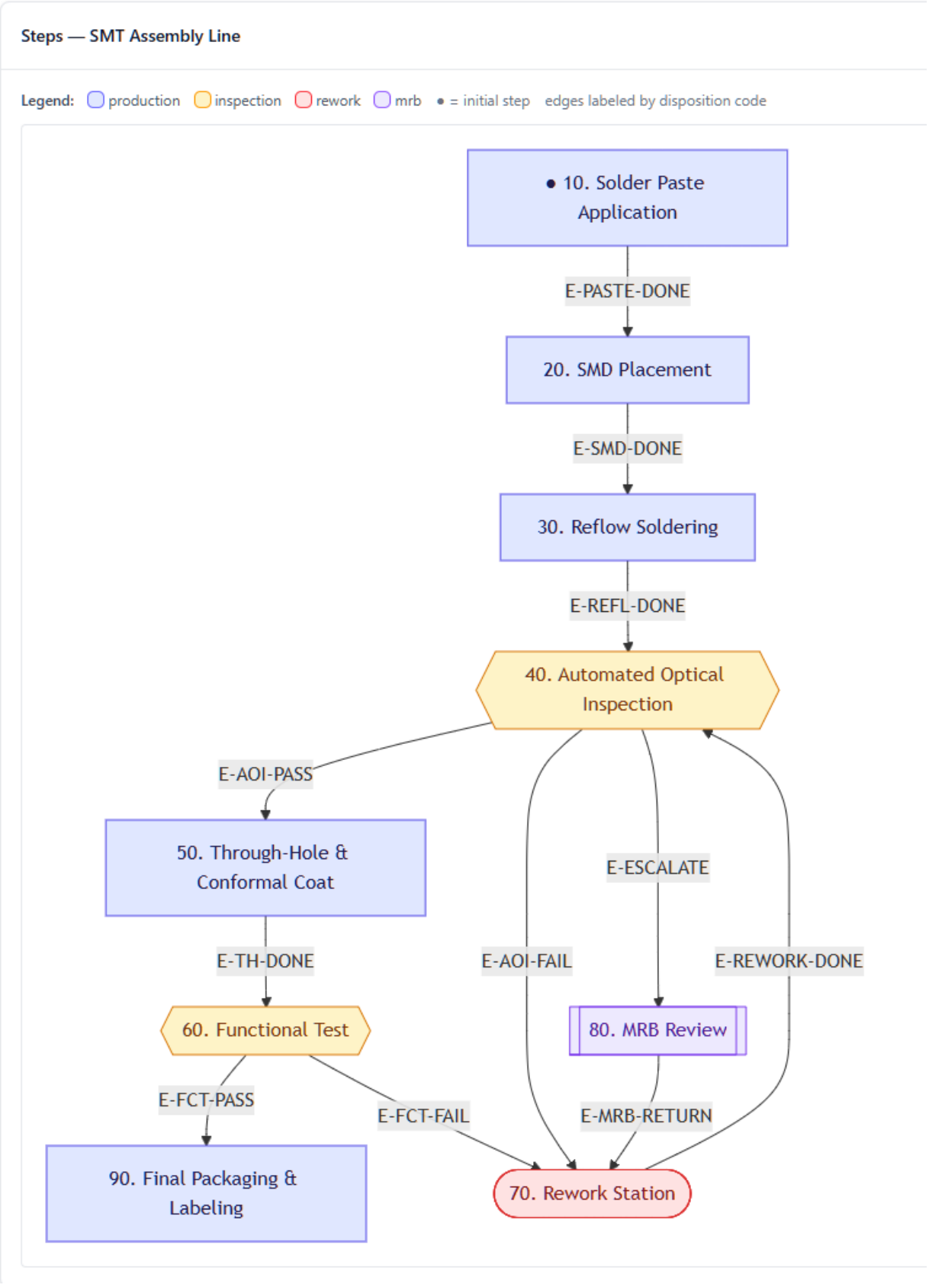
The step parameters panel defines parameters at a route step than are typically setpoints or critical values. At runtime, the actual value can be collected.

The data definitions panel defines data that is not a step parameter to be collected at a route step. Here's an example from the SMT line at Functional Test:

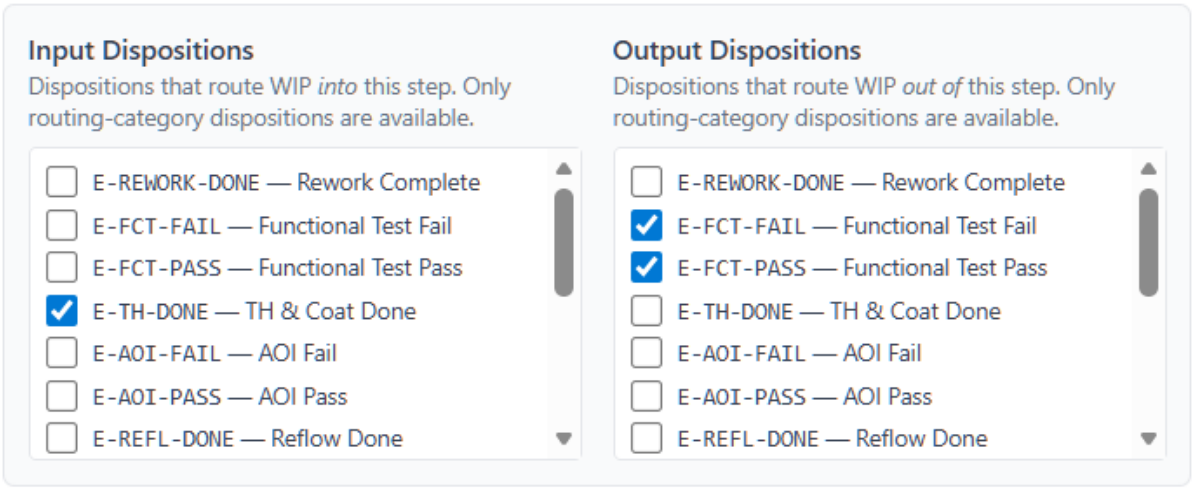


and its route:

Steps — SMT Assembly Line



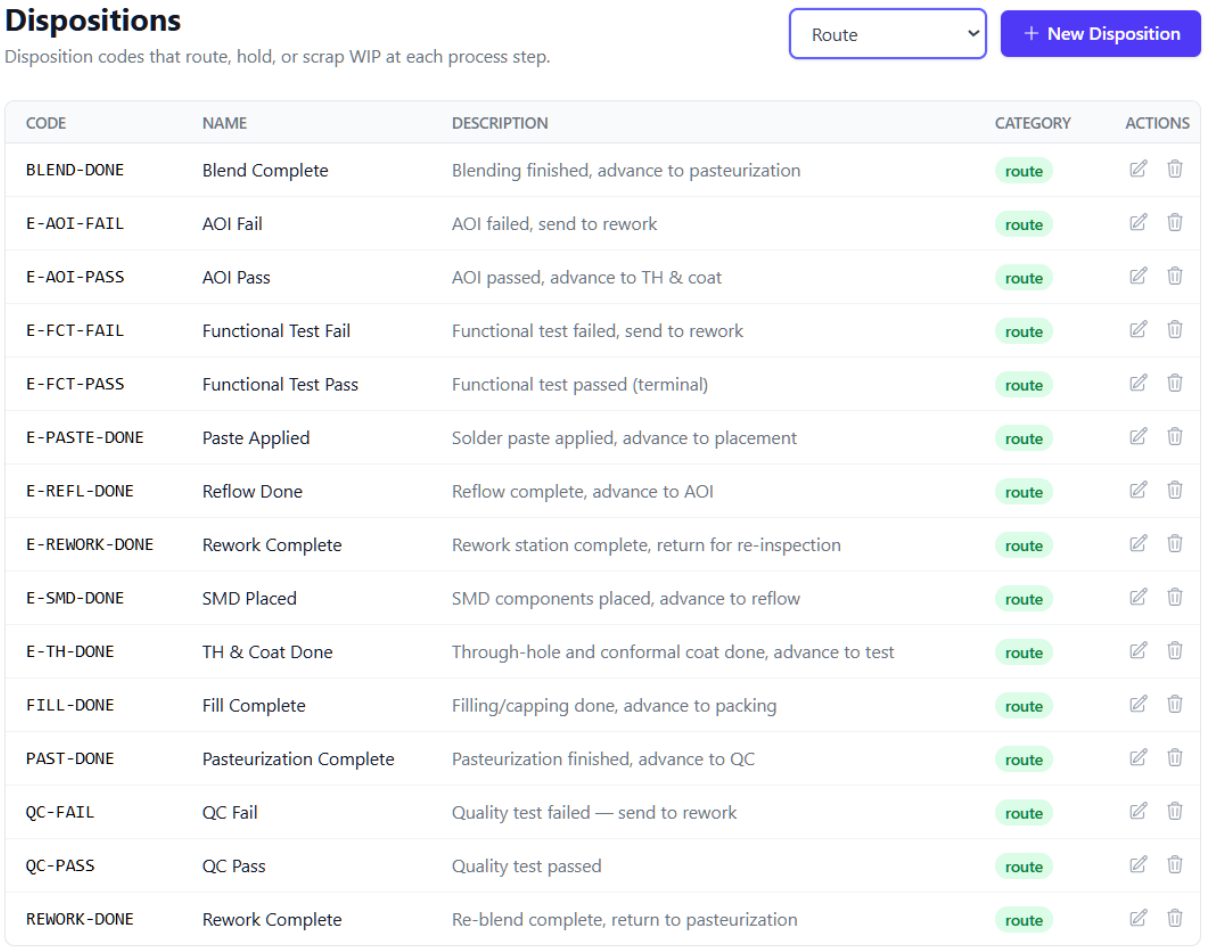
where the step #60 has one input disposition and two outputs:



An optional storage location can be defined as a transfer location for WIP completed from this step. If so defined, a Transfer button will be displayed next to the Complete button. For example, when a WIP lot or unit completes the last step of the route, the good quantity can be transferred to a finished goods warehouse. From that location it can be shipped using the inventory client.

Dispositions

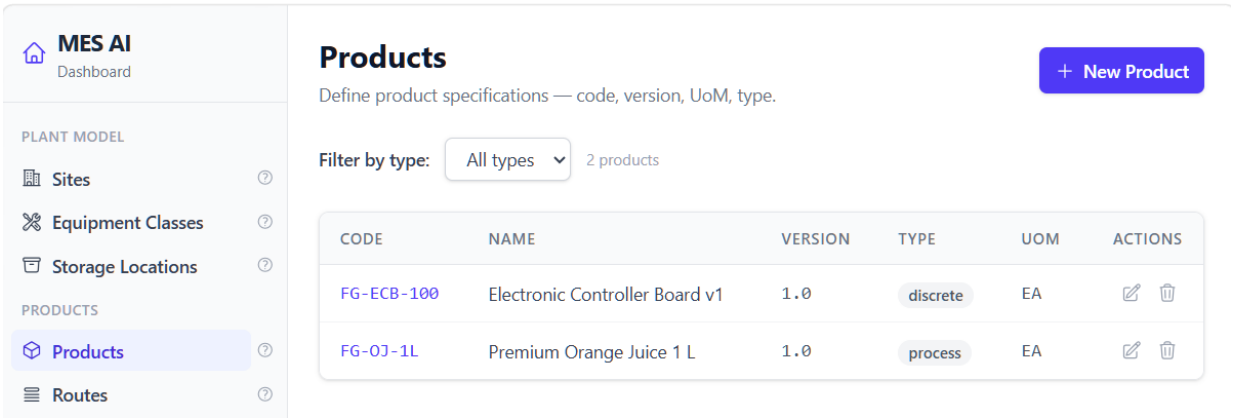
The disposition editor is used to define the dispositions that route WIP through processing steps or put it on hold, release or scrap it.



Products

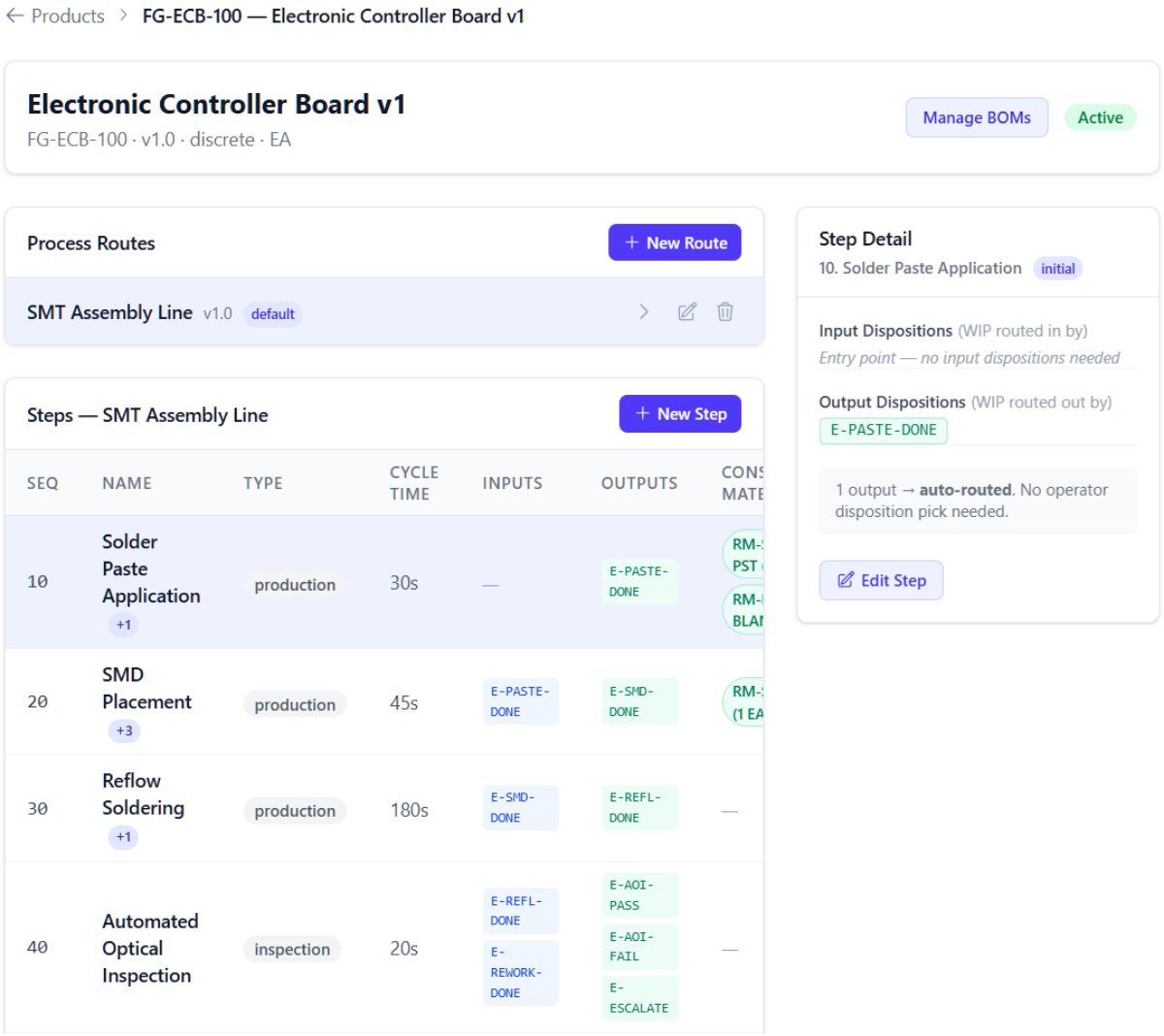
A product is a produced material. It can be either “discrete” in which case the WIP is tracked as a serialized unit, or “process” where it is tracked as a lot.

The products editor looks like this:

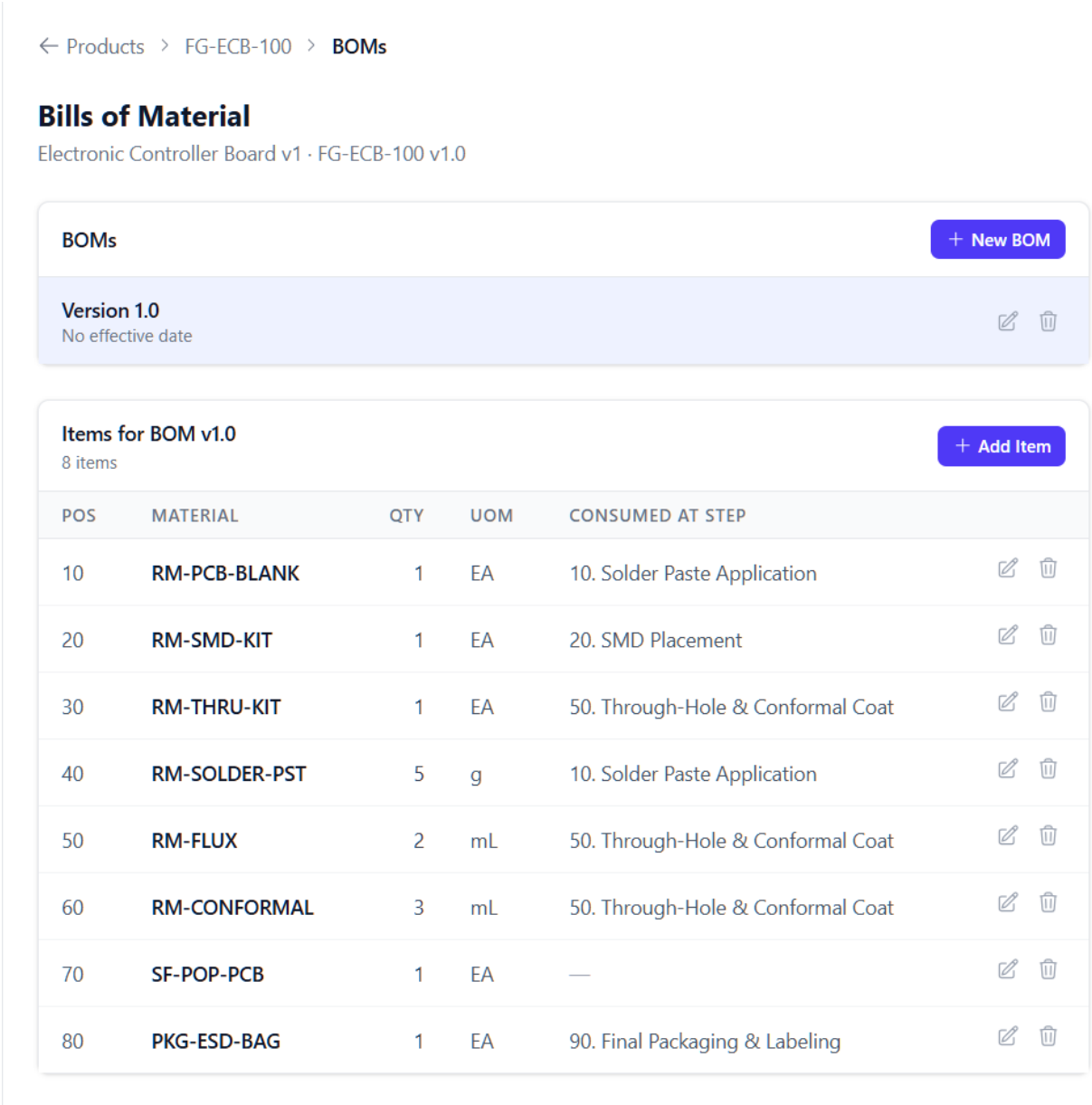


You can edit a product or delete it. Click on a product to view its route and step details

as well as its Bill of Material:



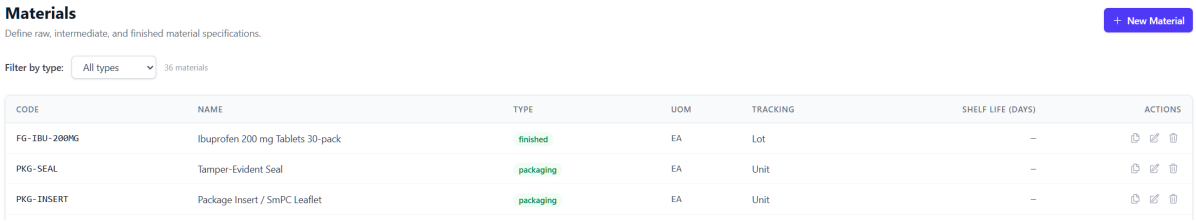
You can also edit its BOM, by clicking on the Manage BOM button:



Materials

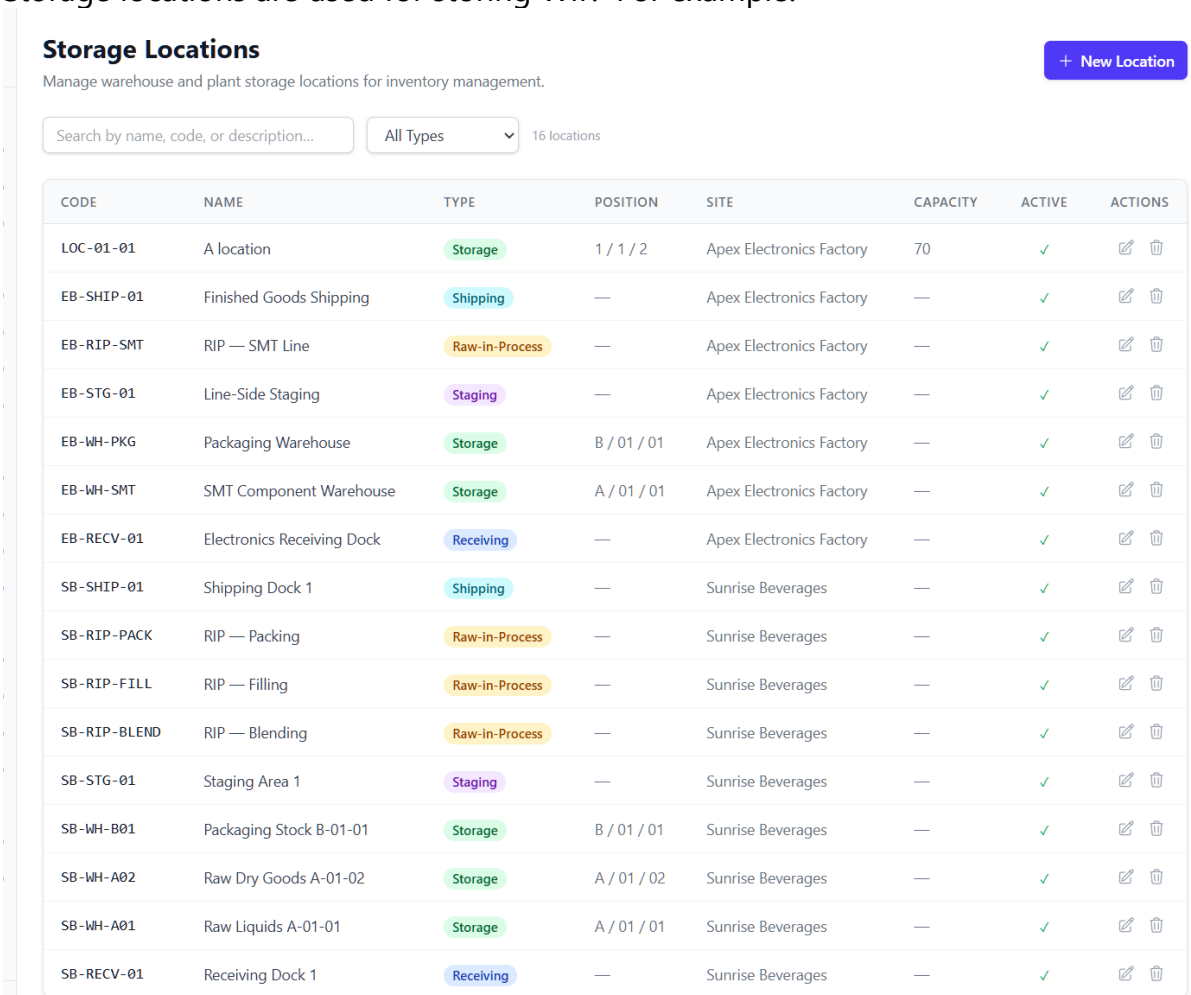
Materials are consumed or produced by equipment. The material can be designated as tracked by lot in which case the inventory item is tracked as a lot, or by unit in which case the item is tracked as a serialized unit.

The Materials editor allows a user to create, update and delete materials and search by type. For example:



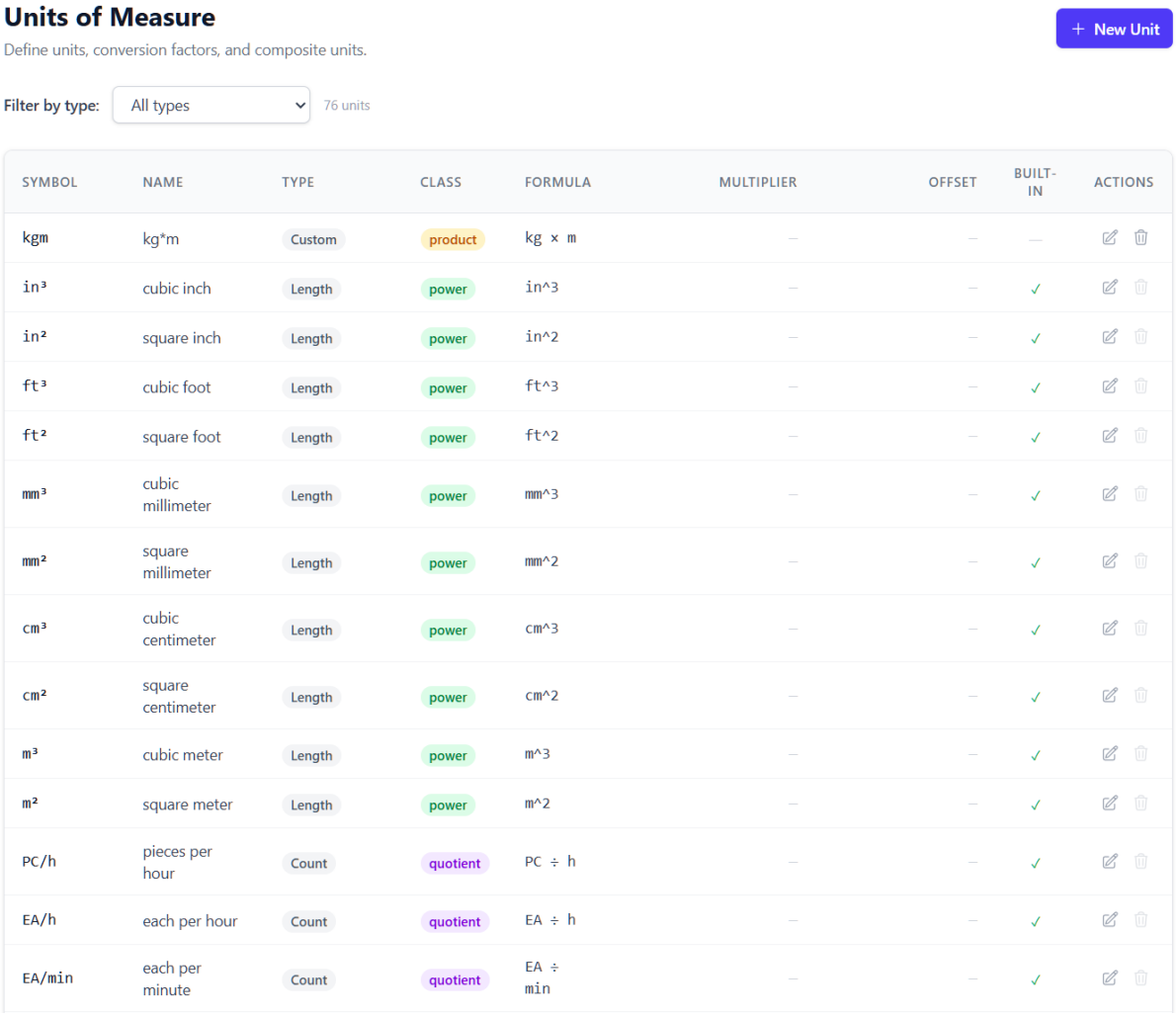
Storage Locations

Storage locations are used for storing WIP. For example:

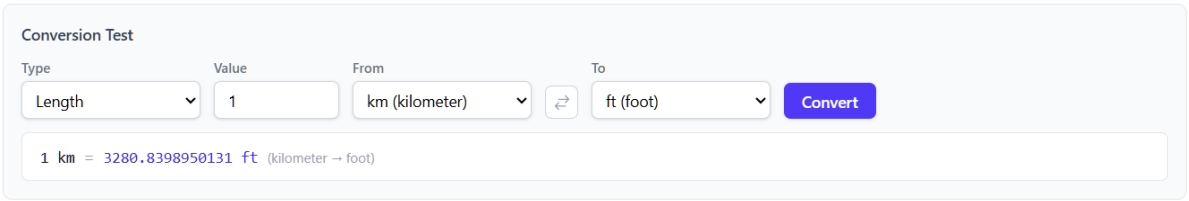


Units of Measure

Units of measure are paired with a numerical quantity of a substance. They have a type (time, length, mass, temperature, etc.) and a class - scalar (e.g. kilogram), quotient (e.g. meters/sec), power (square meters) or product (kg * m). Complex types like a Newton can be created from intermediate UOMs.

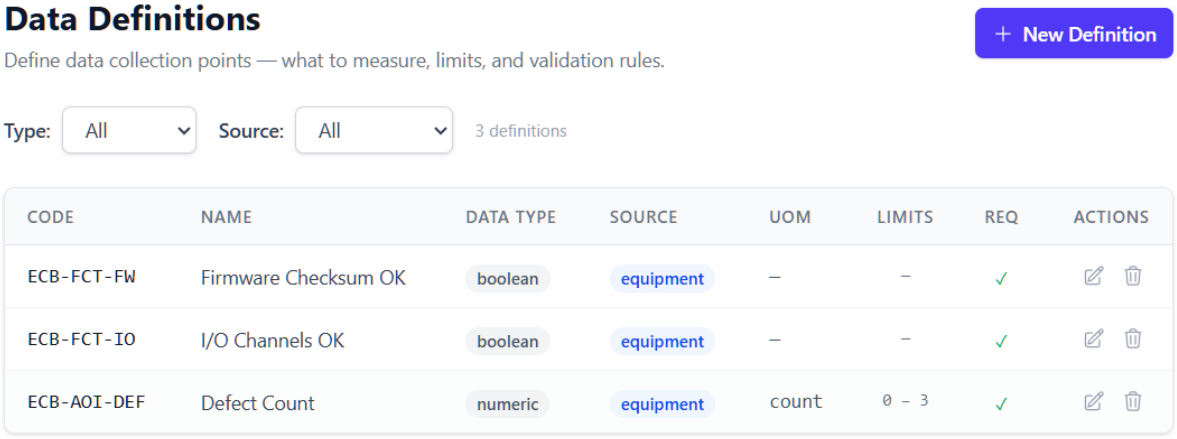


Compatible types can be converted from one to another. For example:



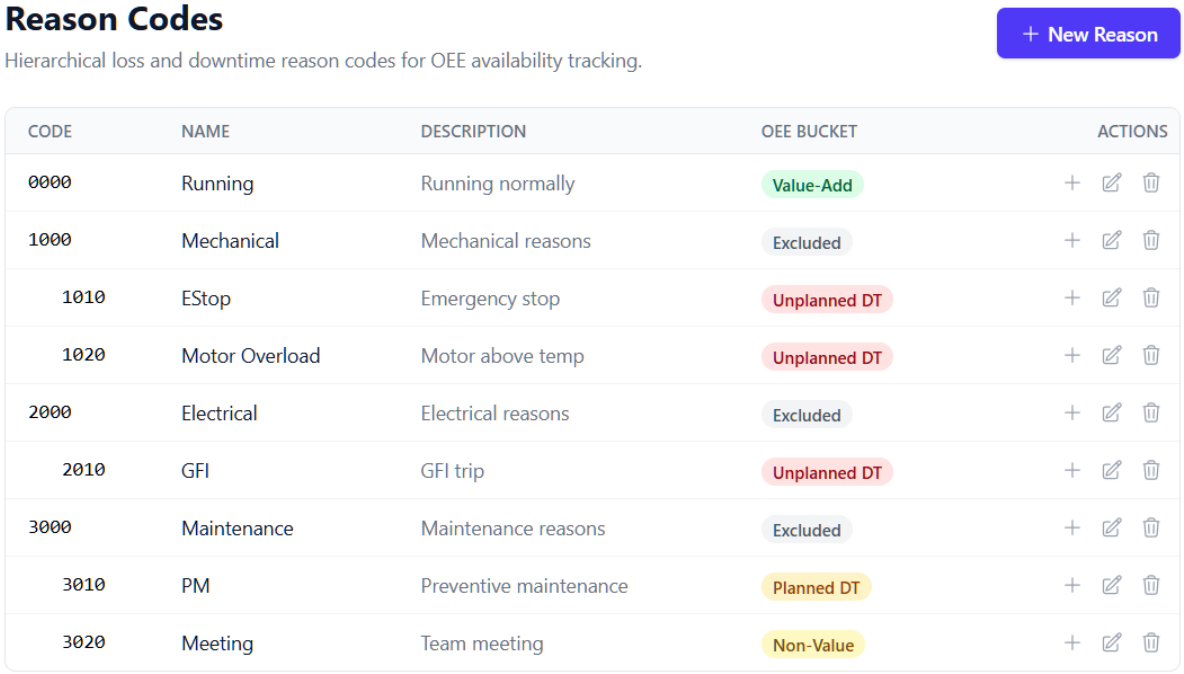
Data Definitions

A data definition is a specification of a data value to be collected during WIP processing. For example:



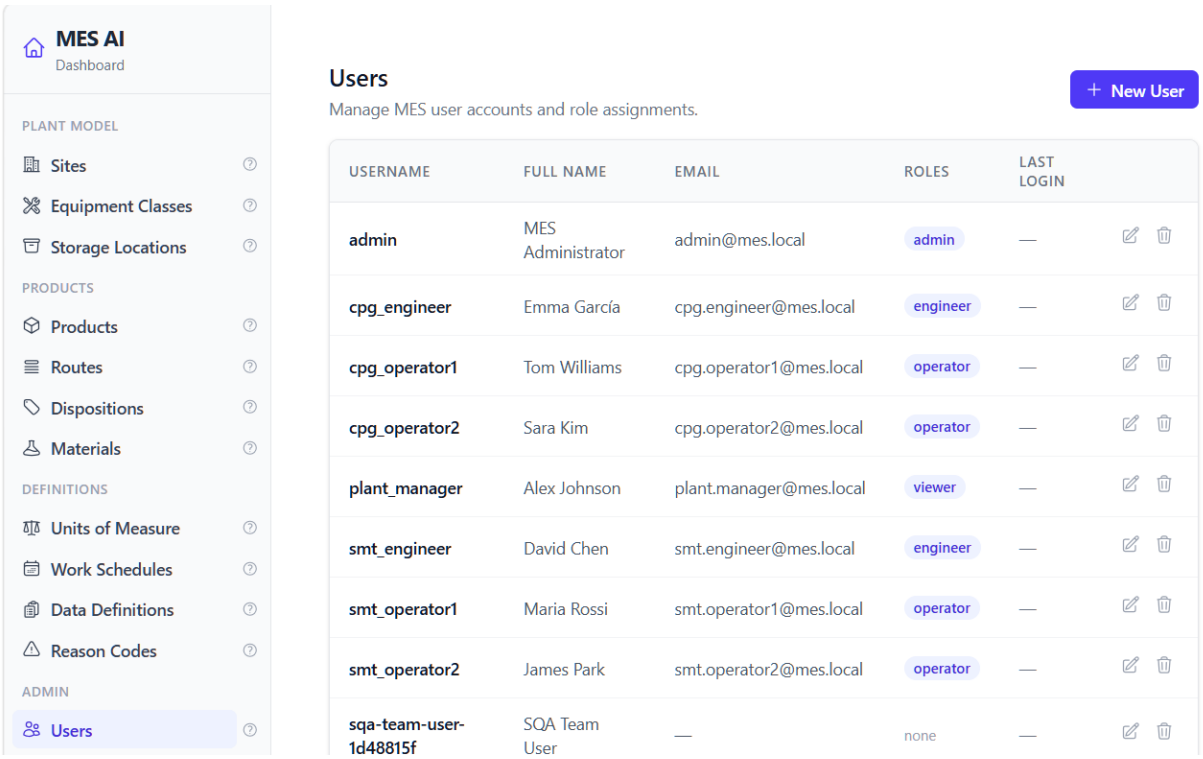
Reason Codes

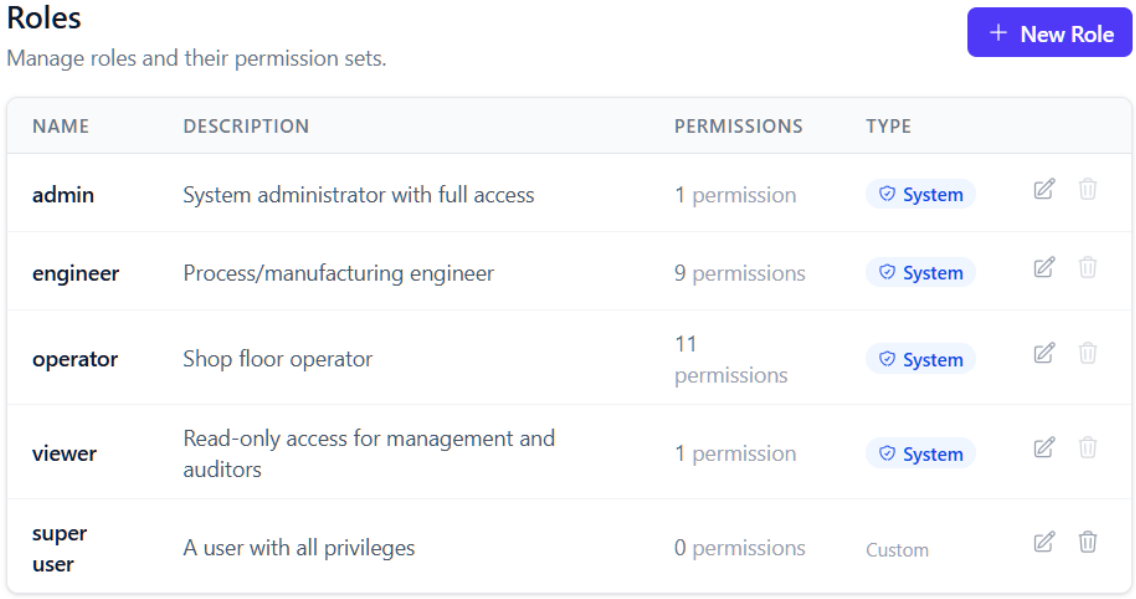
Reason codes are used to provide the OEE availability bucket when an equipment event occurs. For example:



Users and Roles

These two editors are used to create, edit and delete users and roles when managing authentication in the local database. For example:



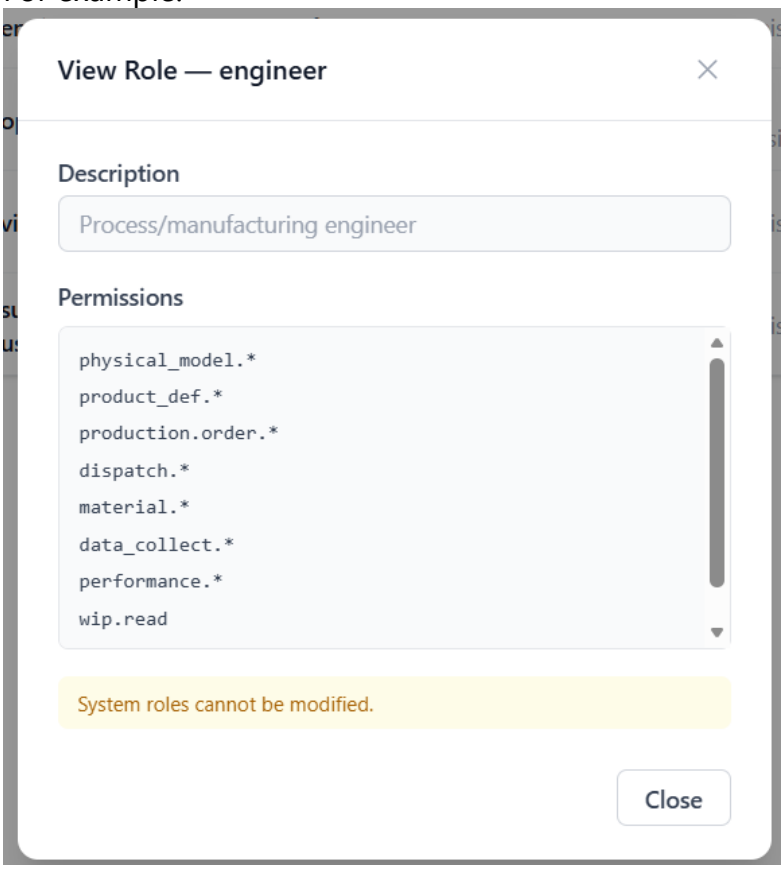


Permissions in the roles editor are dot-separated strings (e.g. material.read) that are stored on a Role and checked server-side via the require_permission() FastAPI dependency on every protected route. When a user makes an API call, their aggregated permissions are matched against the required string — including wildcard support (*, wip.*, *.read).

The permission namespace is divided by domain and action:

Domain	Actions observed
auth.user	read, create, update, delete
auth.role	read, create, update, delete
physical_model	read, create, update, delete
product_def	read, create, update, delete
material	read, create, update, delete
wip	read, create + sub-actions like wip.unit.move
production	read, create, update
performance	read, create
inventory	read, create, update
data_collection	read, create
dispatch	read, execute
genealogy	read

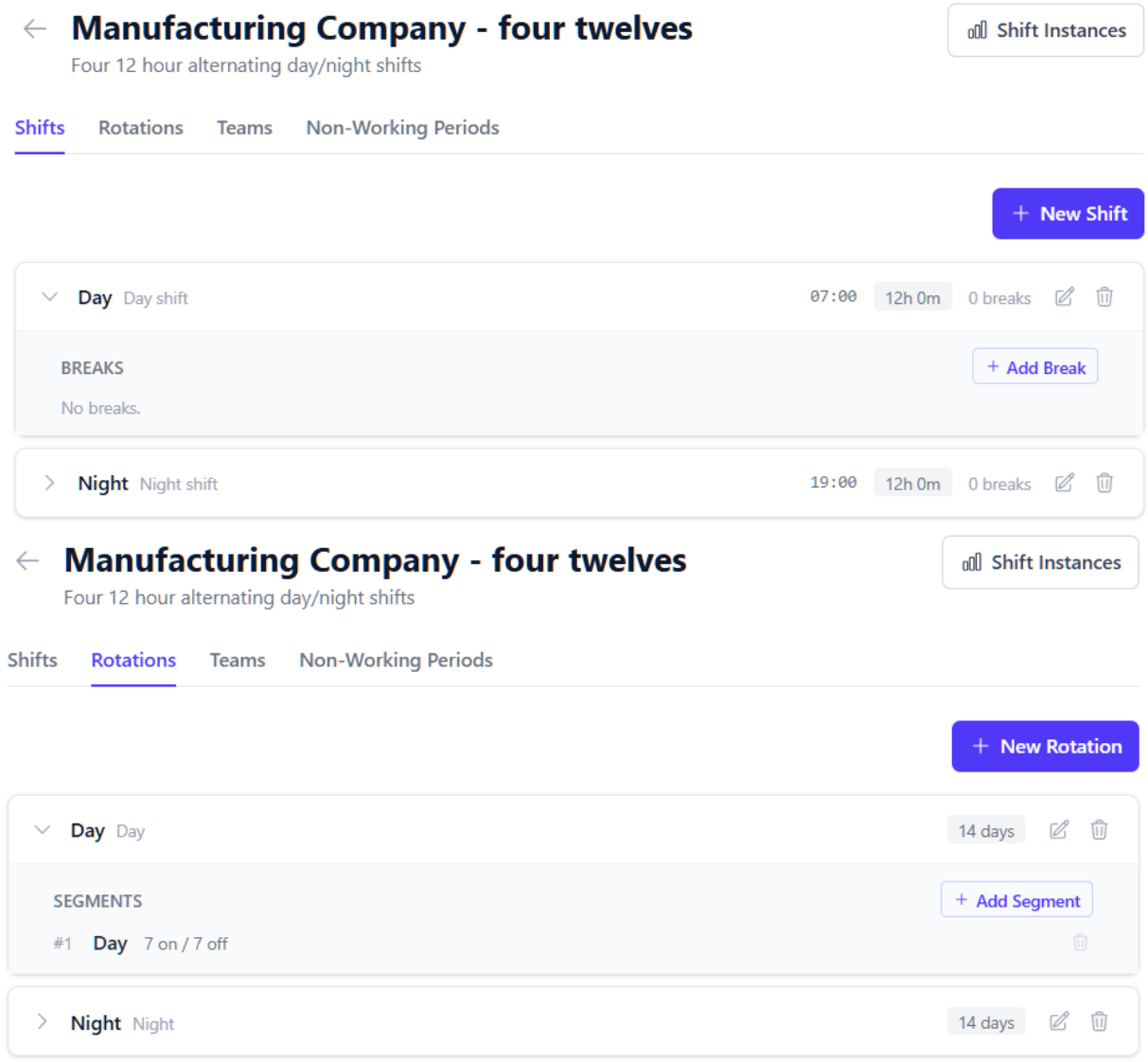
For example:

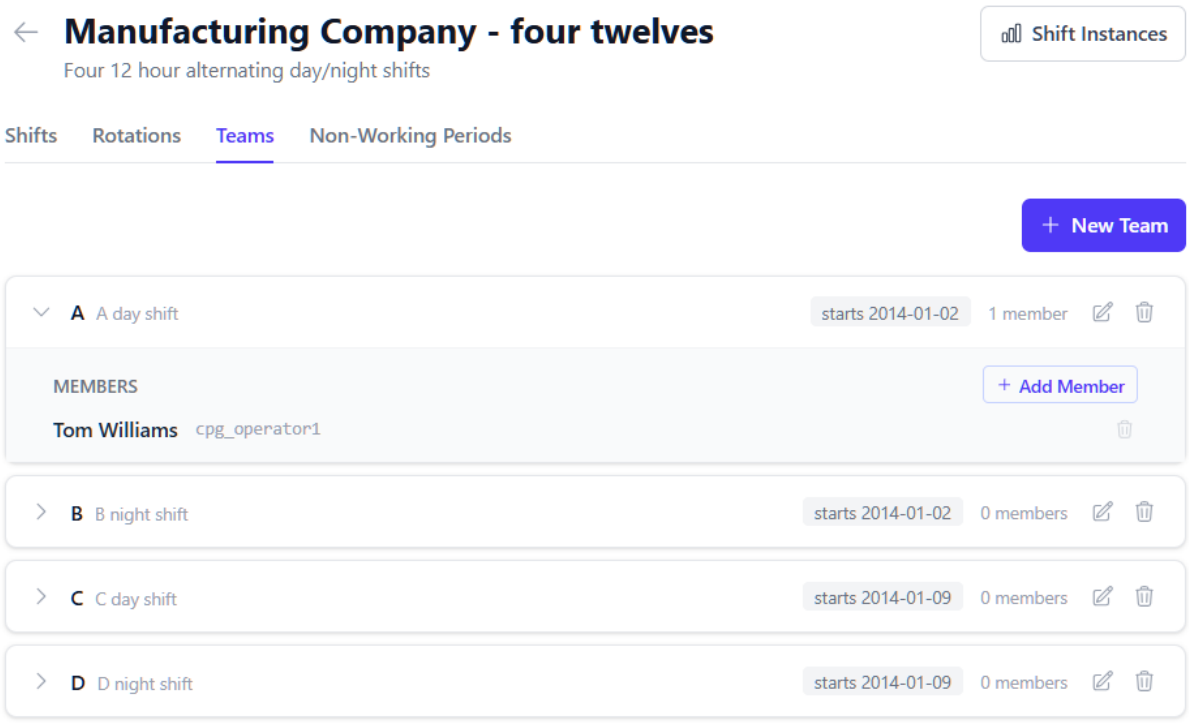


Work Schedule

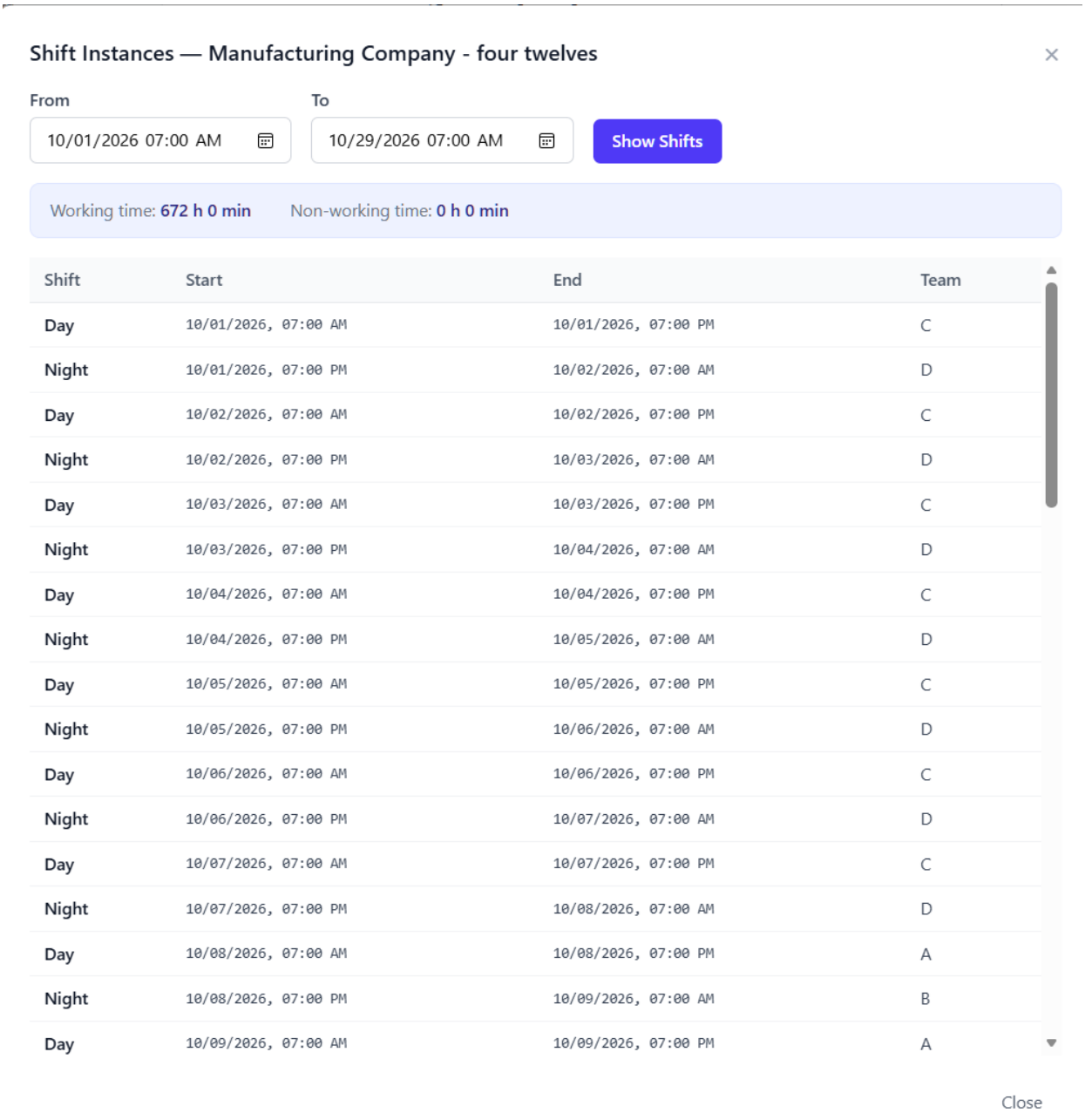
This editor defines the shifts worked by teams as well as non-working time (i.e. holidays). Work schedules are assigned to levels in the S95 hierarchy, for example by site. A lower=level schedule overrides a level higher u.

This schedule is four alternating 12 hours day/night shifts with 7 days on followed by 7 days off. There are two day A/C teams and two night B/D teams.





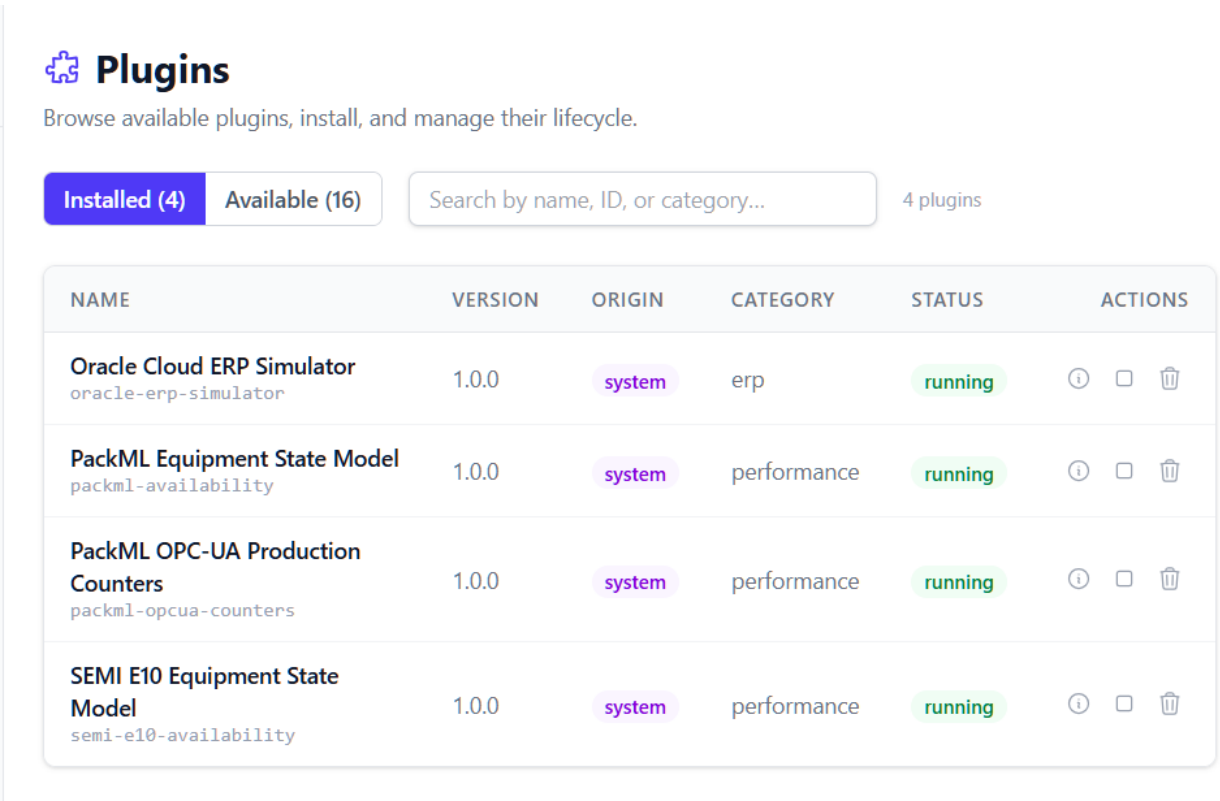
Once a work schedule is defined, the shift instances for a period of time can be calculated:

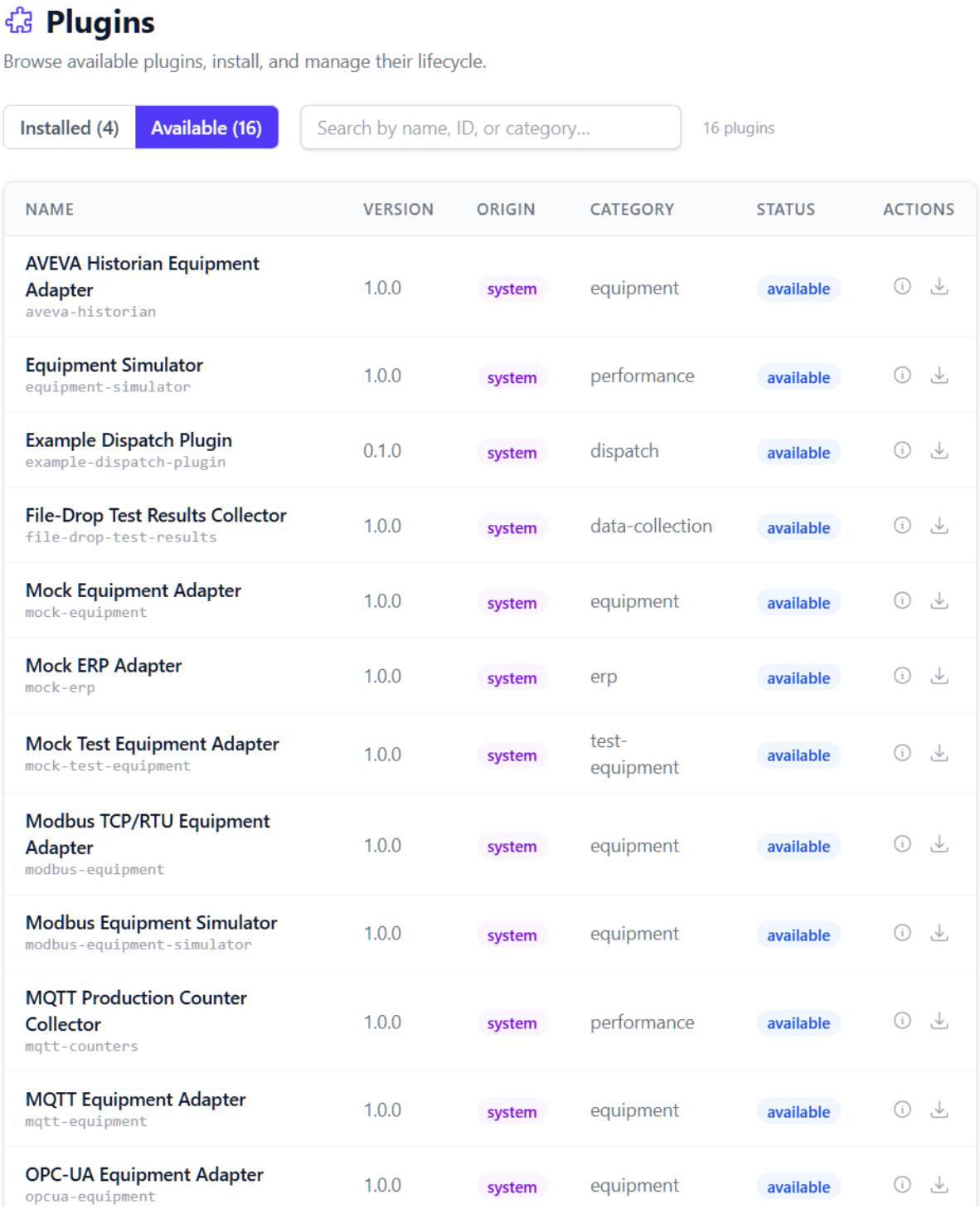


Plugins

Plugins are used to extend functionality beyond the core MES WIP, inventory and equipment tracking. A plugin is implemented with an adapter which interfaces to the REST API, and typically a simulator client for developing and testing purposes. The adapter typically has a configuration editor.

A plugin is first installed from the available list, then enabled. The plugin editor has a tab for installed ones and available ones. For example:

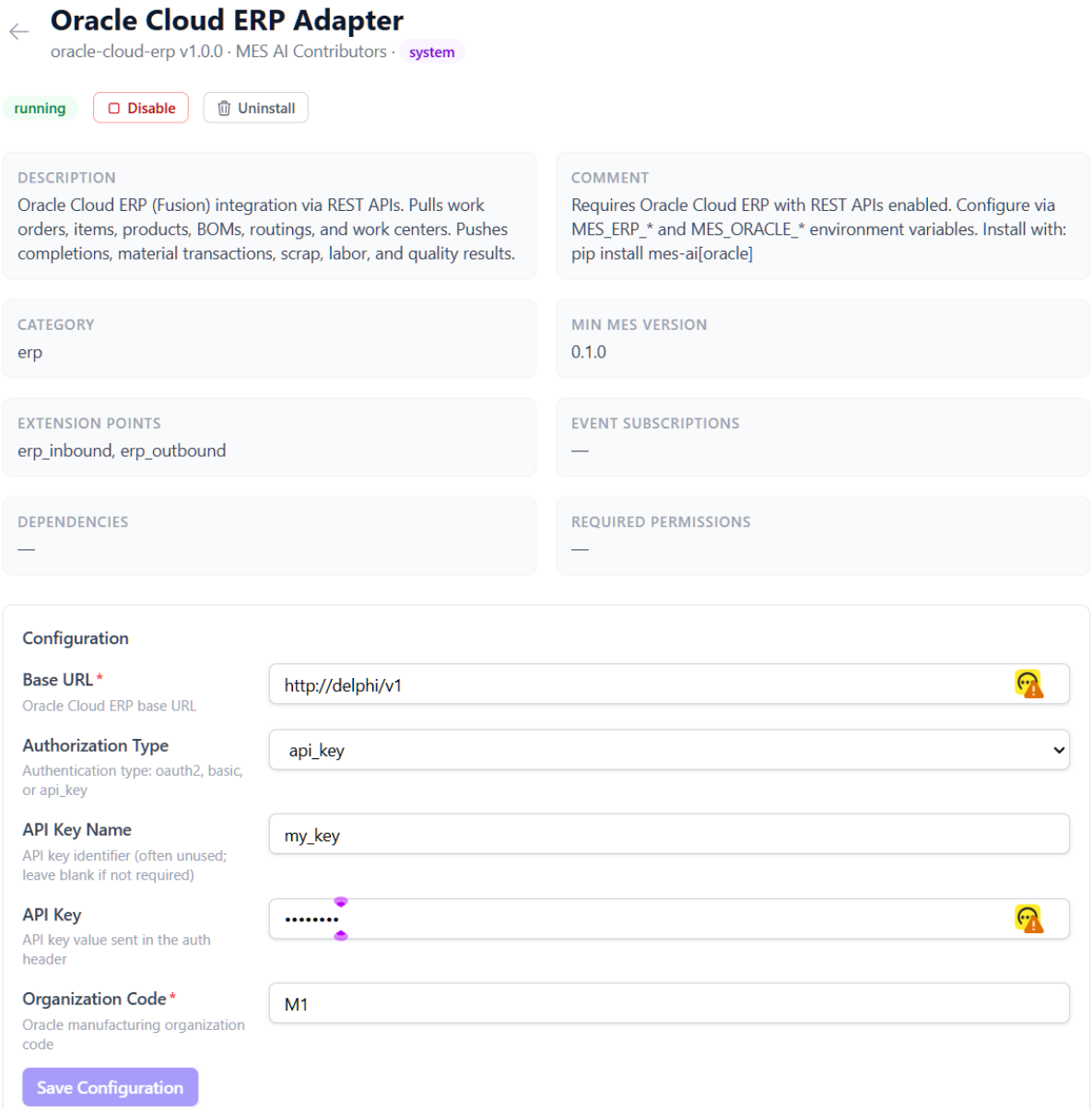

Plugins
Browse available plugins, install, and manage their lifecycle.
Installed (4)
Available (16)
Search by name, ID, or category...
4 plugins
NAME
VERSION
ORIGIN
CATEGORY
STATUS
ACTIONS
Oracle Cloud ERP Simulator
oracle-erp-simulator
1.0.0
system
erp
running
PackML Equipment State Model
packml-availability
1.0.0
system
performance
running
PackML OPC-UA Production Counters
packml-opcua-counters
1.0.0
system
performance
running
SEMI E10 Equipment State Model
semi-e10-availability
1.0.0
system
performance
running



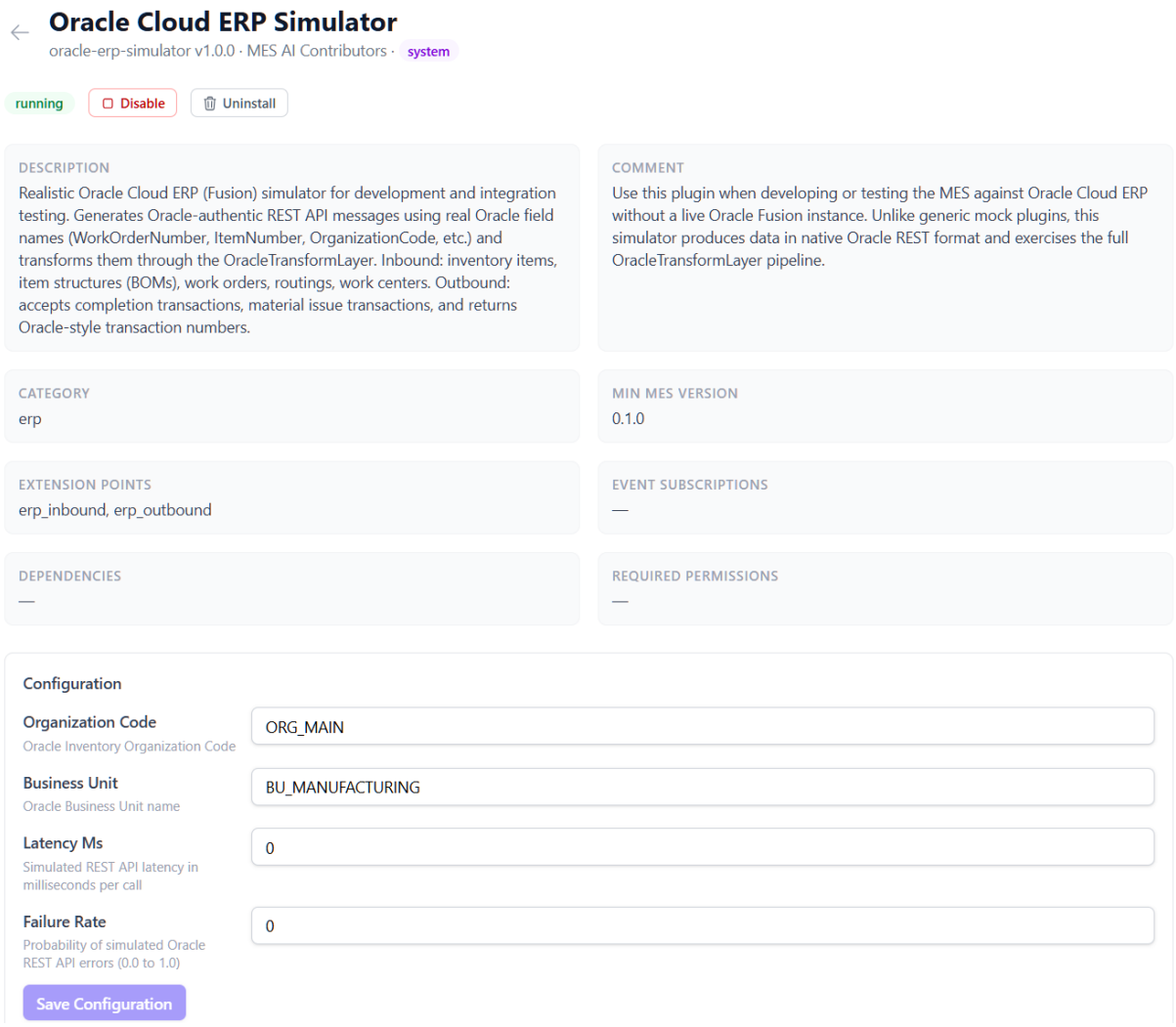
Enterprise Plugins

The MES framework implements two ERP plugins: SAP and Oracle. For example,

here is the Oracle adapter configuration:



and the corresponding simulator configuration:



Only one ERP plugin should be enabled, and either the adapter or the simulator.

Factory Floor Plugins

This MES framework provides 12 factory floor plugins. The equipment related ones for performance, availability and quality are:

- PackML Equipment State Model
- PackML OPC-UA Production Counters
- SEMI E10 Equipment State Model
- Equipment Simulator
- AVEVA Historian Equipment Adapter
- Modbus TCP/RTU Equipment Adapter and Modbus Equipment Simulator
- MQTT Equipment Adapter and MQTT Production Counter Collector
- OPC-UA Equipment Adapter

- STOMP Equipment Adapter for JMS messages
- STOMP Equipment Adapter
- Example Dispatch Plugin for a custom dispatch algorithm template

and for parsing test result data files:
- File-Drop Test Results Collector

Native SDK Bridge Sidecar Plugin

Some industrial systems — Apache Kafka, OPC-UA servers, proprietary historians, and certain ERP systems — are only accessible through vendor SDKs that are written in Java, C++, C#, or Go, with no production-quality Python bindings available. To integrate these systems without forking the MES server into a polyglot monolith, the platform uses a native SDK bridge sidecar pattern.

A Java SDK for a Kafka messaging broker is the reference implementation. Here is the configuration page:

Kafka Java SDK Bridge

kafka-java-bridge v1.0.0 · MES AI Contributors · system

running | Disable | Uninstall | Test

Build Prerequisites

The Kafka bridge requires a Java fat-jar (Maven build) and generated Python gRPC stubs. Both must be present before the plugin can be installed and enabled.

After updating library versions in pom.xml (grpc.version, protobuf.version, kafka.version), click **Rebuild** to force a clean Maven build and regenerate the stubs — then Uninstall → Install → Enable the plugin so the new jar is picked up.

- Java fat-jar: built `bridge/target/kafka-bridge-1.0.0-shaded.jar`
- Python gRPC stubs: generated
- Maven: `C:\dev_support\apache-maven-3.9.4\bin\mvn.CMD`

Rebuild

DESCRIPTION

Connects the MES to Apache Kafka using the official Kafka Java SDK (org.apache.kafka:kafka-clients). A Java fat-jar sidecar process wraps the SDK and exposes a gRPC service on localhost; this plugin manages that process and translates incoming Kafka records into MES events on the internal event bus. Designed for factory data sources that only provide a Java SDK and have no HTTP or Python API. Implements the native_sdk_bridge extension point pattern described in ARCHITECTURE.md.

COMMENT

Prerequisites: Python: pip install grpcio grpcio-tools python proto/generate_stubs.py Java: mvn -f bridge/pom.xml clean package -q (produces bridge/target/kafka-bridge-1.0.0-shaded.jar) Java 17+ and Maven 3.8+ must be on the PATH of the MES server host.

CATEGORY

equipment

MIN MES VERSION

1.0.0

EXTENSION POINTS

native_sdk_bridge

EVENT SUBSCRIPTIONS

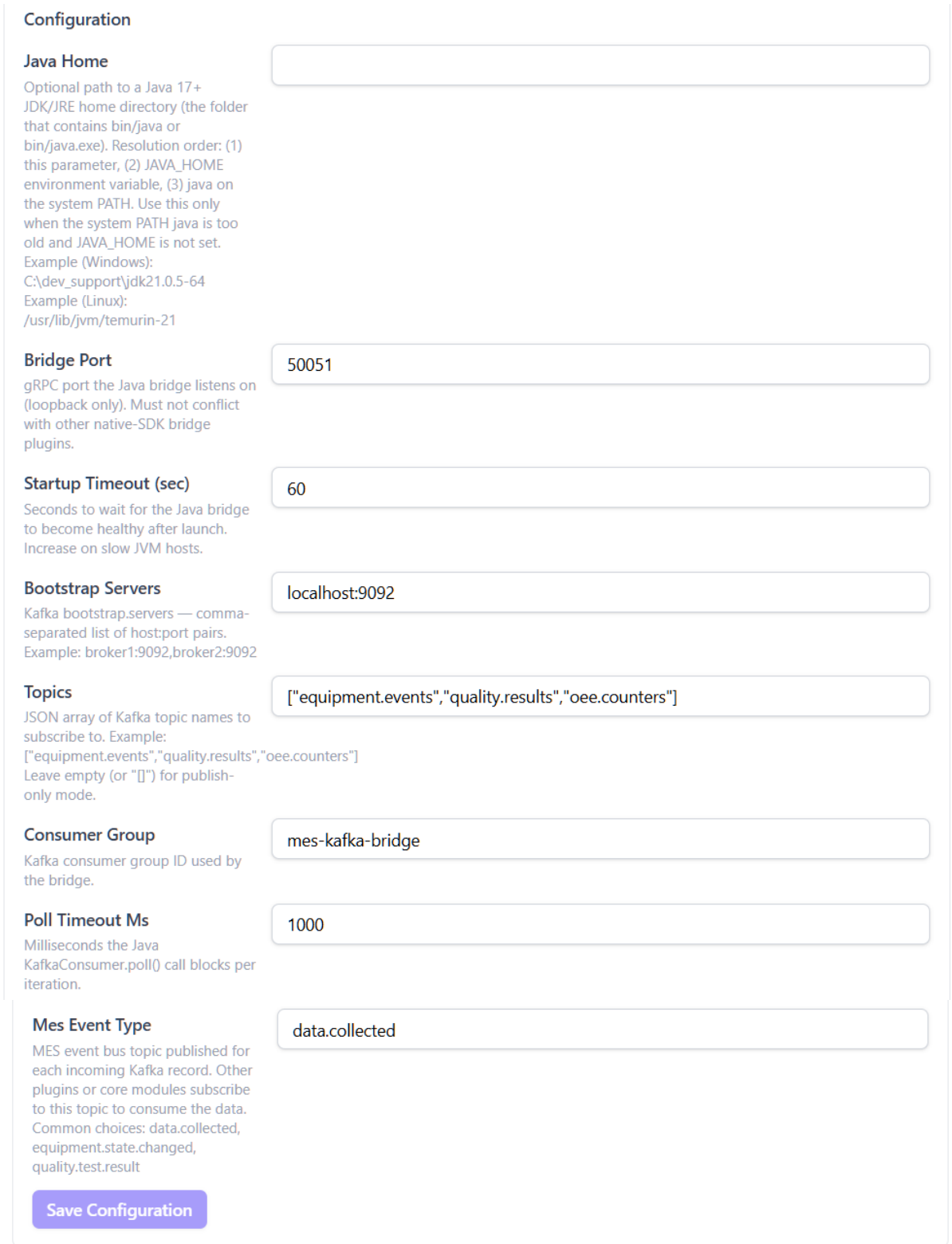
—

DEPENDENCIES

—

REQUIRED PERMISSIONS

—



The Java jar must contain the Kafka client as well as all of its dependencies (a.k.a “fat” or “shaded” jar) as specified in its pom.xml. Clicking the Rebuild button builds the jar for the SDK plugin. The Java JDK or JRE is located either by specifying the path parameter,

JAVA_HOME environment variable or the system PATH.

After the plugin is installed and enabled, clicking the Test button publishes a text message to a test topic on the configured broker and then consumes it for a round-trip verification of message handling.

Mock Plugins

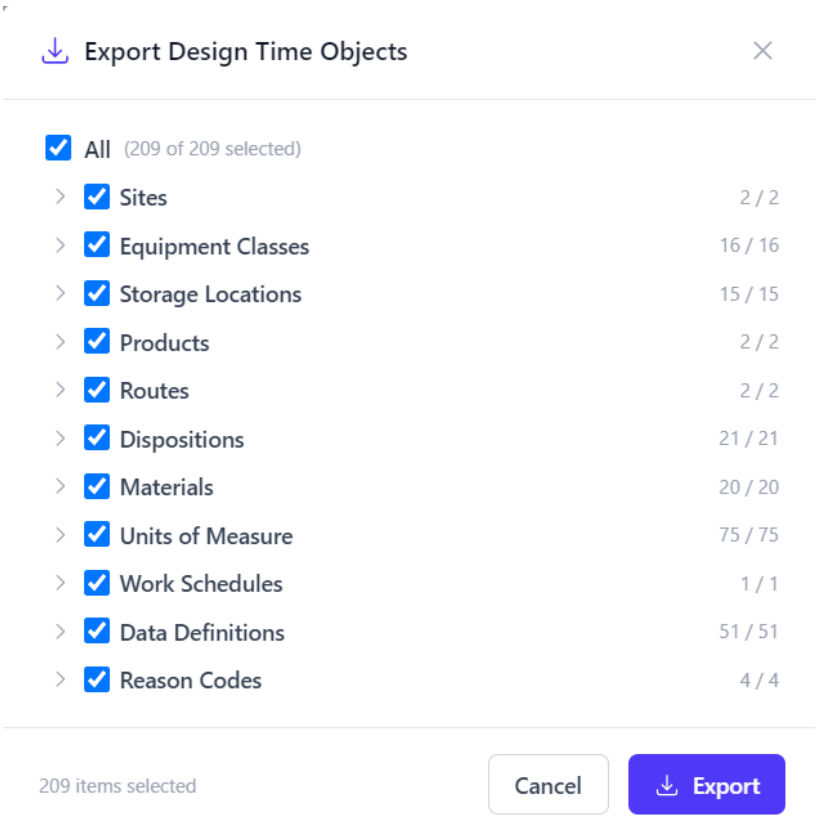
The mock plugins provide a template for further development of a plugin of the associated type.

Import/Export

Design time objects can be exported and archived as a zip file for later import into another system. For example from a development environment to an SQA environment.

Export

To begin the export, click on the Export link in the sidebar. A dialog is then displayed listing all exportable objects in a hierarchical view. For example:

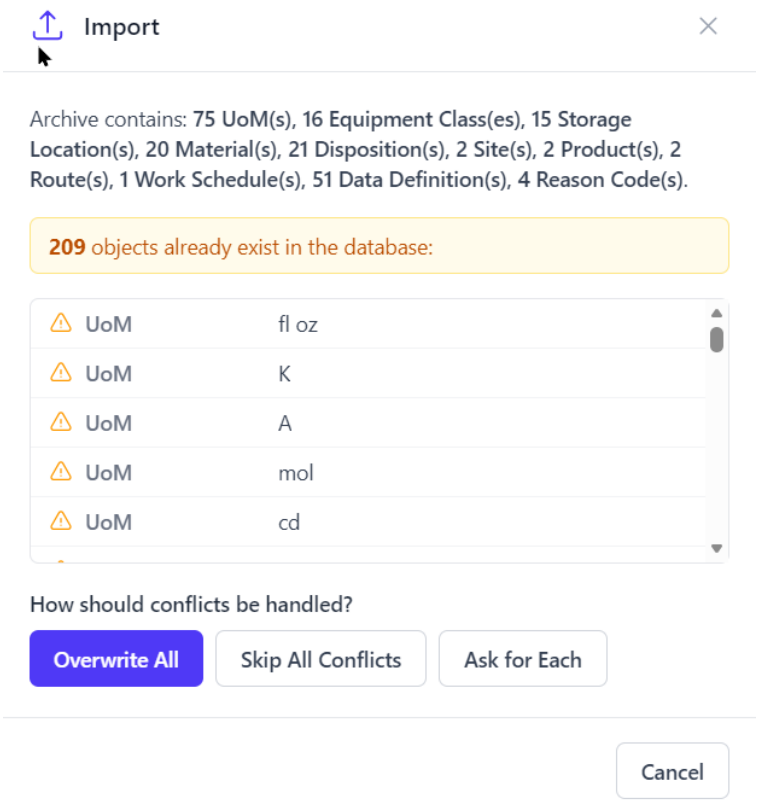


Select all objects or a combination of them by checking the checkboxes in the tree view.

Click on the Export button to select the output folder and file name of the archive.

Import

To import design time objects, click on the Import link in the sidebar. A dialog is presented to launch a file chooser dialog to choose the previously exported archive. Duplicates are then checked for and any conflicts displayed. For example:



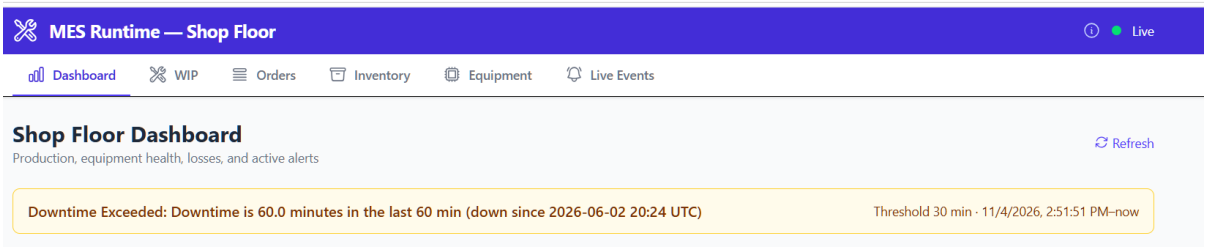
Conflicts can be handled by overwriting (updating) all, skipping all or asking the user for each such conflict. The import will then proceed to its conclusing with a summary of the objects that were imported.

Settings

The settings editor is used to change default MES properties in server, event bus and logging categories.

Runtime Client (rt-client)

rt-client is the runtime application for creating and processing WIP, dispatching work, recording inventory activity, and handling shop-floor execution. It has 6 tabs: Dashboard, WIP, Orders, Inventory, Equipment and Live Events.

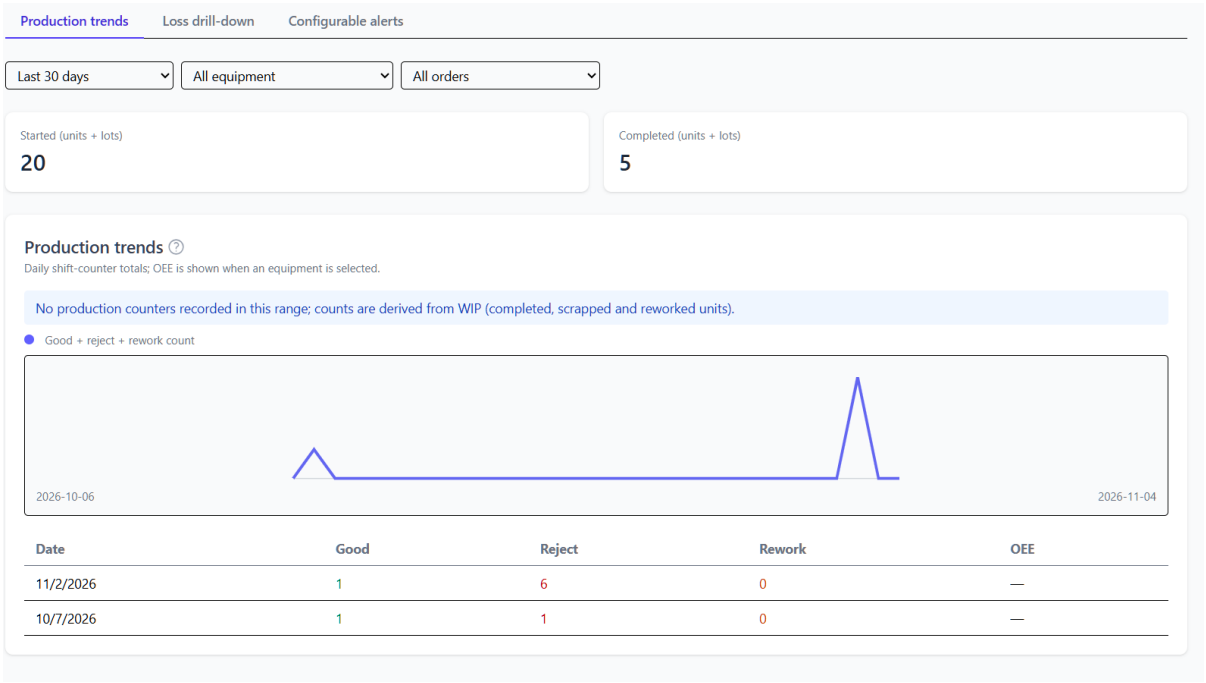


Dashboard

The Dashboard shows the progress of released orders and recent events, e.g. start and complete transactions, production metrics and active alerts.. It has three tabs:

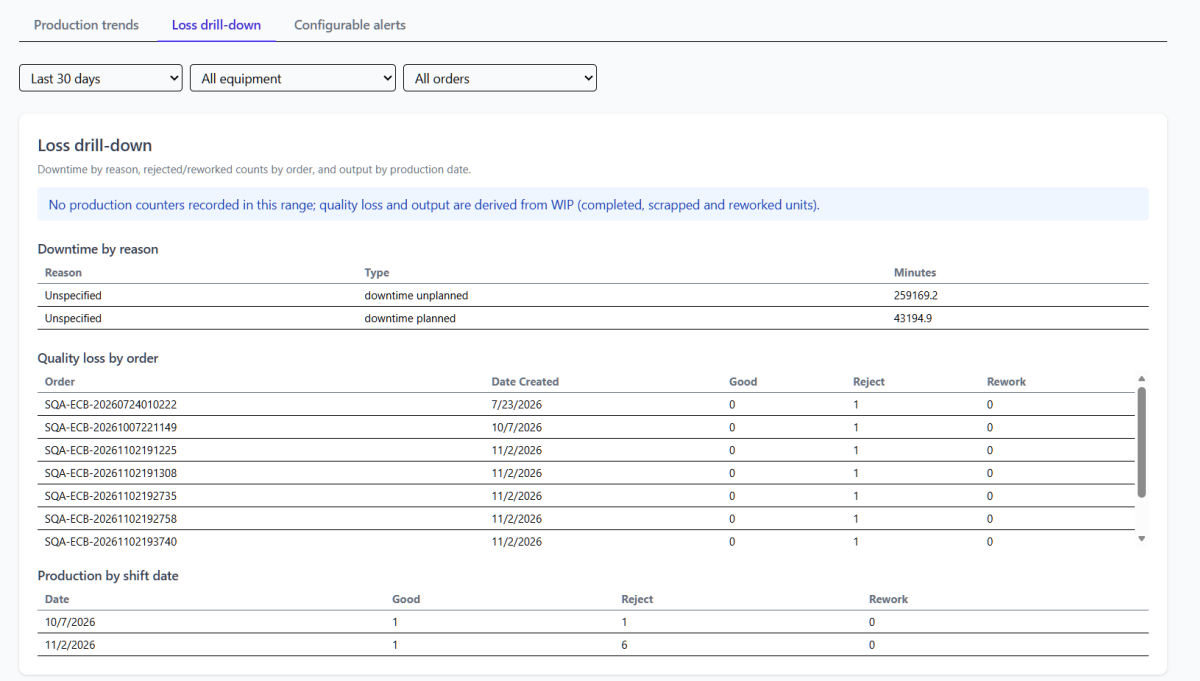
1. Production trends
2. Loss drill-down
3. Configurable alerts

Production trends



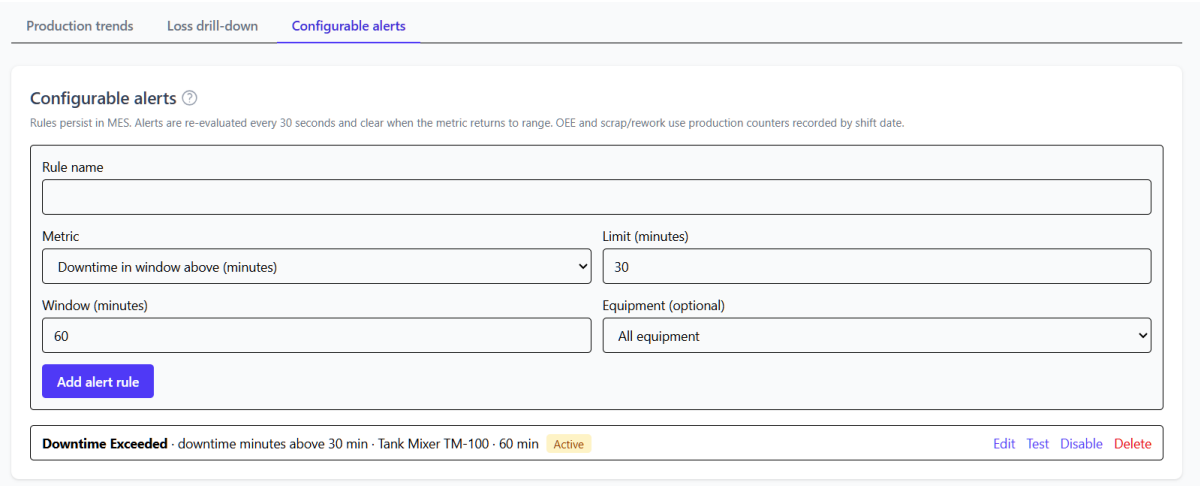
Trends are built from daily production counters (good, reject and rework counts per equipment, shift date and order). Without counters, they fall back to counts derived from WIP units, and OEE accuracy is limited.

Loss drill-down



Downtime by reason, rejected/reworked counts by order, and output by production date.

Configurable alerts



Alert rule settings

Rule name
Label shown on the alert banner.
Metric

- OEE below: OEE for the window falls under the limit (needs an equipment and recorded counters). Scrap/rework rate above: (reject + rework) ÷ total counts exceeds the limit. Downtime in window above: total downtime minutes inside the window exceeds the limit; older downtime is not counted. Current continuous downtime above: equipment has been in a downtime state, uninterrupted, for longer than the limit, measured from when the state began.

Limit

- Percent for OEE and scrap rate; minutes for the downtime metrics. The alert fires when the value is beyond the limit.

Window (minutes)

- Look-back period for OEE, scrap rate and downtime-in-window. Not used by the current continuous downtime metric.

Equipment

- Limit the rule to one machine. Leave as "All equipment" to cover everything (required for OEE).

Test / Edit

- Test dry-runs a rule with sample values; Edit changes a saved rule. Alerts are re-evaluated every 30 seconds and clear on their own.

Dispatch

Dispatch answers one question: given a unit or lot queued at a route step, which piece of equipment should process it?

Trigger

Dispatch can be invoked three ways:
- Manual — a call to POST /dispatch/evaluate, reviews ranked options, then calls POST /dispatch/execute
- Auto — POST /dispatch/auto evaluates with the specified strategy and immediately executes the top recommendation
- Event-driven — a WIP completion event (OPC-UA, MQTT, etc.) triggers auto_dispatch via the event handler

Evaluation Pipeline

1. Resolve WIP — load the unit (by serial number) or lot (by lot number); read its current_step_id
2. Find candidate equipment at the current step using ISA-95 priority:
   - SegmentEquipmentRequirement rows → specific named equipment for this step
   - equipment_class_id on the step → all equipment belonging to that class

- Neither → no options, dispatch is not applicable

3. Resolve strategy — use the strategy passed in the request, or fall back to the work cell's default_dispatch_strategy, or default to first_available
4. Filter candidates — three hard gates, in order:

Gate	Condition	Result if failed
Availability	dispatch_category == "available" (derived from PackML/SEMI E10 state)	excluded
Capability	EquipmentMaterial row exists for the WIP's material	excluded
Capacity	queue_depth < max_queue_depth (if limit set)	excluded

5. Rank eligible options by strategy (see below)
6. Return ranked options + recommended (top result) + excluded_options with reasons. If no eligible equipment: blocked=True

Strategies

Strategy	Behavior
first_available	Return equipment in natural (database) order; first one wins
shortest_queue	Sort by ascending queue_depth; routes to least-loaded equipment
round_robin	Also sorts by queue_depth as a least-recently-used proxy
capability_match	Currently treats all eligible equipment as capable (placeholder for future capability property matching
manual	No ranking; all options returned with score 0 for operator selection
custom	Natural-language prompt stored on the work cell (WorkCell.custom_strategy_prompt); currently keyword-matched (e.g. "shortest"→shortest_queue, "balance"→round_robin); designed to be replaced with an LLM call

For capability match, a production system would need to complete the algorithm.

- Load the EquipmentCapability rows for each candidate equipment where capability_type == "available" (and time bounds cover now)
- Compare the capability's property values against the step's requirements (e.g. SegmentEquipmentRequirement or a future SegmentCapabilityRequirement table) — e.g. "this step needs max_fill_rate ≥ 400; FL-400A declares 300 → exclude"
- Score/rank survivors by how well their declared property values satisfy the step requirements

Execution

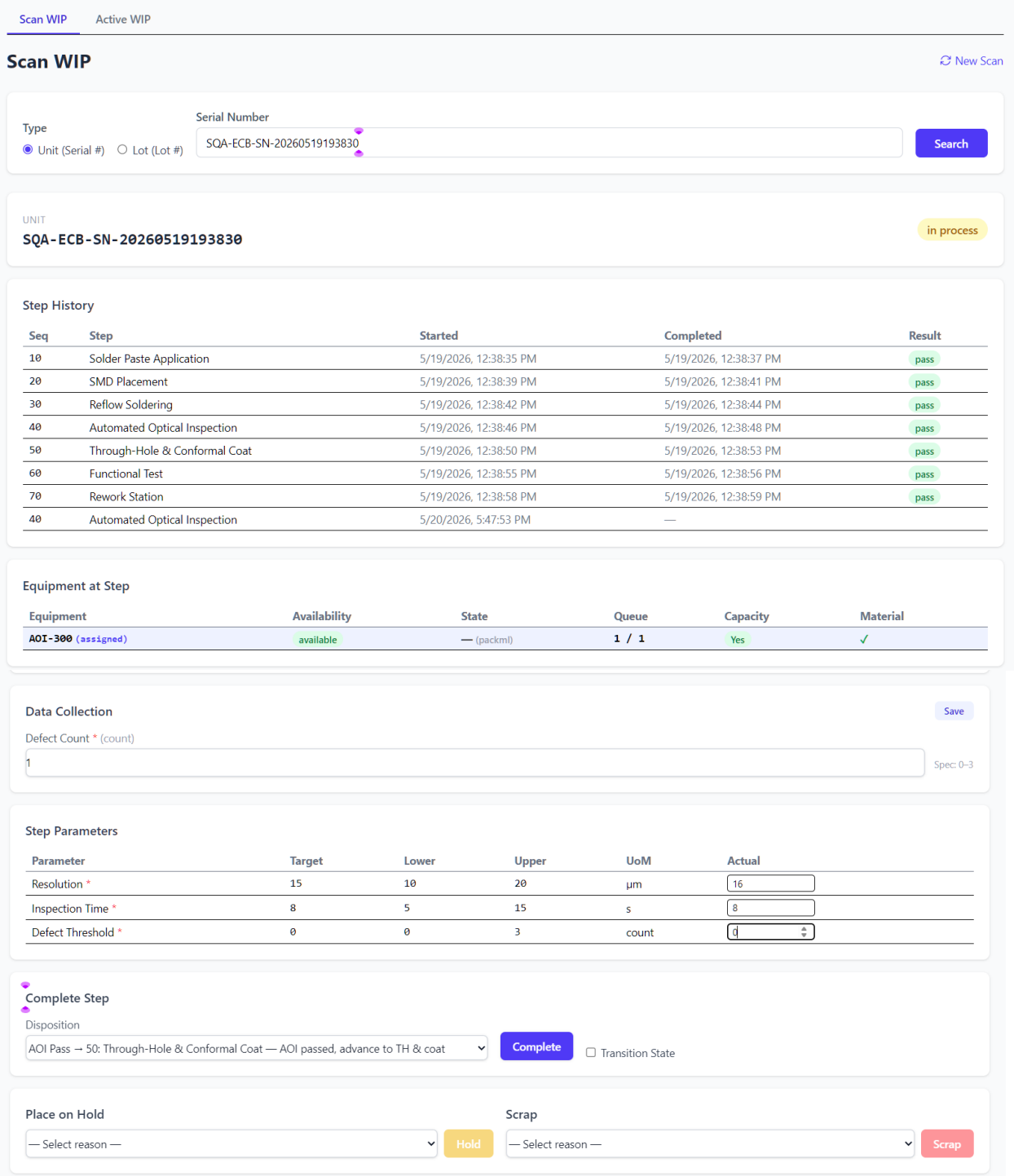
execute() re-validates availability, capacity, and material setup, checks that the target step still matches the WIP's current_step_id (prevents stale dispatches from moving WIP), then sets unit.current_equipment_id / lot.current_equipment_id. Step advancement is not touched.

Status Signals

- Blocked — no equipment passed all three gates; dispatch_blocked event emitted
- Starved — equipment is available but queue_depth == 0; equipment_starved event emitted
- At capacity — queue_depth >= max_queue_depth

WIP

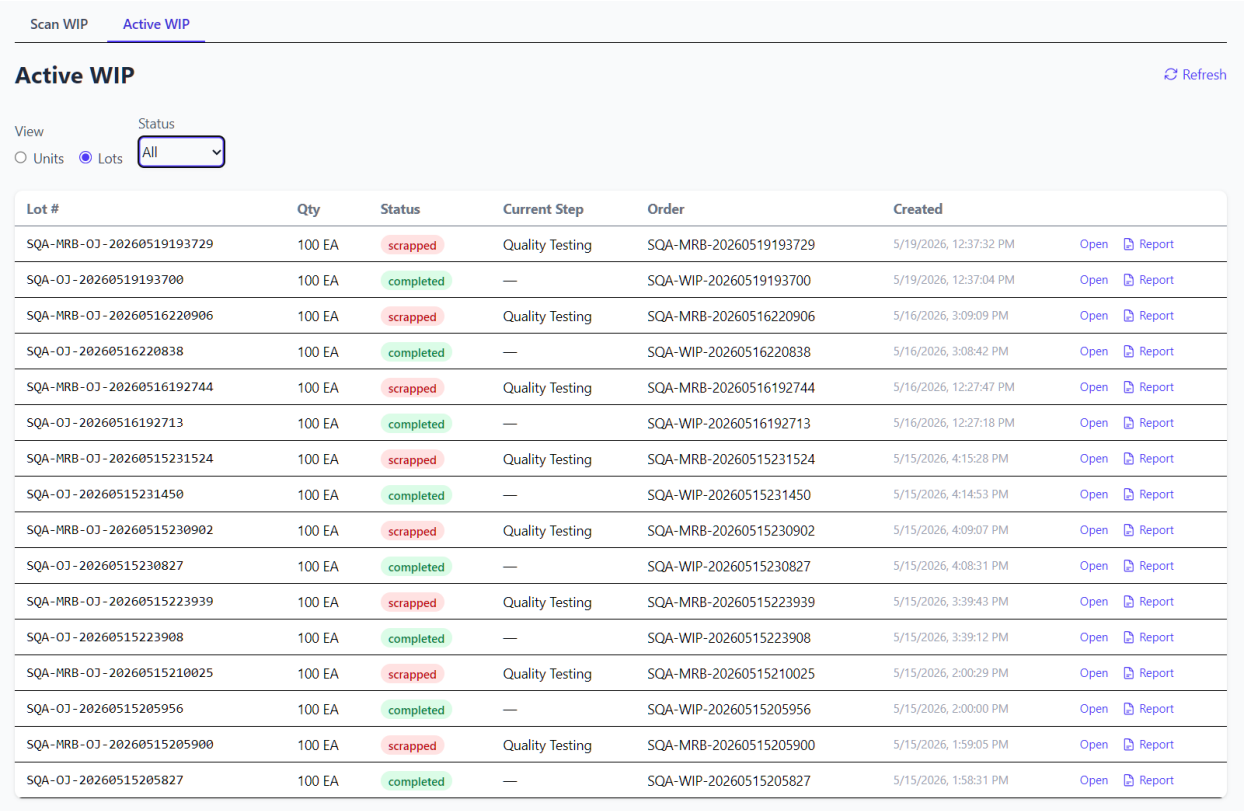
The WIP tab has two sub-tabs: Scan WIP and Active WIP. On the scan tab, a unit's serial number or lot's lot number can be entered/scanned to display information about this WIP. In the screen shot below, the UI shows an SMT unit that is started at AOI.



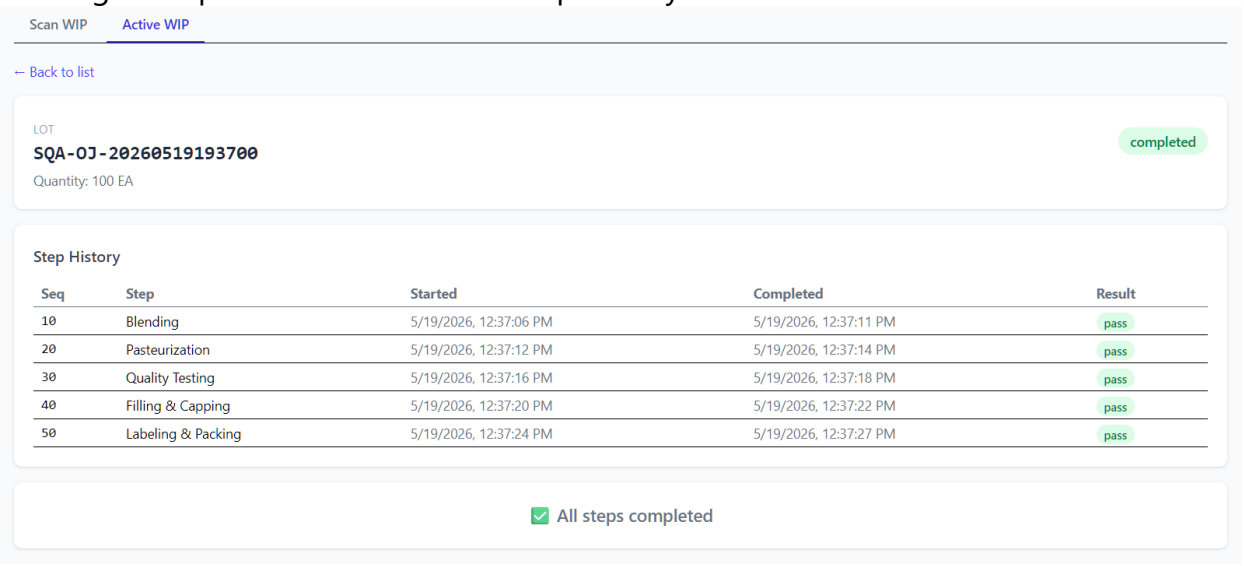
At this step, there is defect count to be collected and three step parameters to record. Clicking the Complete button will complete the unit at AOI to the step determined by the selected disposition.. If the equipment is being monitored in a state model (PackML or Semi-E10), checking the “Transition State” checkbox will first transition the equipment to the proper state before doing a start or complete transaction in order to avoid a runtime error.

The unit can also be placed on hold or scrapped here.

The Active WIP tab provides status filtering to search for units or lots. For example, for CPG lots:



Clicking the Open link shows the lot step history:



Clicking the Report link produces a WIP History Report as a pdf that can be printed or saved. For example:

Print / Save as PDF

WIP History Report

Generated: 5/21/2026, 10:50:44 AM

WIP:	**Lot — #SQA-OJ-20260519193700**	Status:	**completed**
Order:	**SQA-WIP-20260519193700**	Product:	**Premium Orange Juice 1 L**

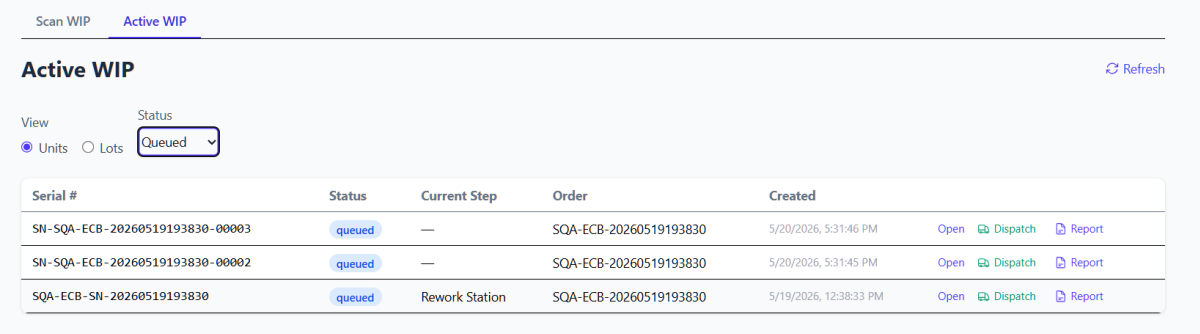
Processing History (5 steps)

Step	Equipment	Started	Completed	Duration	Result
10 — Blending	Tank Mixer TM-100	5/19/2026, 12:37:06 PM	5/19/2026, 12:37:11 PM	5s	Pass
Mix Time			15		
Mix Temperature			25		
Blend Ratio			4		
20 — Pasteurization	HTST Pasteurizer PS-200	5/19/2026, 12:37:12 PM	5/19/2026, 12:37:14 PM	2s	Pass
HTST Temperature			72		
Hold Time			15		
Exit Temperature			4		
30 — Quality Testing	Lab Analyzer QC-300	5/19/2026, 12:37:16 PM	5/19/2026, 12:37:18 PM	2s	Pass
Brix			11.5		
pH			3.8		
Microbial Count			0		
Color Index			35		
Taste Approved			true		
40 — Filling & Capping	Filler FL-400B	5/19/2026, 12:37:20 PM	5/19/2026, 12:37:22 PM	2s	Pass
Fill Volume			1000		
Cap Torque			1.2		
Headspace			15		
50 — Labeling & Packing	Labeler/Packer LP-500	5/19/2026, 12:37:24 PM	5/19/2026, 12:37:27 PM	3s	Pass
Label Aligned			true		
Date Code Legible			true		
Case Count			12		

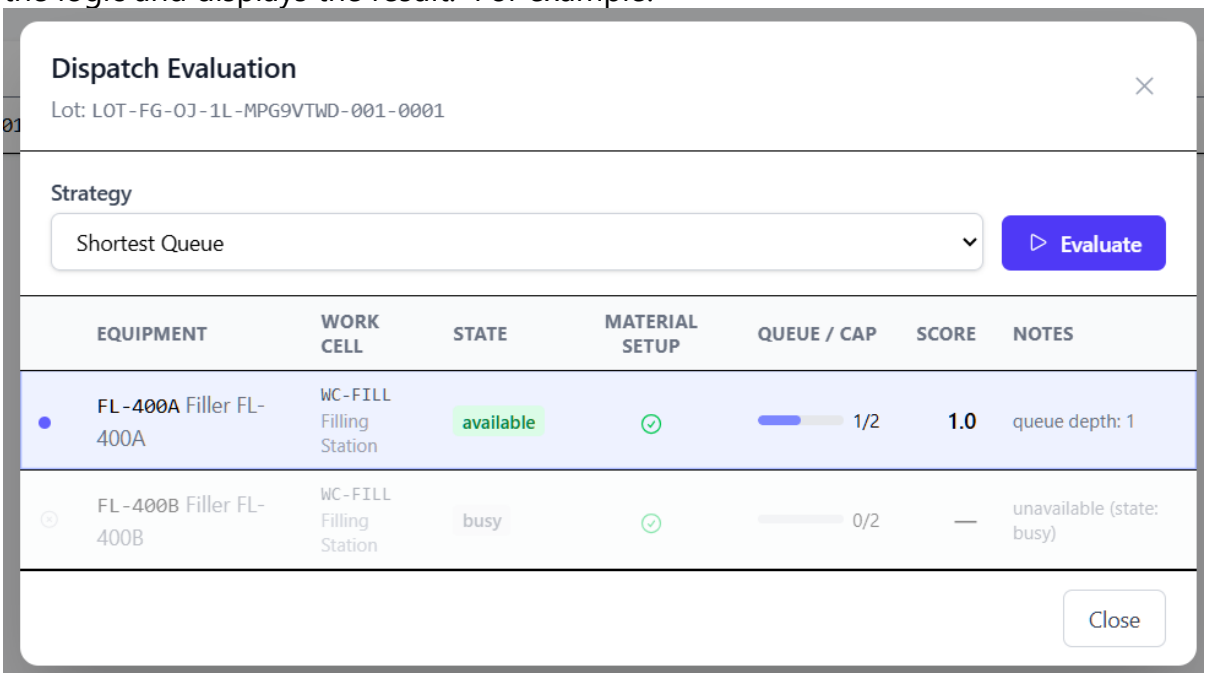
Materials Consumed (9)

Code	Name	Lot #	Qty Consumed	Consumed At
RM-OJ-CONC	Orange Juice Concentrate	OJC-2026-001	20	5/19/2026, 12:37:08 PM
RM-WATER	Purified Water	H2O-2026-001	80	5/19/2026, 12:37:09 PM
RM-SUGAR	Cane Sugar	SUG-2026-001	5	5/19/2026, 12:37:09 PM
RM-CITRIC	Citric Acid	CIT-2026-001	0.2	5/19/2026, 12:37:10 PM
RM-VITC	Vitamin C Powder	VTC-2026-001	0.05	5/19/2026, 12:37:10 PM
PKG-BOTTLE-1L	PET Bottle 1 L	BTL-2026-001	100	5/19/2026, 12:37:22 PM

For queued WIP, the dispatch algorithm can be evaluated. First find the WIP of interest then click the Dispatch button, e.g.:

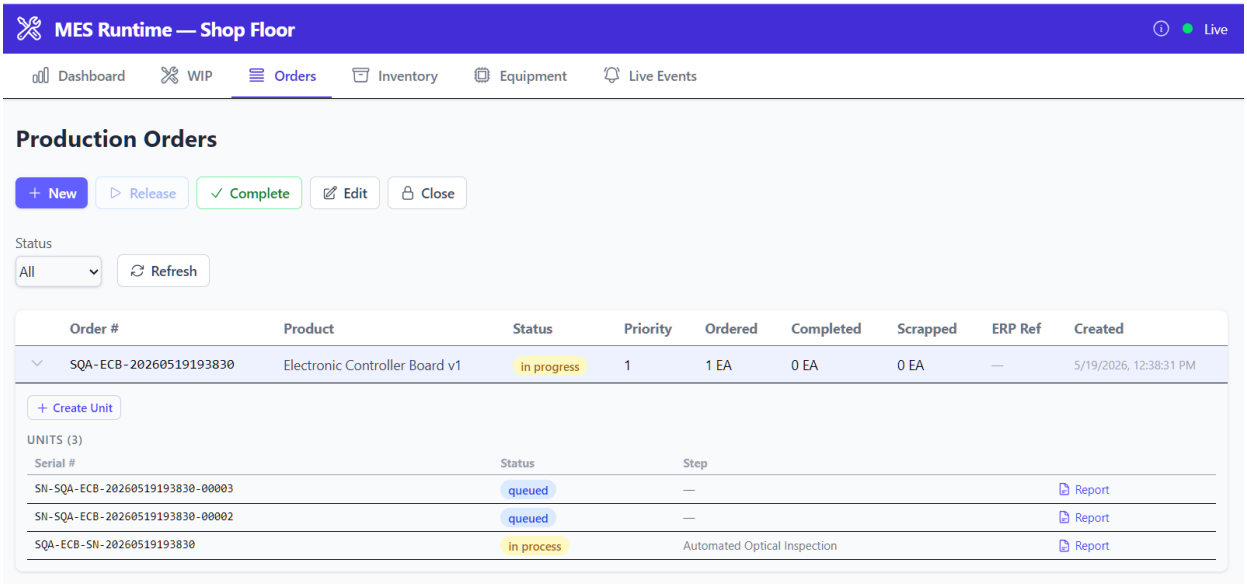


A dialog will be displayed listing the available strategies. The Evaluate button executes the logic and displays the result. For example:



Orders

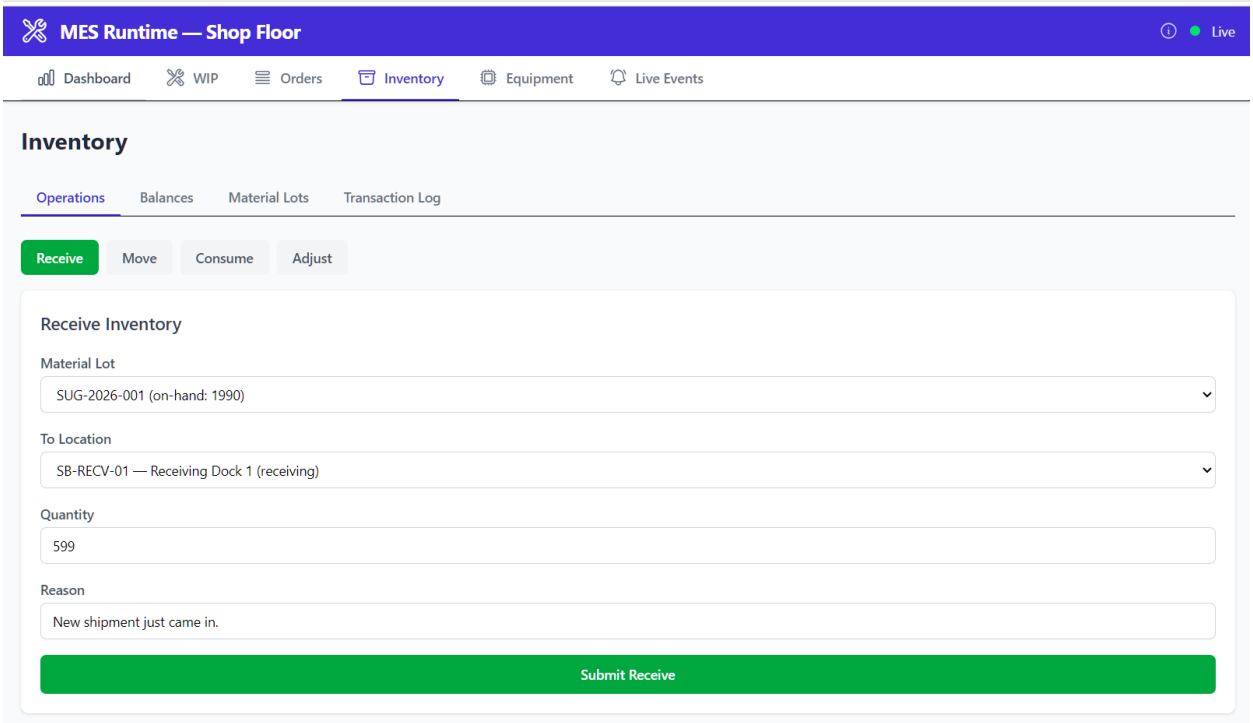
The Orders tab displays information about orders that can be filtered by status. For example:



An order can be created or edited here as well as released to production, completed or closed. Units or lots can also be created here. The Report link on a unit or lot displays the WIP History Report.

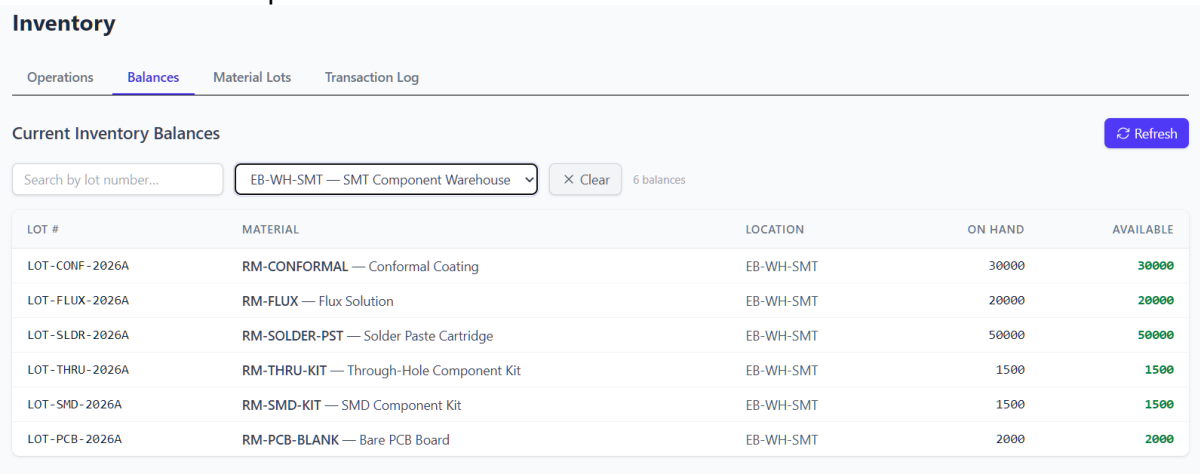
Inventory

The inventory tab allows the user to execute inventory transactions. For example, Receive in the Operations tab which creates material or adds to material to a lot:

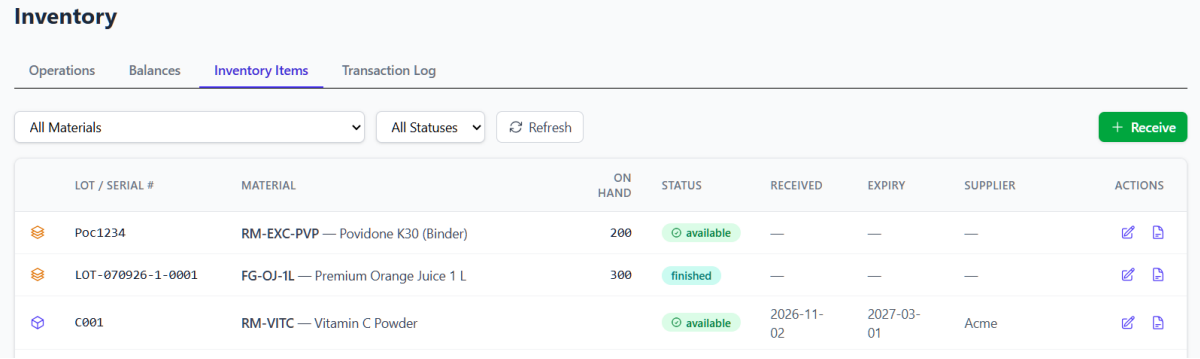


Other transactions are Move, Consume and Adjust.

The Balances tab allows the user to search for inventory units or lots by id or by location. For example:

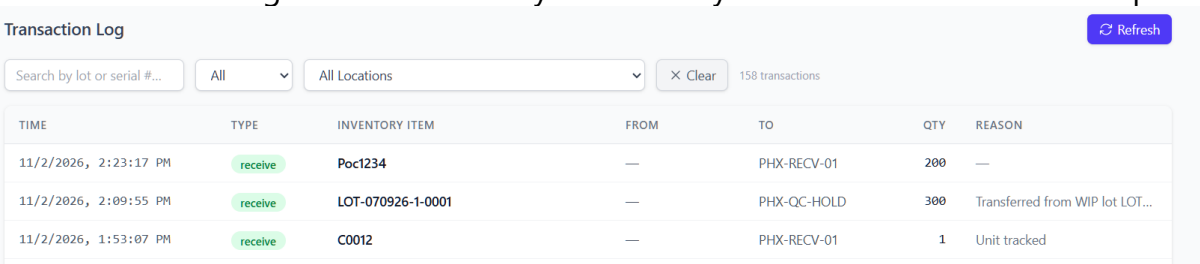


The Inventory Items tab allows the user to search for inventory items by material type or status. For example:



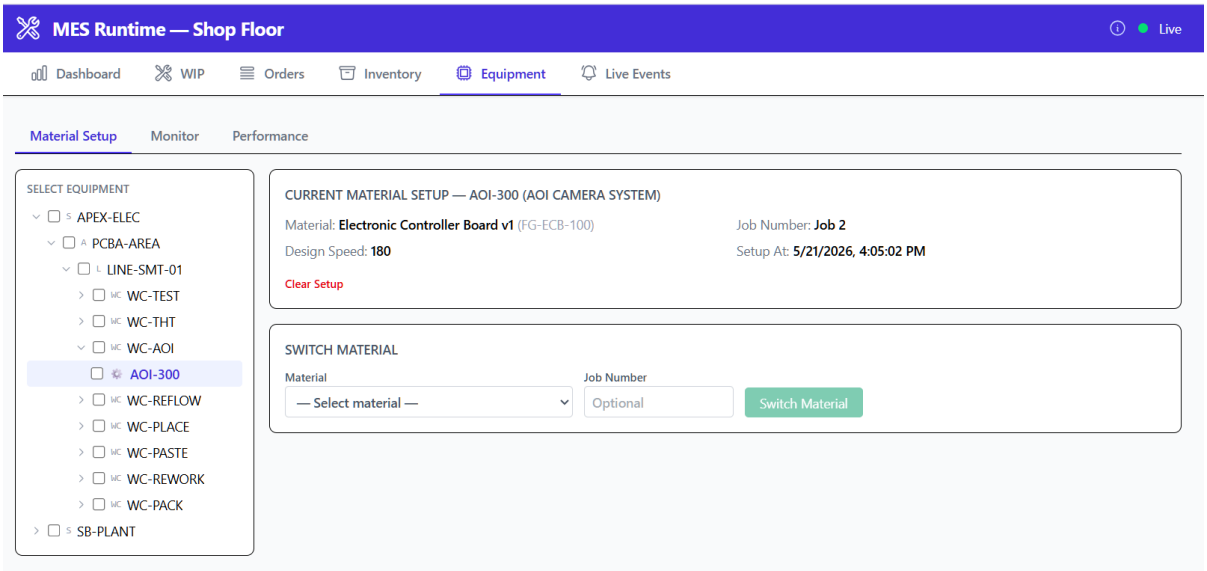
An existing inventory item's properties can be edited, or a new item can also be created (received) here. Clicking on the report icon under the Actions column creates an inventory history report of transactions on this item.

The Transaction Log tab show a history of inventory item transactions. For example:



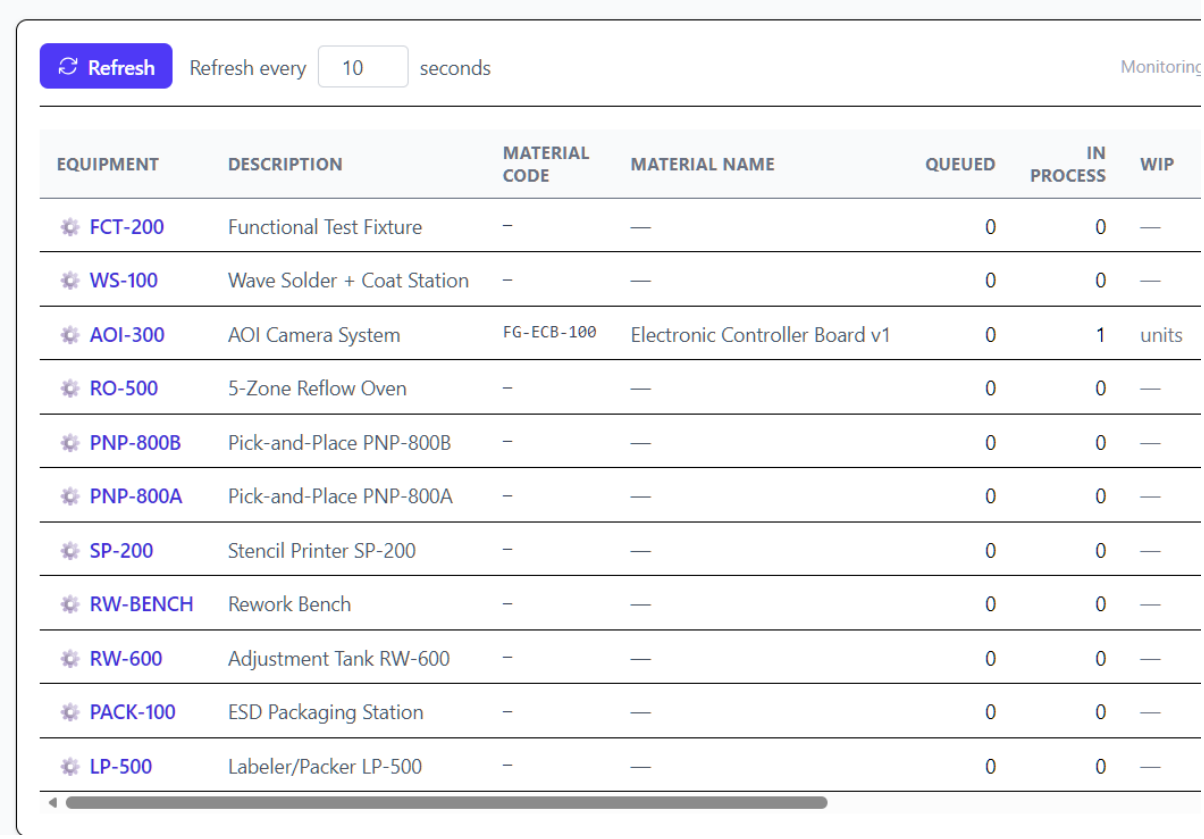
Equipment

The equipment tab allows the user to perform a material setup, and monitor and record equipment performance. For example:

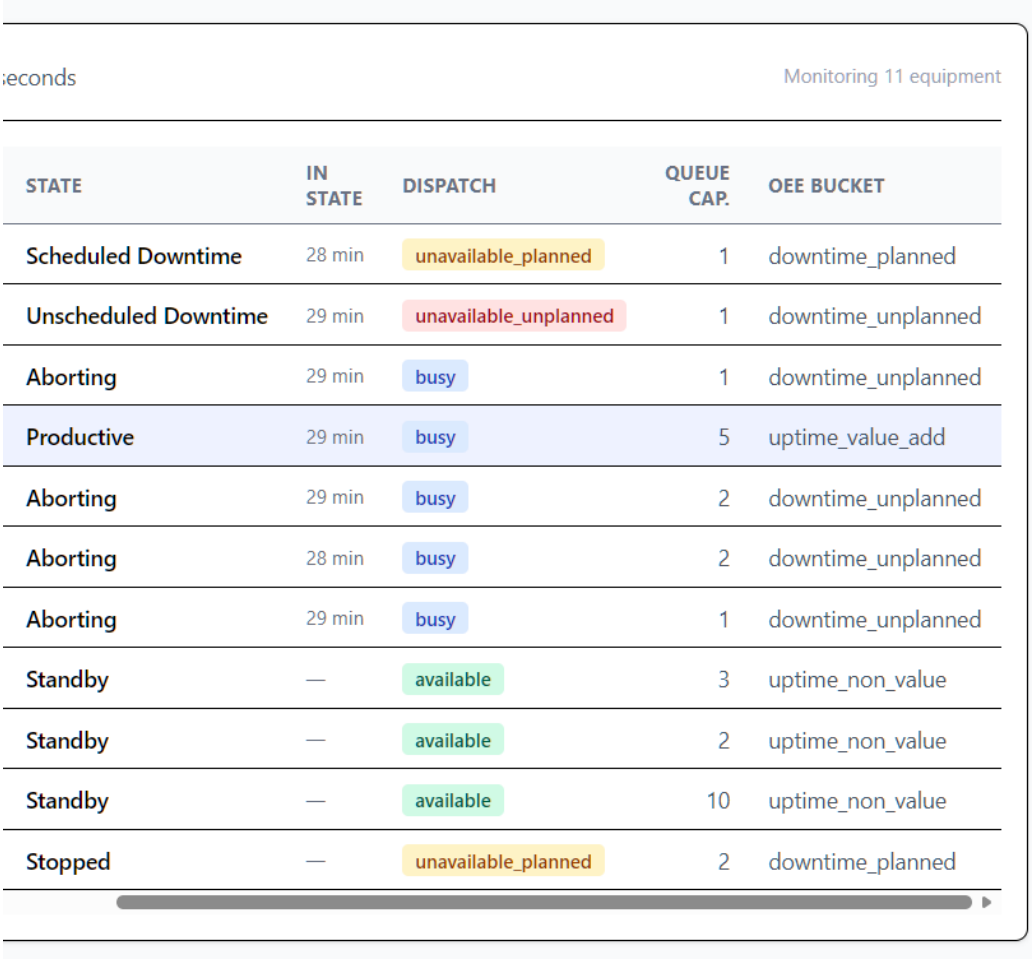


On the material setup tab, material and/or job number can be set.

On the Monitor tab, the refresh rate can be set to update the display of equipment properties. For example:



EQUIPMENT	DESCRIPTION	MATERIAL CODE	MATERIAL NAME	QUEUED	IN PROCESS	WIP
FCT-200	Functional Test Fixture	–	—	0	0	—
WS-100	Wave Solder + Coat Station	–	—	0	0	—
AOI-300	AOI Camera System	FG-ECB-100	Electronic Controller Board v1	0	1	units
RO-500	5-Zone Reflow Oven	–	—	0	0	—
PNP-800B	Pick-and-Place PNP-800B	–	—	0	0	—
PNP-800A	Pick-and-Place PNP-800A	–	—	0	0	—
SP-200	Stencil Printer SP-200	–	—	0	0	—
RW-BENCH	Rework Bench	–	—	0	0	—
RW-600	Adjustment Tank RW-600	–	—	0	0	—
PACK-100	ESD Packaging Station	–	—	0	0	—
LP-500	Labeler/Packer LP-500	–	—	0	0	—



Live Events

The Live Events tab shows events received on the event bus. They can be filtered by type. For example:



WIP Client (wip-client)

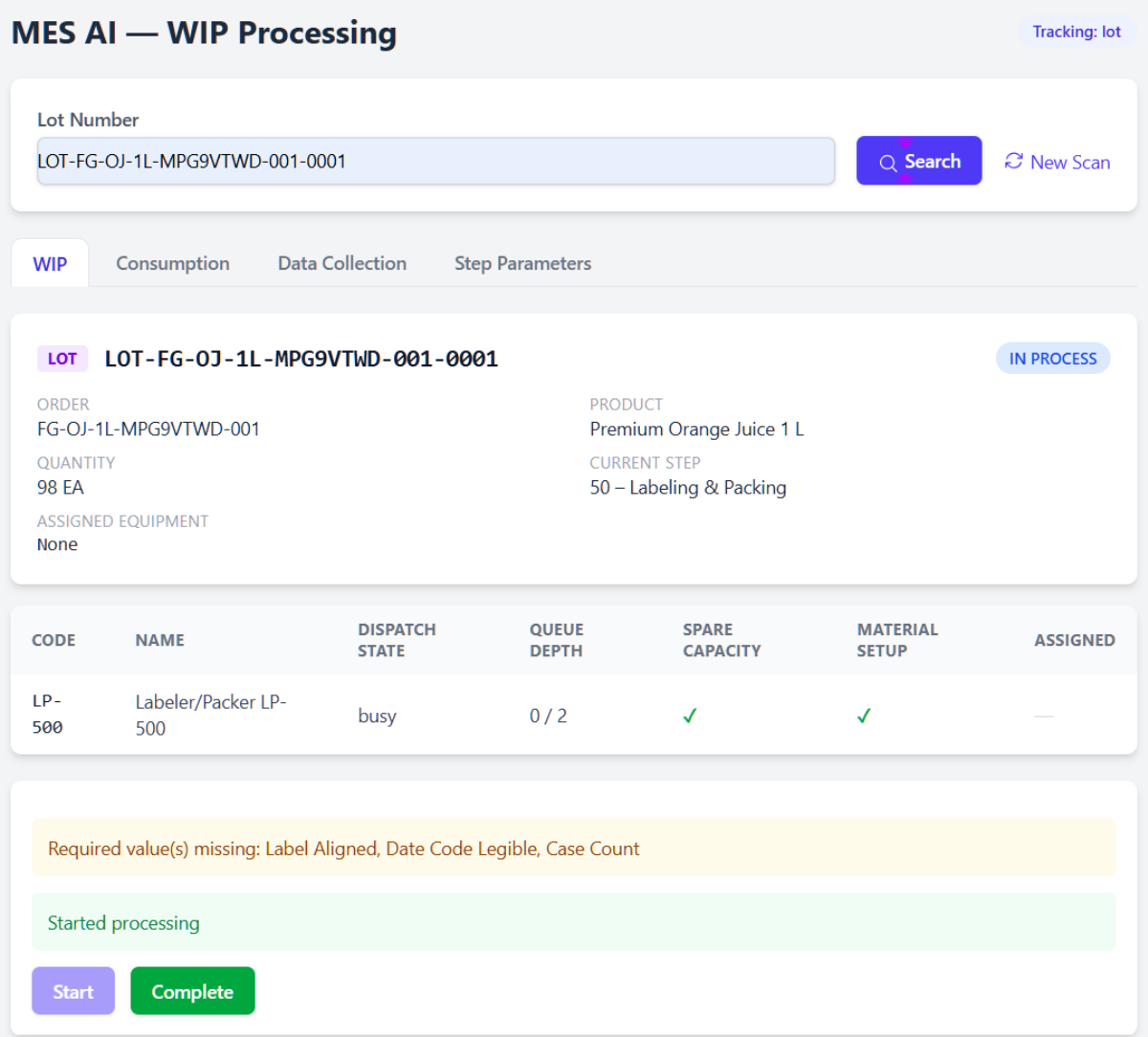
wip-client is the runtime application focused on the operator experience while working on lots and units. The client is launched from the shell by .\run-client.ps1/sh. The -WIP argument is “lot” or “unit”. If “lot” then the operator scans a lot number into the app, or unit serial number if “unit”.

The UI consists of four tabs:

1. WIP: for viewing processing information, starting and completing WIP
2. Consumption: for recording materials consumed
3. Data Collection: for collecting data values
4. Step Parameters: for recording route step parameter values

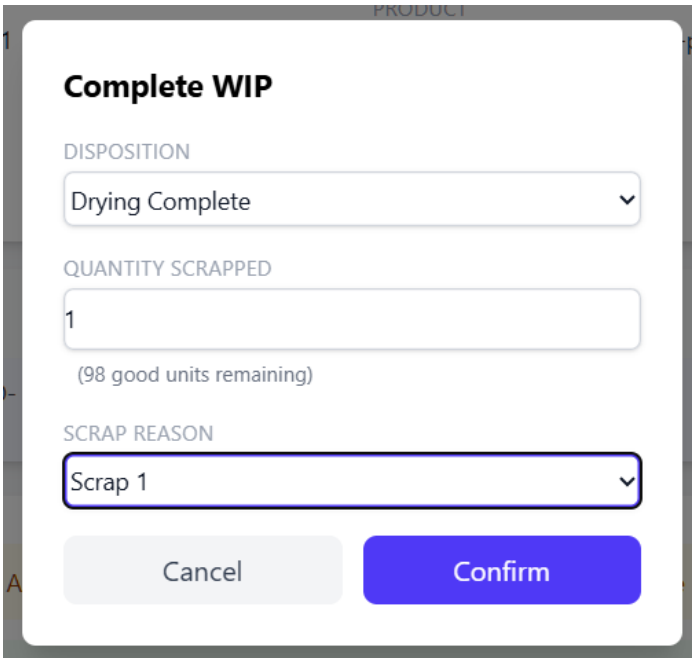
WIP

The WIP tab is similar to:



Here we have the started lot's processing information displayed along information about the equipment at the route step. Click the Start button to start processing WIP.

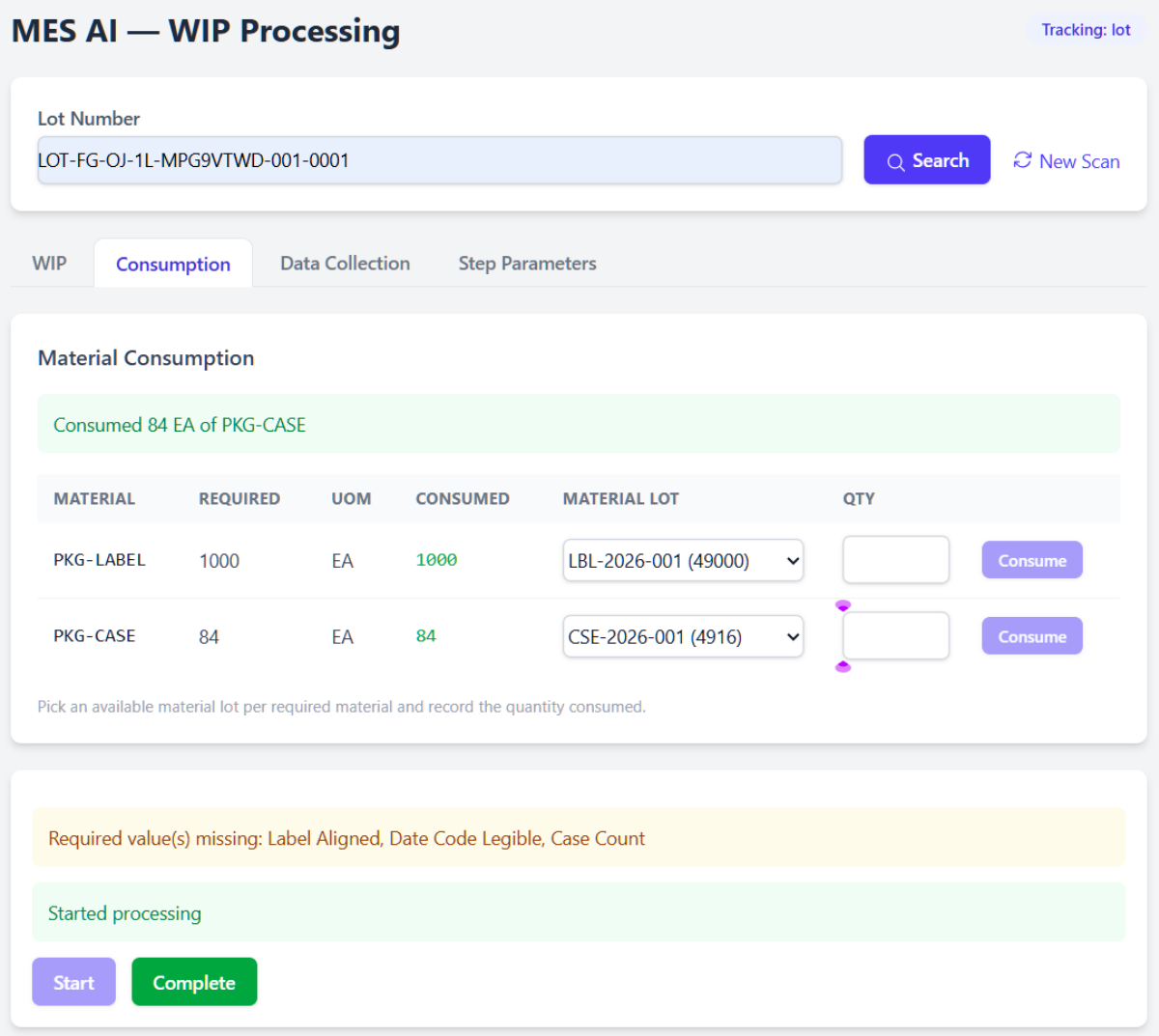
When all required consumption, data collection and step parameters have been statisfied, the operator can complete the WIP by clicking on the Complete button. A dialog similar to the following is displayed.



A disposition must be selected if not on the last step of the route. If a scrap quantity is entered, the scrap reason must also be chosen.

Consumption

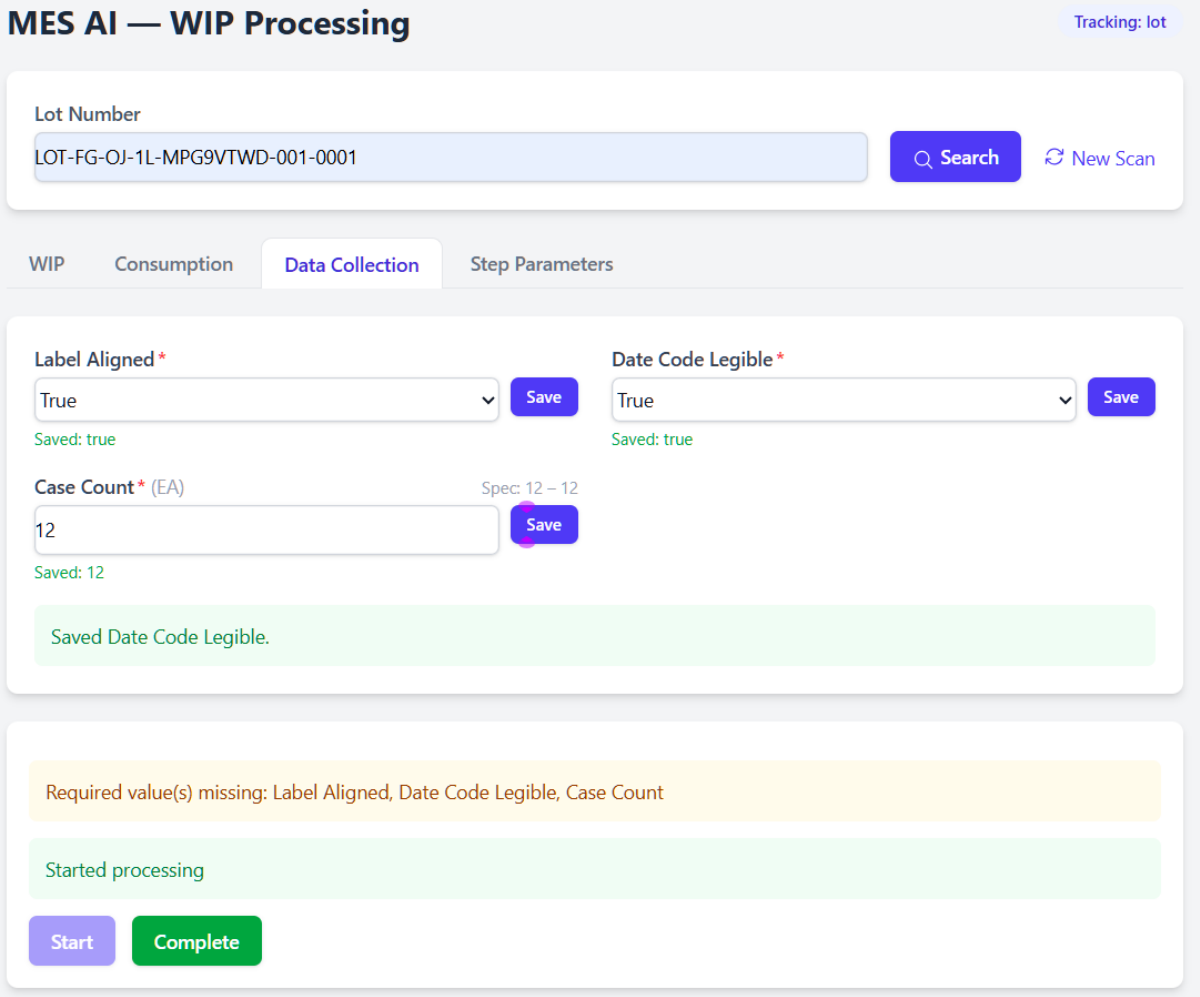
The consumption tab (which is visible only if material consumption has been defined) is similar to:



Here the operator has recorded consumption from two material lots.

Data Collection

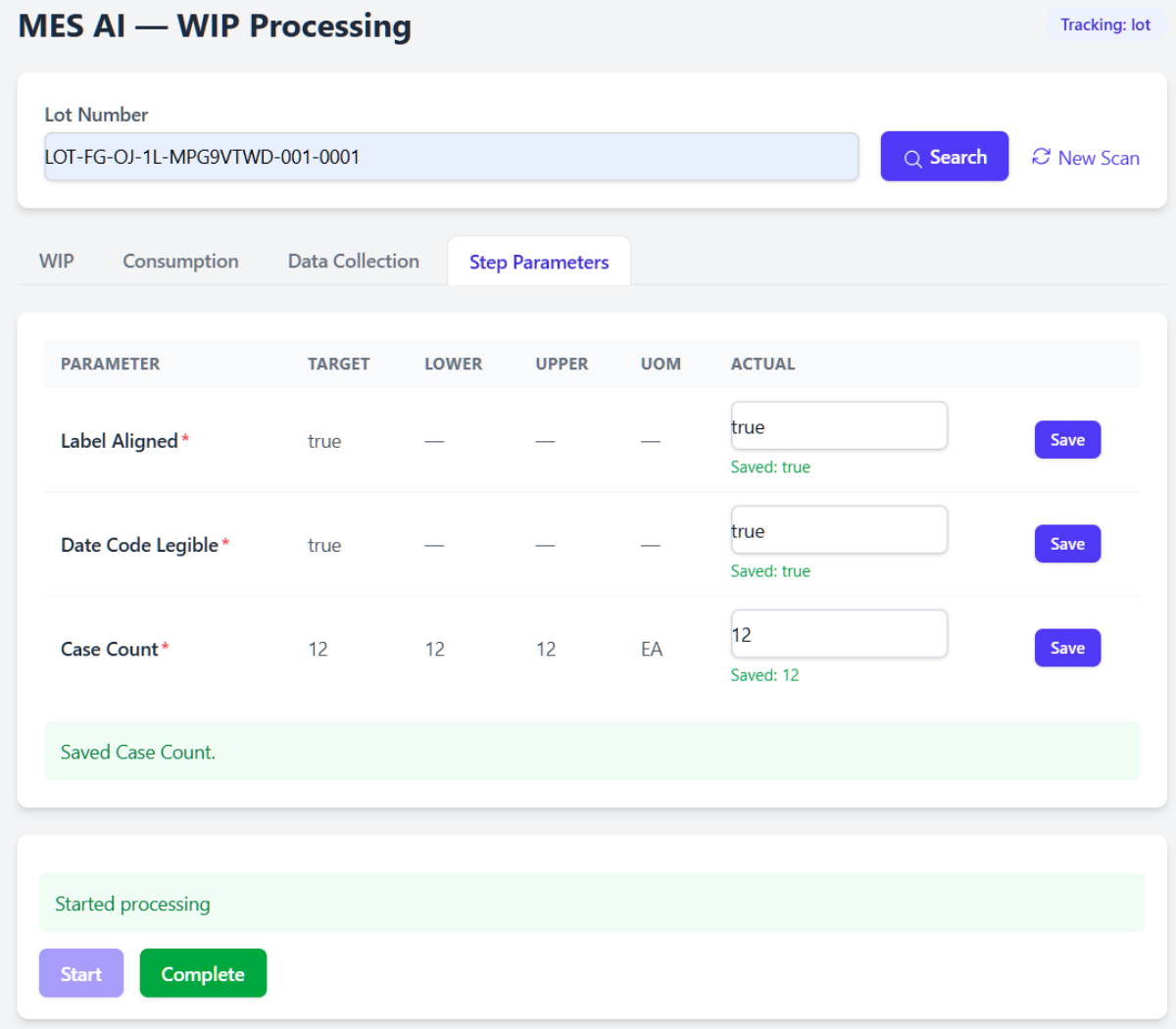
The data collection tab (which is visible only if data collection has been defined) is similar to:



Here the operator has collected data values for three required items.

Step Parameters

The step parameters tab (which is visible only if route step parameters have been defined) is similar to:



Here the operator has collected parameter values for three required items.

Inventory Client (inv-client)

The inv-client is a runtime application focused on the operator experience while working on material lots and units at storage locations. The client is launched from the shell by .\run-client.ps1/sh inv-client.

The UI consists of two tabs:

1. Operations: for receiving, transferring, adjusting or shipping inventory items
2. Transaction Log: for viewing inventory item transactions over a specified period

Operations

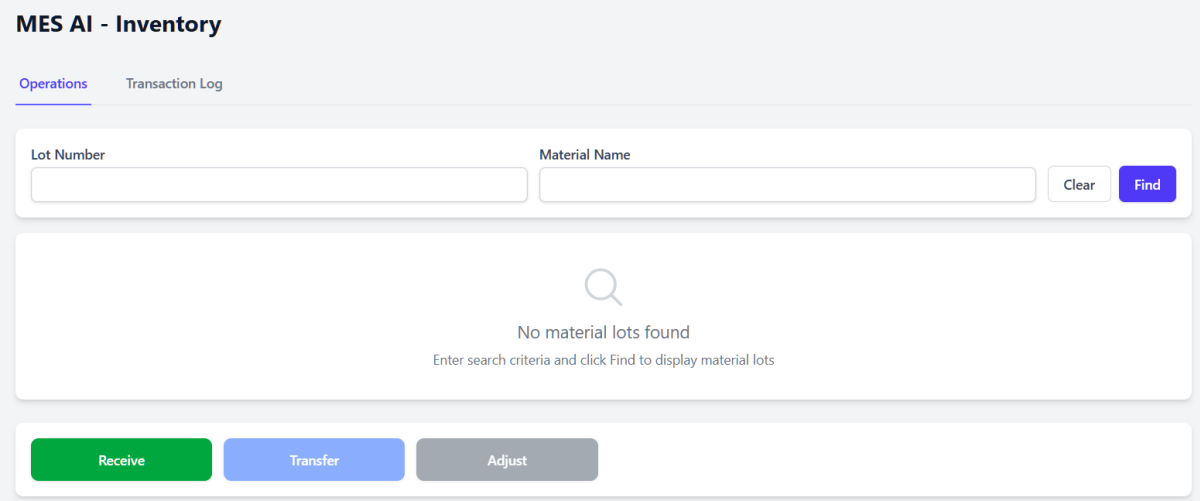
The operations tab has four transactions:

- Receive: create an inventory lot or unit or add material to an existing lot

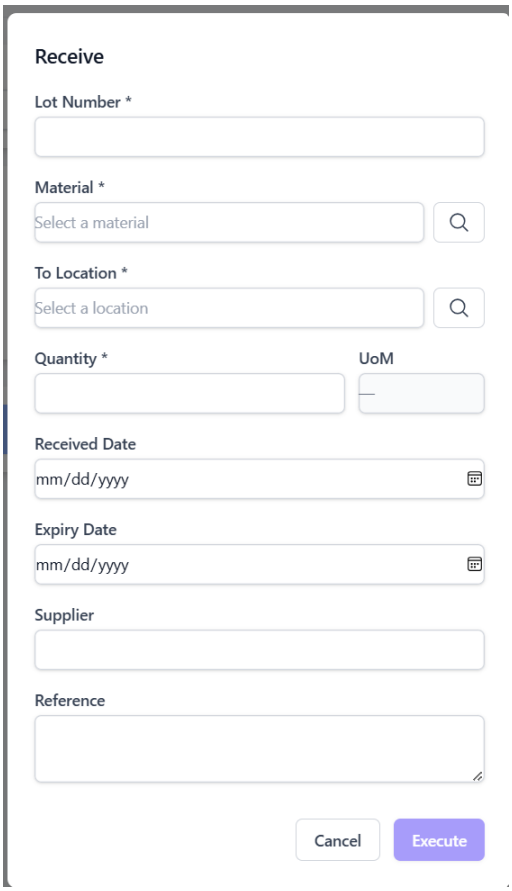
- Transfer: put-away, pick or move an inventory item from one location to another
- Adjust: change the quantity of a lot
- Ship: execute a ship transaction on a finished WIP lot that has been transferred off of a route. Shipped inventory items are no longer visible.

Receive Lot

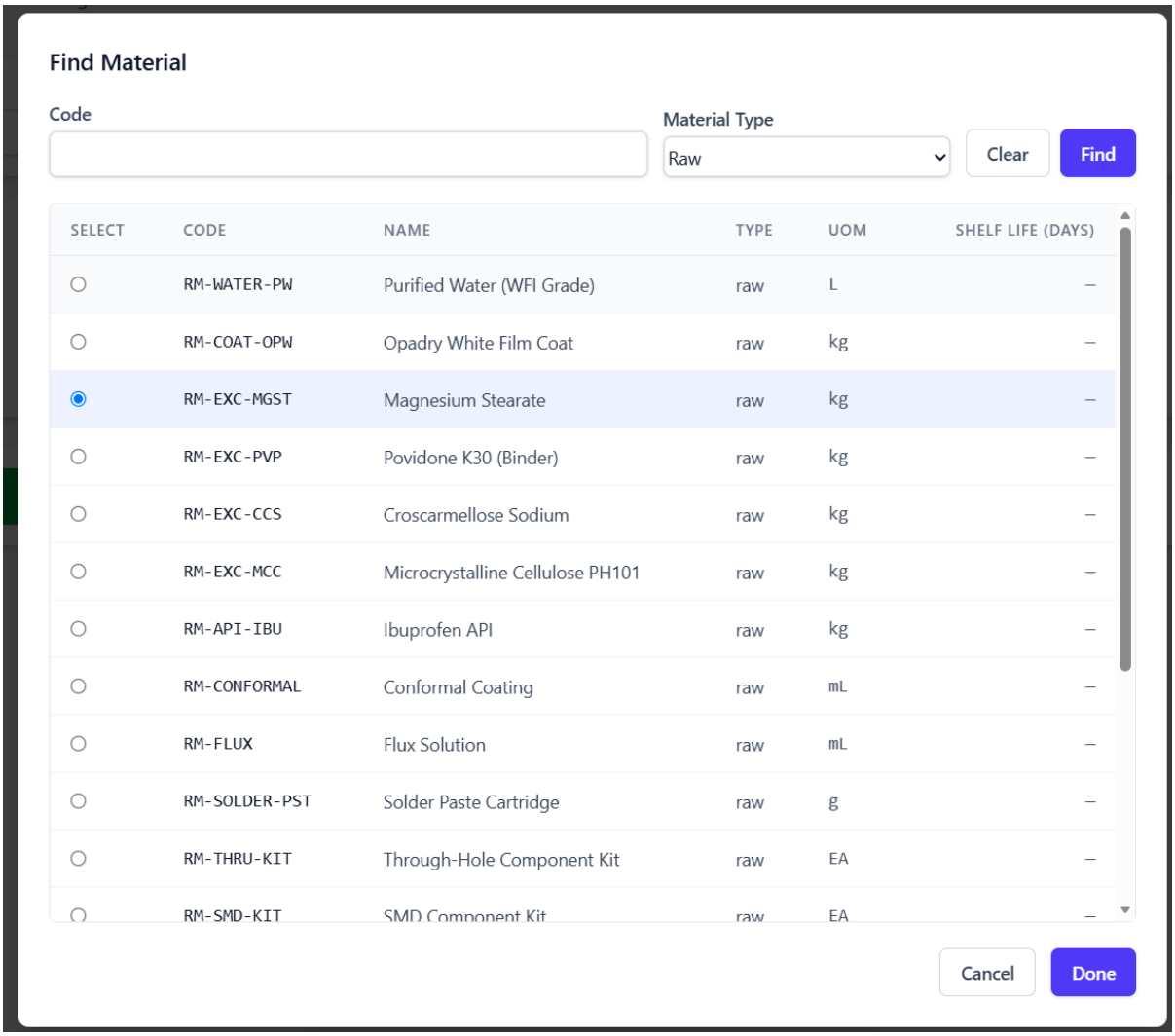
To create an inventory lot, start with no previous lot selected by clicking on the Clear button to reset the lot number and/or material name search fields:



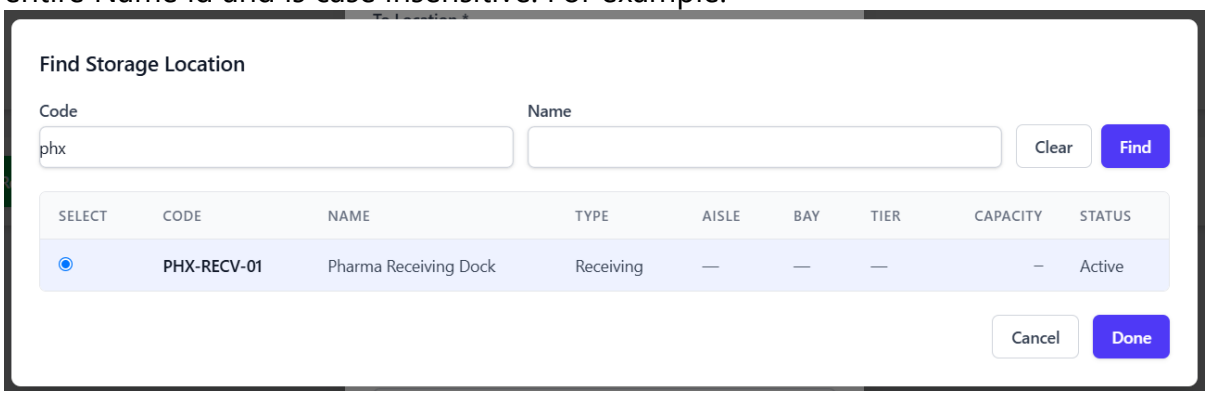
Click Receive to display the following dialog:



- Lot Number: Enter the lot number
- Material: Click the search button to display a dialog to find the material of the lot. The code text is a partial or the entire Code id and is case insensitive. The material type can be Raw, Intermediate or Finished. Click the find button to display results and then select the material. For example:

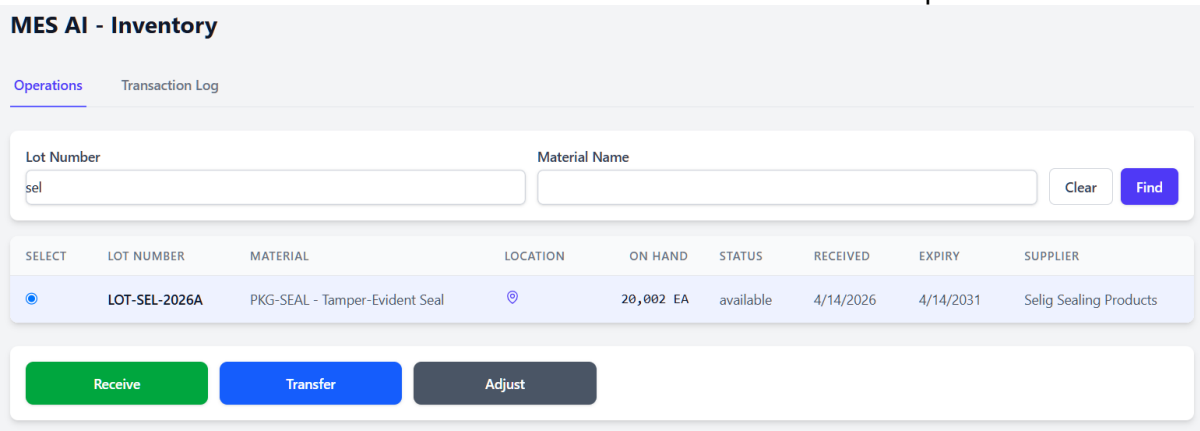


- To Location: Click the search button to display a dialog to find the receiving location of the lot. These locations must be created with type “Receiving”. The code text is a partial or the entire Code id and is case insensitive. The Name text is a partial or the entire Name id and is case insensitive. For example:

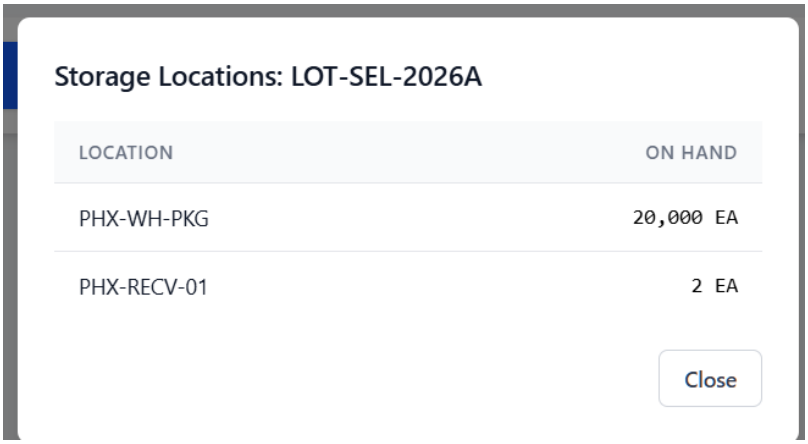


- Quantity: Enter the lot quantity for the material's unit of measure.
- Received Date: Pick the date the lot was received.
- Expiry Date: Pick an expiration date of the lot. If a shelf like is defined, it will default to the received data plus the number of shelf like days.
- Supplier: Enter the supplier
- Reference: Enter an optional PO number, comment or other text

To add more material to an existing lot, first find the target lot by entering the lot partial or full name (case insensitive) and/or the material partial or full name (case insensitive). Then click the Find button and select the lot of interest. For example:



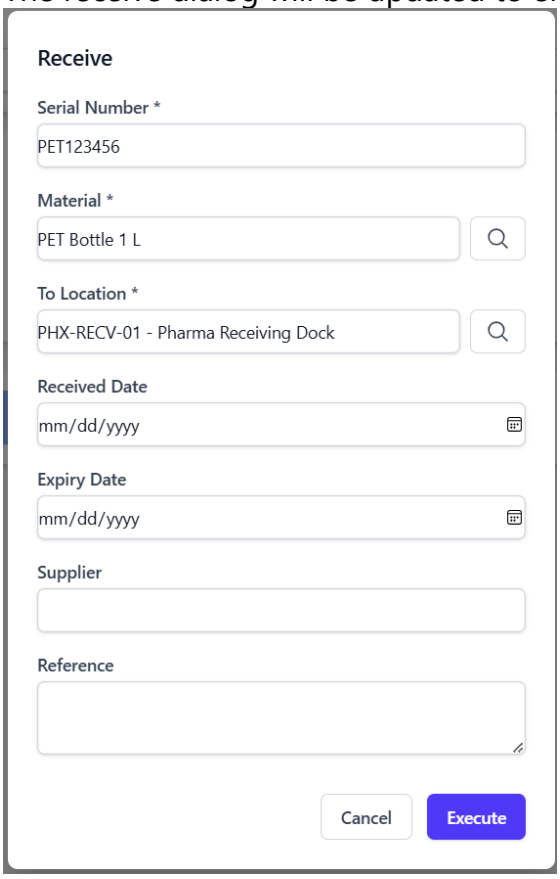
Click the location's icon button to see how much quantity is at each location:



Click the Receive button and fill out the destination location and quantity for the additional amount.

Receive Unit

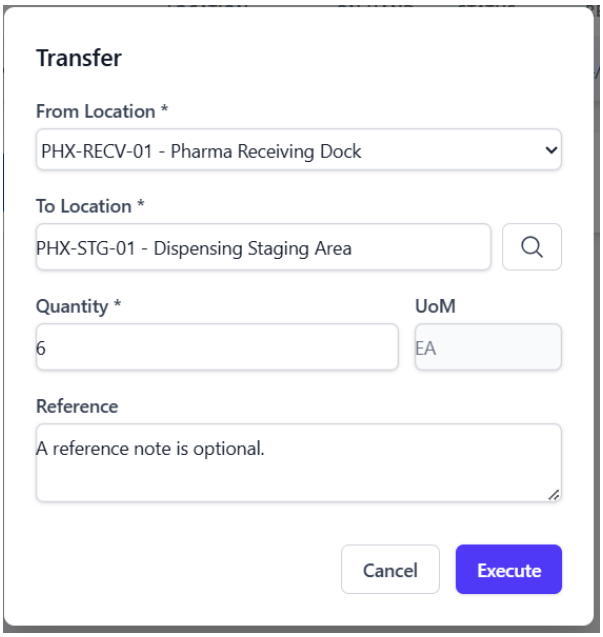
To create an inventory unit, select a material that has a unit of measure of each (“EA”). The receive dialog will be updated to enter a serial number and no quantity or UOM:



Transfer

A put-away, pick or inventory move is implemented as a transfer between storage locations. First select the material lot, then click the transfer button to display the transfer dialog.

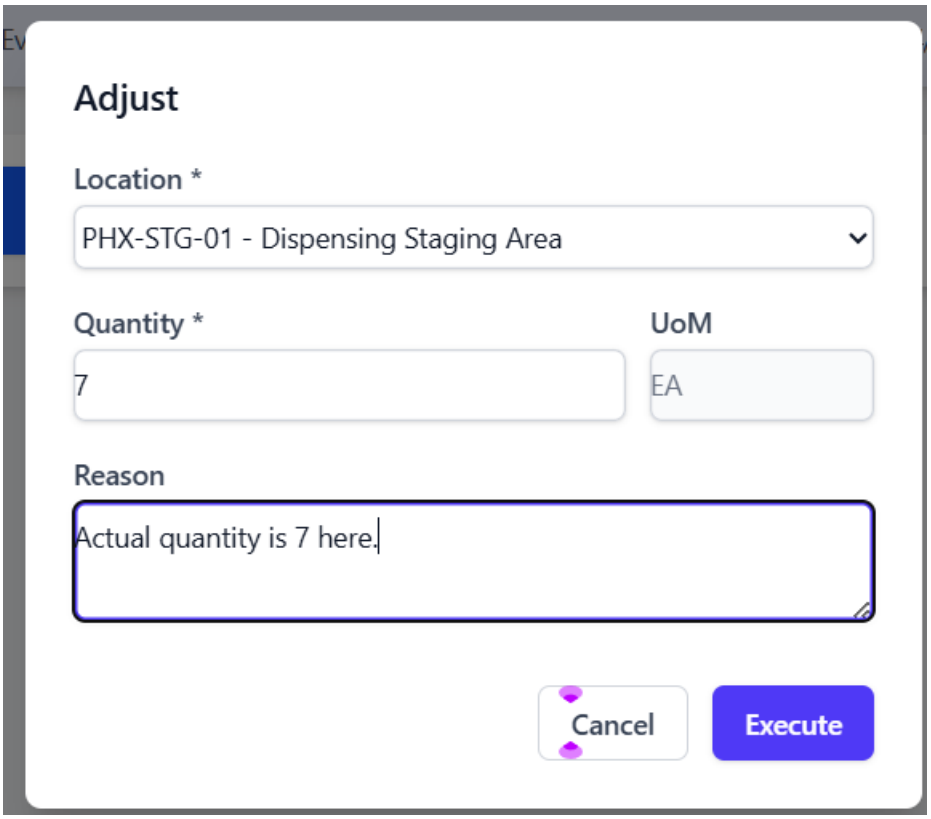
If the target lot is at more than one location, choose the from location in the dropdown. The quantity to be transferred will default to the entire amount. Click the destination location search button to find the destination location. The location types are restricted to storage, RIP and staging. Provide reference note if desired. For example:



Adjust

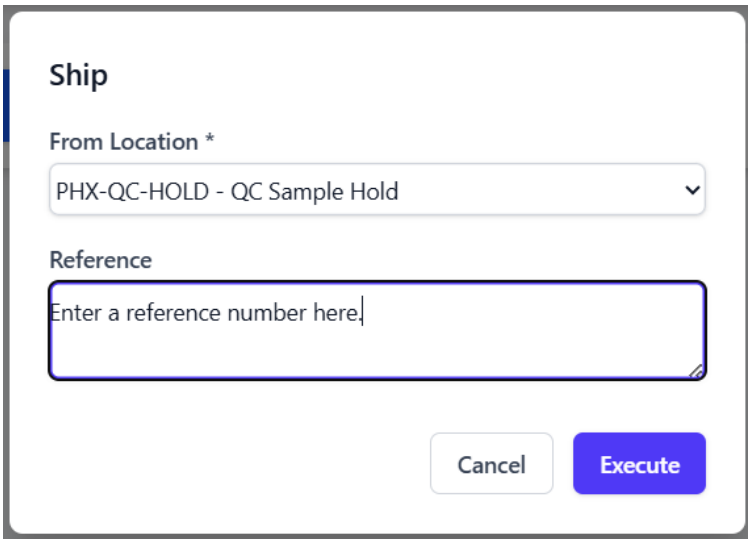
The actual quantity at a location can be adjusted as the result of a physical inventory for example. First select the material lot, then click the adjust button to display the adjust dialog.

If the lot is at more than one location, select the location to adjust, then enter the correct amount. For example:



Ship

A WIP lot or unit completed from a route step and transferred to a storage location can be shipped. Clicking the Ship button displays a dialog similar to:



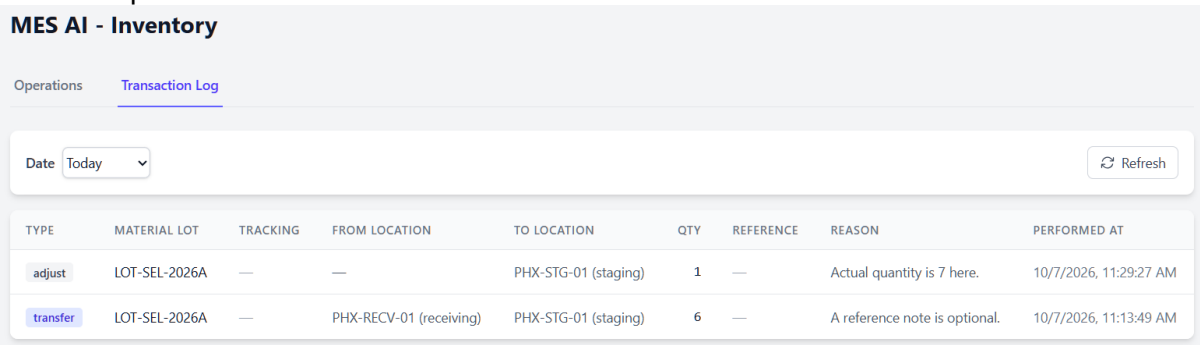
Once shipped, the inventory item no longer appears in inventory.

Transaction Log

Historical inventory lot transactions over a relative time period can be viewed on the transaction log tab. The choices are:

- Today
- Yesterday
- This Week
- Last Week
- This Month
- Last Month

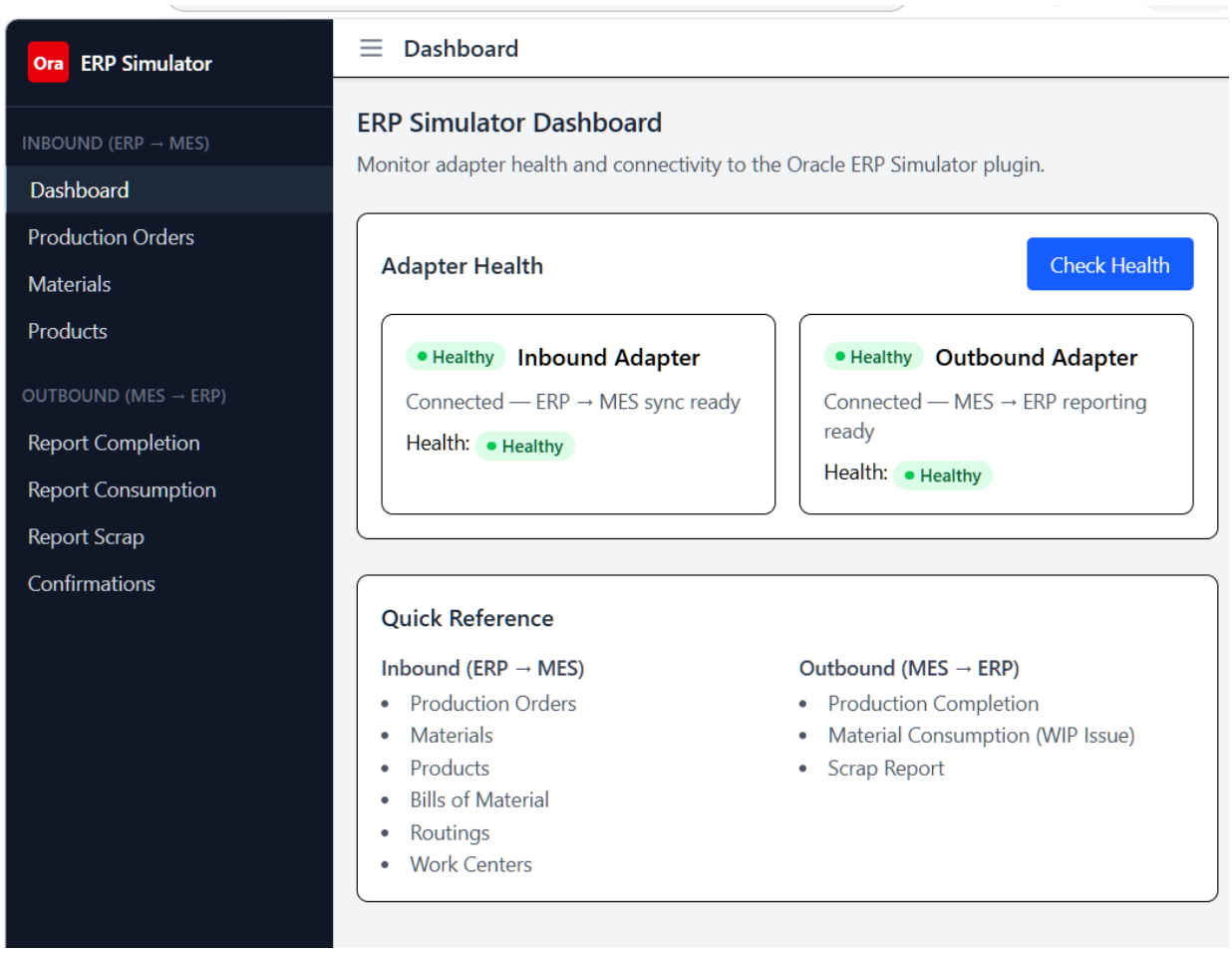
For example:



ERP Simulator (erp-sim)

erp-sim is a simulator used to create and release operations requests and exercise ERP-style inbound and outbound touchpoints. It is installed as an optional plugin for Oracle or SAP. Only one ERP plugin can be enabled at a time.

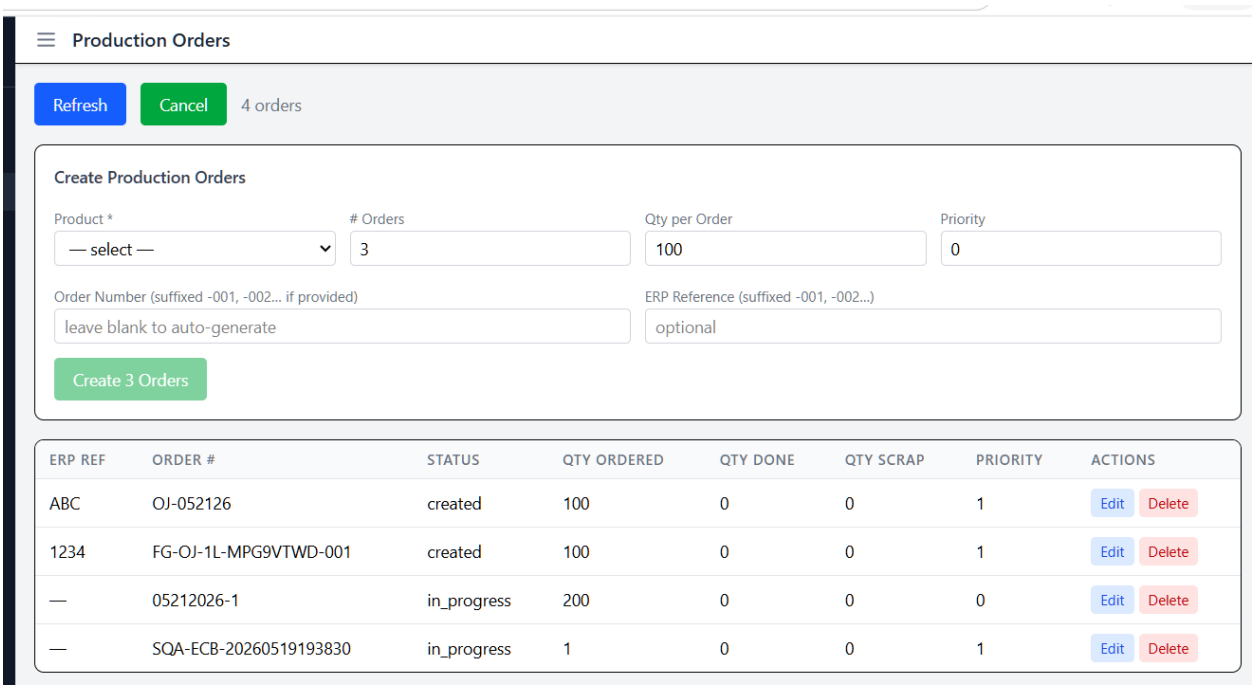
For example, here is the dashboard for Oracle:



Inbound

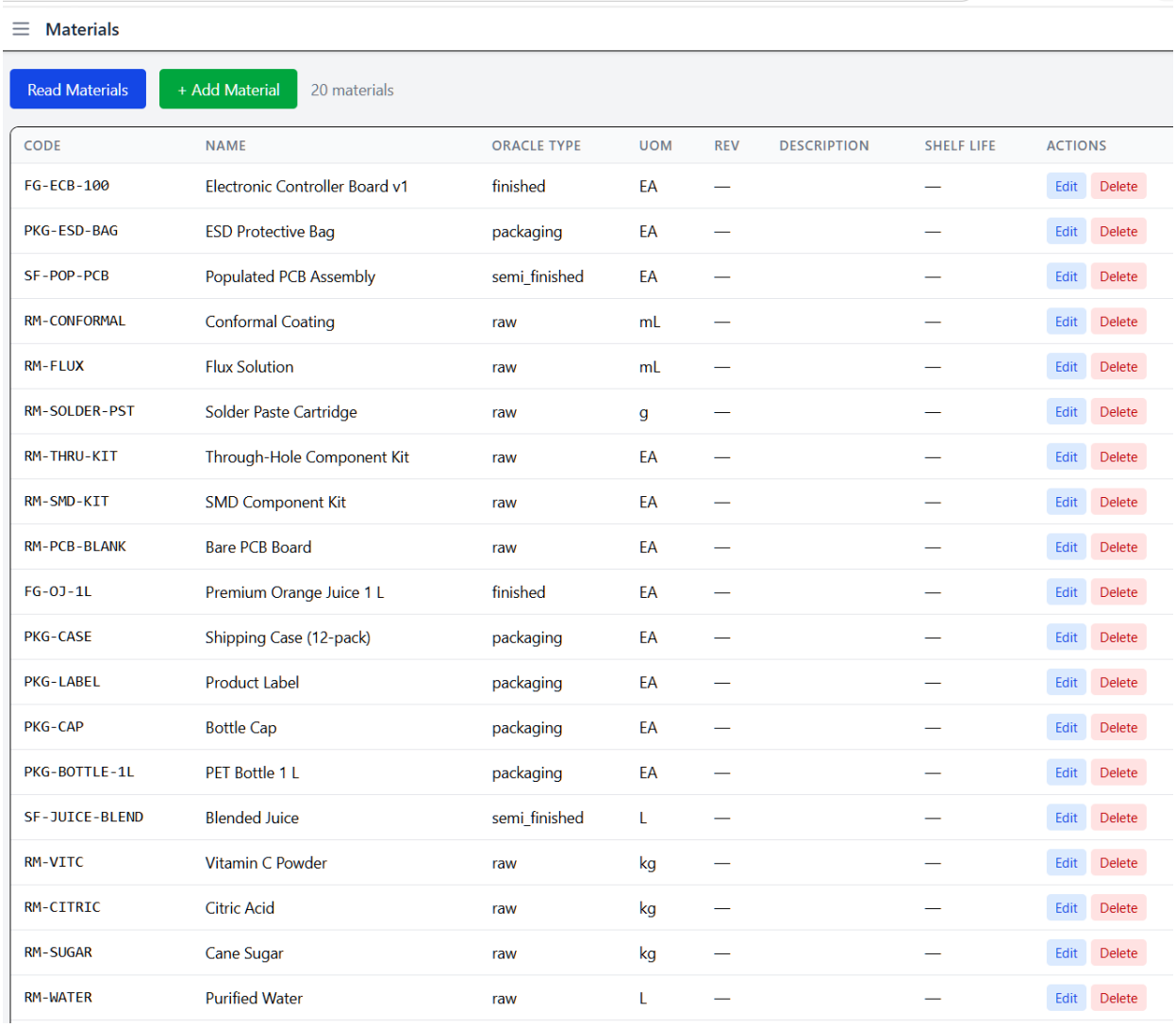
Production Orders

The Production Orders link shows a page with active orders and the capability to create, edit and delete orders:



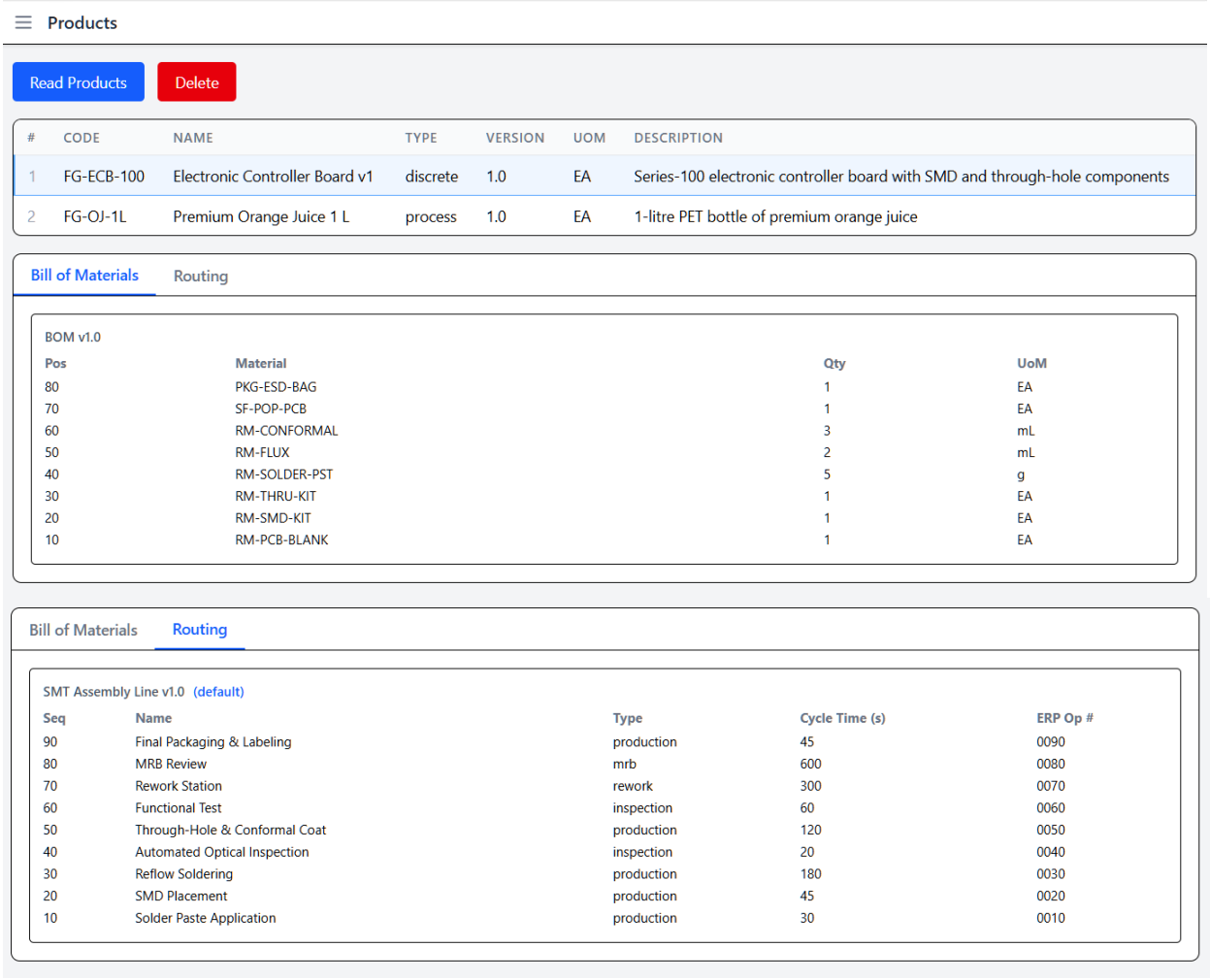
Materials

The Materials link displays materials and allows the user to edit or delete them. For example:



Products

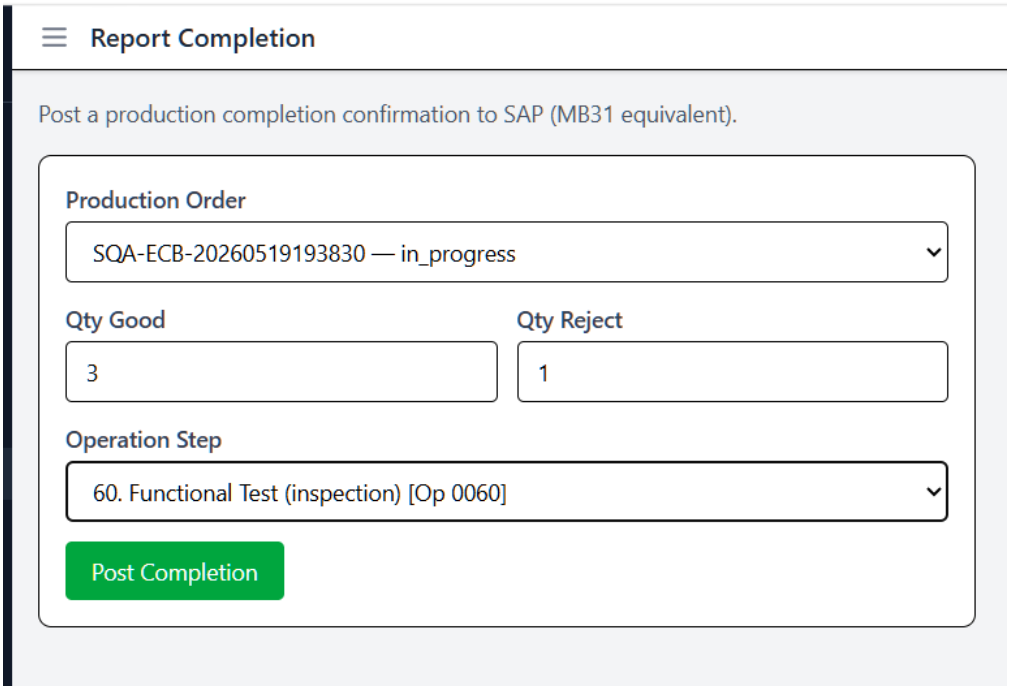
The Products link displays produced material along with its BOM and ERP routing. It allows the user to delete a product. For example:



Outbound

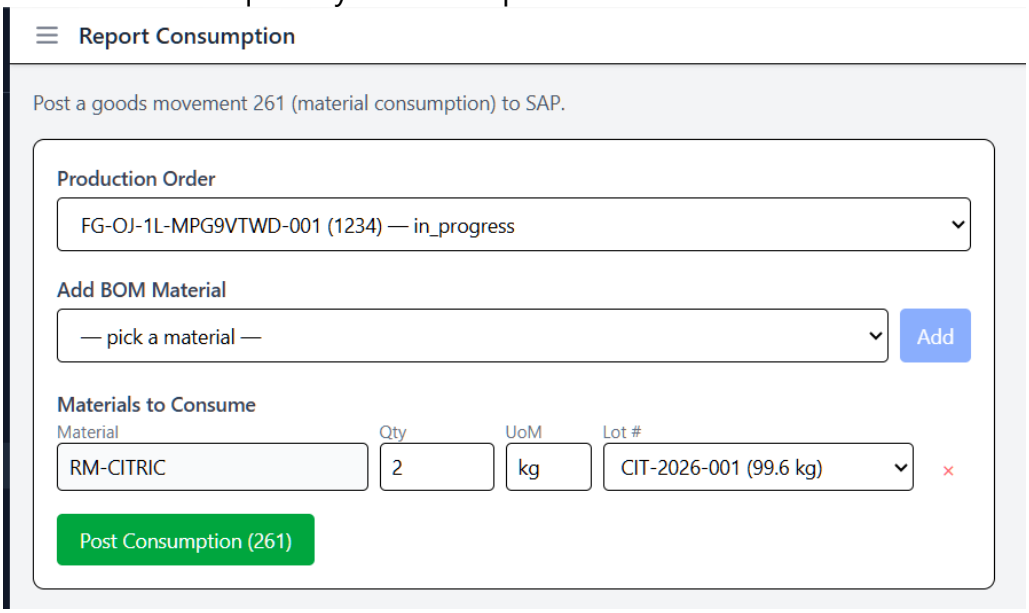
Report Completion

This dialog tests sending a step completion with good and rejected quantities. Fro example:



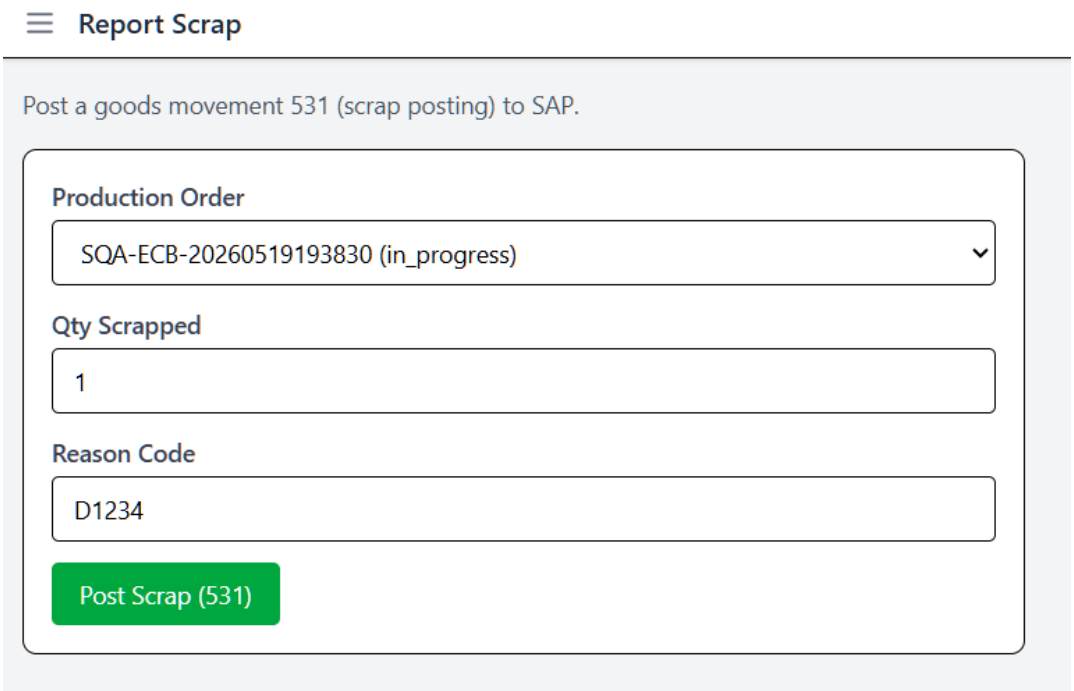
Report Consumption

This dialog tests sending a consumption report with the order, BOM item material, material lot and quantity. For example:



Report Scrap

This dialog tests sending a scrap report with the order number, quantity scrapped and reason. For example:



Confirmations

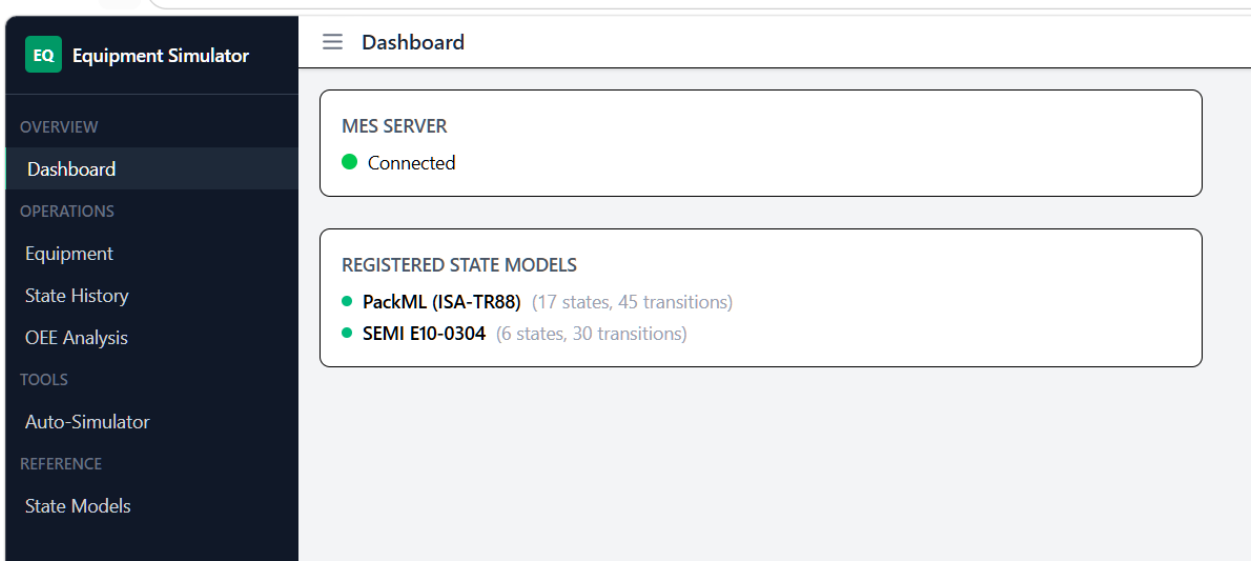
Successful reports to the ERP will be listed in this page.

Equipment Simulator (equipment-sim)

equipment-sim is a simulator used for equipment development and testing. It is installed as an optional plugin.

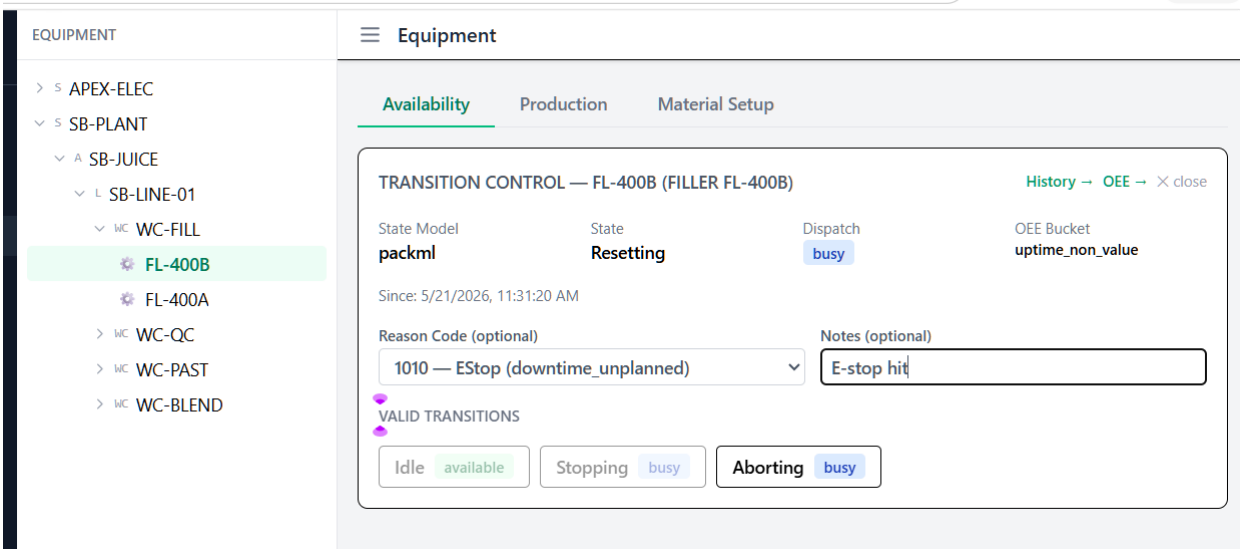
Dashboard

For example, PackML and SEMI E10 are installed state models:

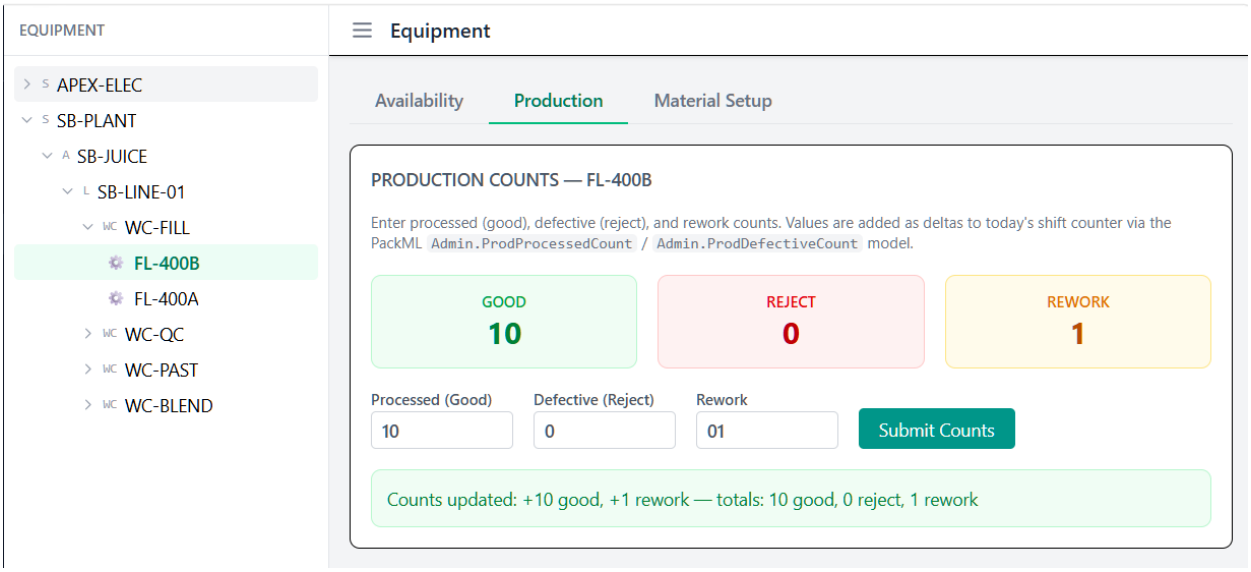


Equipment

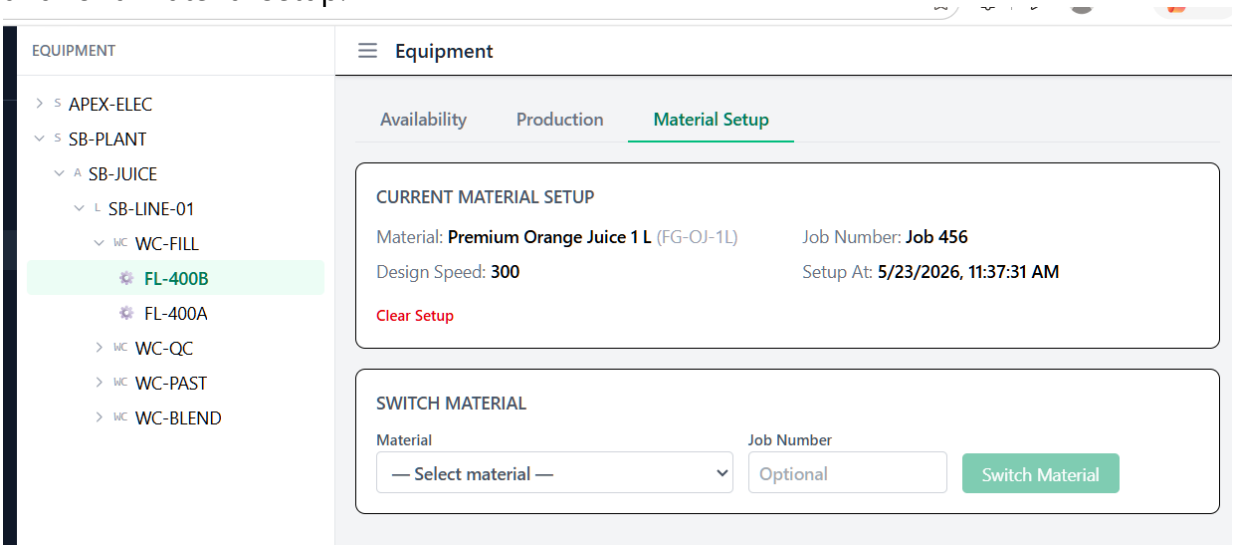
Availability, production and material setup events are entered on this page. For example filler FL-400B has an E-stop event and will be transitioned to the PackML Aborting state by clicking on the Aborting button:



and for production using the PackML counter tags:

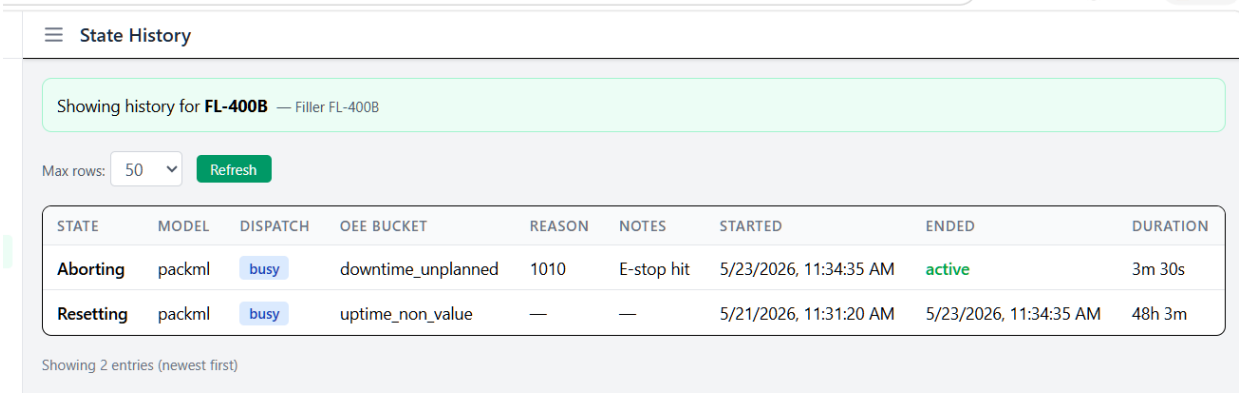


and for a material setup:



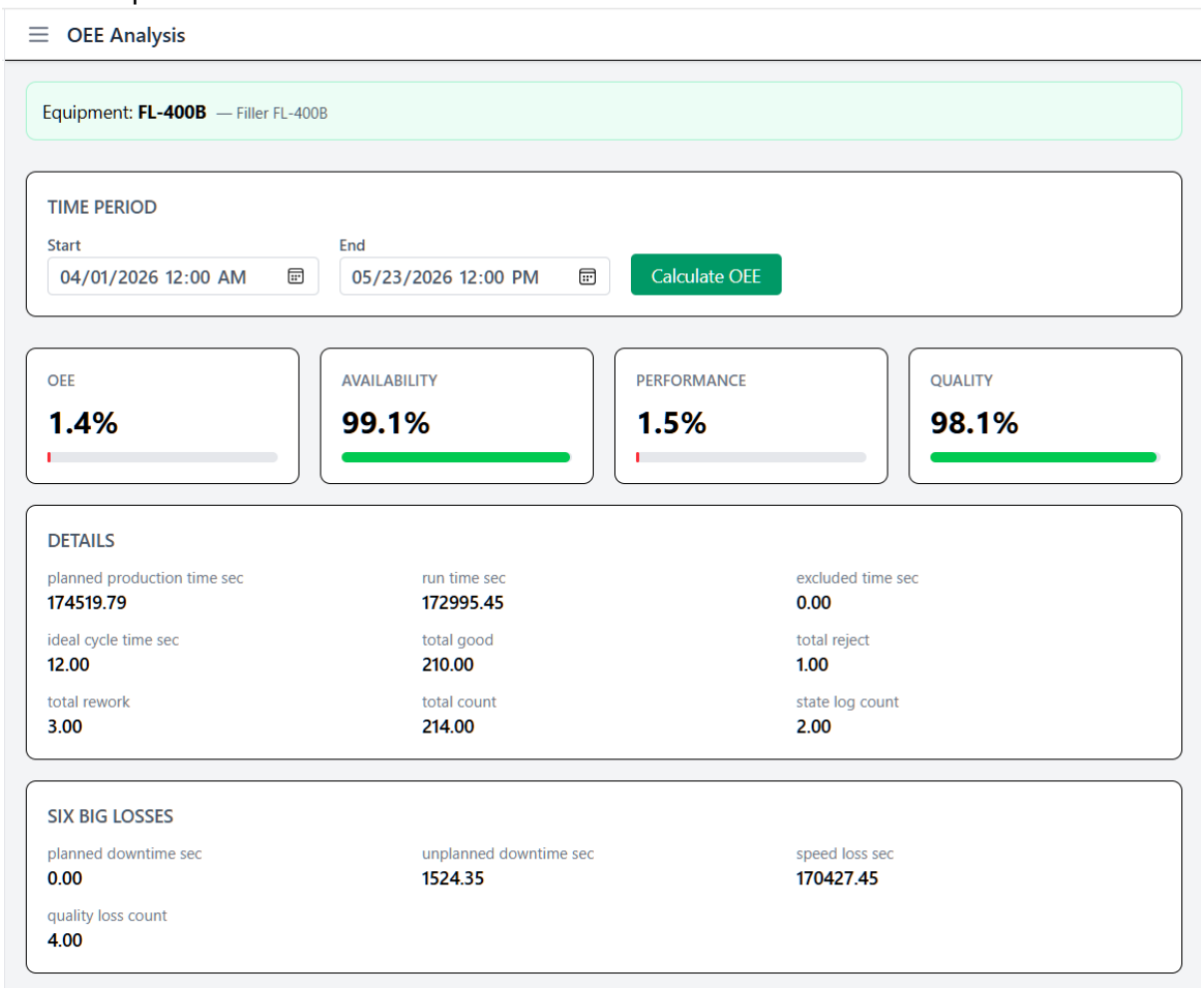
State History

This page displays the history of state transitions. For example:



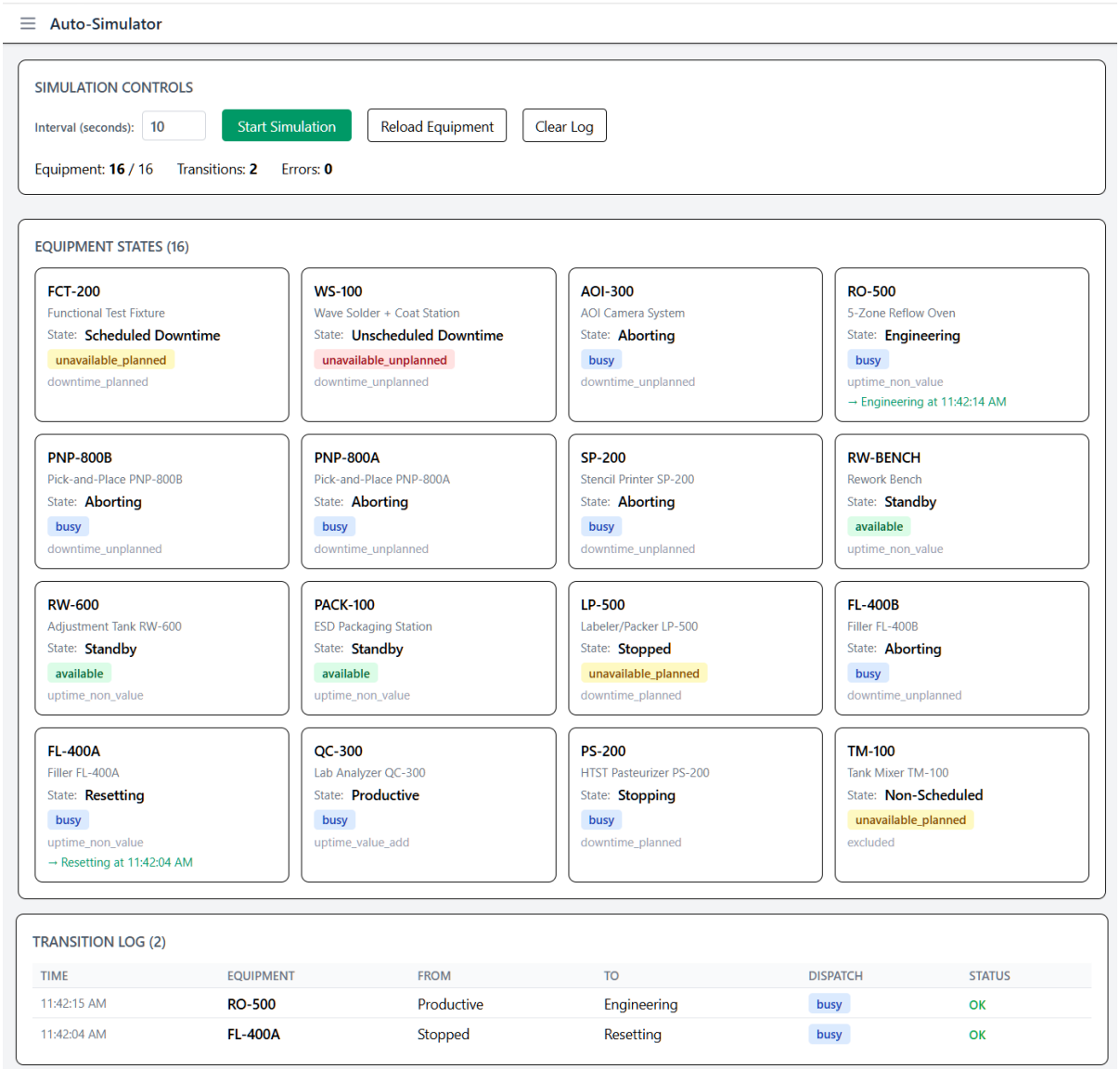
OEE Analysis

This page calculates Overall Equipment Effectiveness (OEE) for a specified time period. For example:



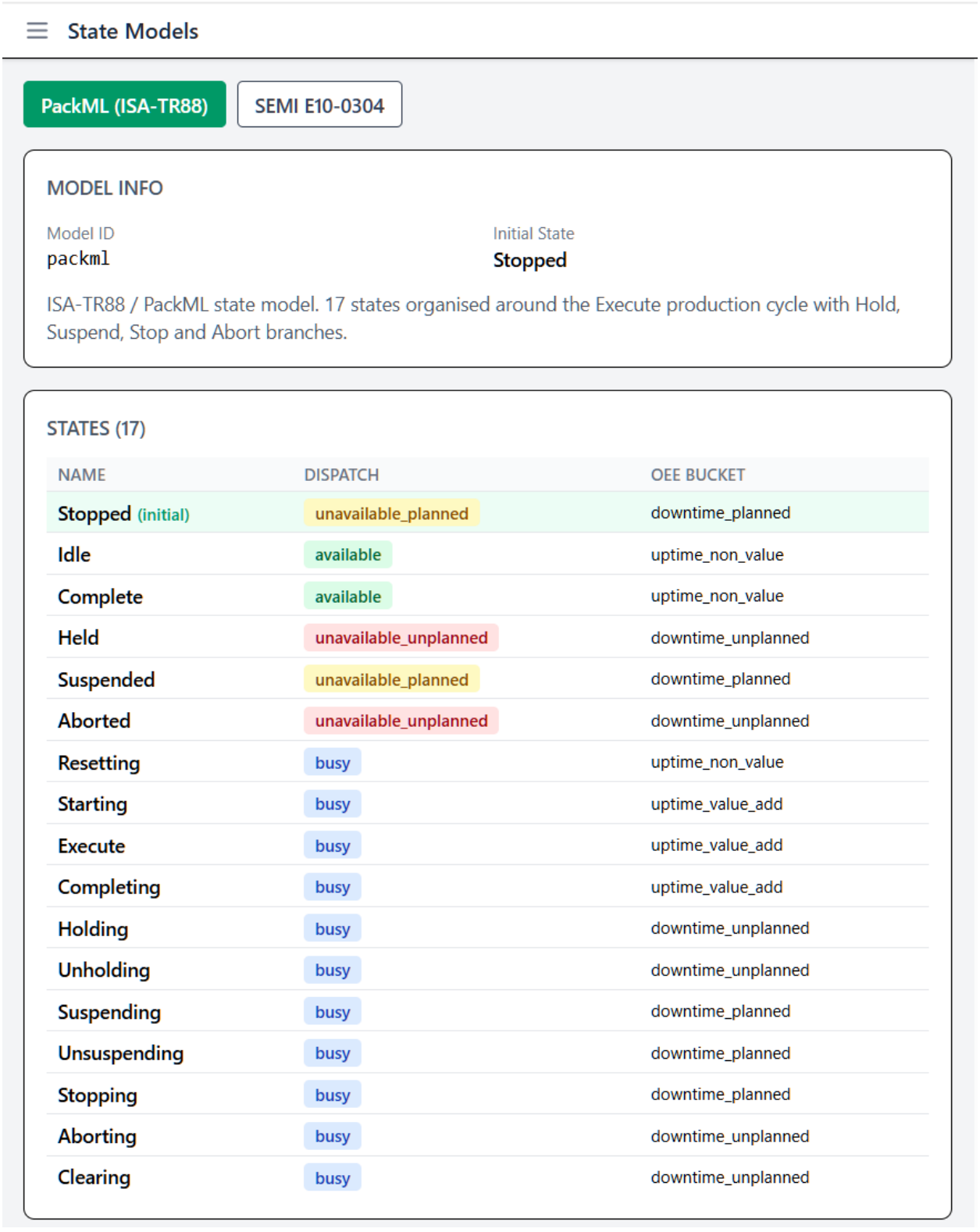
Auto-Simulator

This page simulates equipment state changes using valid transitions according to the assigned state model. For example:



State Models

This page is a reference to the installed state models, e.g. PackML and SEMI E10. For example:


State Models
PackML (ISA-TR88)
SEMI E10-0304
MODEL INFO
Model ID
packml
Initial State
Stopped
ISA-TR88 / PackML state model. 17 states organised around the Execute production cycle with Hold, Suspend, Stop and Abort branches.
STATES (17)
NAME
DISPATCH
OEE BUCKET
Stopped (initial)
unavailable_planned
downtime_planned
Idle
available
uptime_non_value
Complete
available
uptime_non_value
Held
unavailable_unplanned
downtime_unplanned
Suspended
unavailable_planned
downtime_planned
Aborted
unavailable_unplanned
downtime_unplanned
Resetting
busy
uptime_non_value
Starting
busy
uptime_value_add
Execute
busy
uptime_value_add
Completing
busy
uptime_value_add
Holding
busy
downtime_unplanned
Unholding
busy
downtime_unplanned
Suspending
busy
downtime_planned
Unsuspending
busy
downtime_planned
Stopping
busy
downtime_planned
Aborting
busy
downtime_unplanned
Clearing
busy
downtime_unplanned

TRANSITIONS (45)

FROM	TO	TRIGGER
Stopped	Resetting	—
Resetting	Idle	—
Idle	Starting	—
Starting	Execute	—
Execute	Completing	—
Completing	Complete	—
Complete	Resetting	—
Execute	Holding	—

Software Quality Assurance

REST API Unit Tests

A the units tests can be run by executing pytest:

```
cd server
pytest tests/unit
```

There are scoping options which can be found by prompting the AI for them.

User Interface Tests

Automated user interface CRUD (Create, Read, Update and Delete) testing for design time client (dt-client) editors were implemented. In the SQA folder, the script is called run-dt-audit.ps1/sh. The scope argument defines the scope of the test, for example:

```
./run-rt-audit.ps1 -Scope all
```

Executing the script without any arguments prints a usage/help.

Documentation

REST API

The REST API is documented in three ways:.

Swagger UI	http://<mes host:port>/api/v1/docs
ReDoc	http://<mes host:port>/api/v1/redoc
OpenAPI JSON	http://<mes host:port>api/v1/openapi.json

Swagger UI http://localhost:8082/api/v1/docs
ReDoc http://localhost:8082/api/v1/redoc
OpenAPI JSON http://localhost:8082/api/v1/openapi.json

Architecture

The docs\ARCHITECTURE.md document contains the technical details of the system architecture. It was created and maintained by an AI agent. Before making changes with an AI agent, have the AI read this document and then keep it up-to-date as the project progresses.

As changes are made, have the AI maintain a PROJECT_STATE.json document that is created at the beginning of a project with detailed task instructions, then kept current as the project progresses. Along with this document, have the AI maintain a SESSION_LOG.md which preserves what happened in the current session. When a new session is started, instruct the AI to read both of these documents in order to continue where you left off.

Enhancements for Forward Deployed Engineers

Using your preferred IDE and AI model, implement one or more of these items:

- Write an operator app in another technology stack that calls the REST API, for example a native iOS or Android application.
- Implement an equipment preventive maintenance and calibration plugin. Make unavailable equipment prevent dispatch.
- Localize text, dates and times
- Implement GitHub Continuous Integration (CI) workflow
- Implement database index and view creation scripts
- Implement a realistic custom runtime dispatch algorithm
- Implement an order management workflow prior to order release
- Provide a realistic implementation of the file drop equipment interface
- Implement NonConformance Reporting (NCR) and Corrective Action/Preventive Action (CAPA) resolution tracking
- Implement operator clockon/off, labor booking against a job/step, qualifications or certifications, and qualification checks before execution.
- Implement a monitoring dashboard to show Key Performance Indicators (KPIs) such as Overall Equipment Effectiveness (OEE) and display notifications from the event bus.
- Implement local and OIDC authentication for runtime client, ERP and equipment

simulators

- Implement client authorization access based on the user authentication
- Implement a finite schedule that allocates orders against constrained equipment/labor, manages sequencing and due dates, and replans when execution deviates.
- Start from a suspect supplier inventory item and identify affected WIP and finished goods, then support containment and recall reporting.
- Implement electronic-signature, controlled change/revision approval, and tamper-evident audit workflow.